



香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems

Lecture 26

Modern computer systems: file system

Prof. Yunming XIAO
School of Data Science

Table of contents

- A file system is no longer tied to one storage device
 - **Strata**: A Cross Media File System, **SOSP'17**
- The file system as a distributed service:
 - **Assise**: Performance and Availability via Client local NVM in a Distributed File System, **OSDI'20**
- Persistence is now also about recovery speed, not just durability
 - **Gemini**: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints, **SOSP'23**

Table of contents

- A file system is no longer tied to one storage device
 - **Strata**: A Cross Media File System, **SOSP'17**
- The file system as a distributed service:
 - **Assise**: Performance and Availability via Client local NVM in a Distributed File System, **OSDI'20**
- Persistence is now also about recovery speed, not just durability
 - **Gemini**: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints, **SOSP'23**

Strata: A Cross Media File System

Youngjin Kwon, Henrique Fingler, Tyler Hunt,
Simon Peter, Emmett Witchel, Thomas Anderson



The University of Texas at Austin



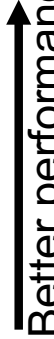
Let's build a fast server

NoSQL store, Database, File server, Mail server ...

Requirements

- Small updates (1 Kbytes) dominate
- Dataset scales up to 10 TB
- Updates must be crash consistent

Storage diversification

	Latency	\$/GB	
DRAM	100 ns	8.6	 Better performance  Higher capacity
NVM (soon)	300 ns	4.0	
SSD	10 us	0.25	
HDD	10 ms	0.02	

Storage diversification

Byte-addressable: cache-line granularity IO

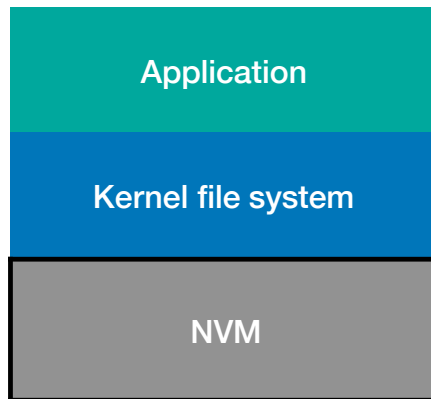
	Latency	\$/GB	
DRAM	100 ns	8.6	Better performance Higher capacity
NVM (soon)	300 ns	4.0	
SSD	10 us	0.25	
HDD	10 ms	0.02	

Storage diversification

Byte-addressable: cache-line granularity IO			Better performance Higher capacity
	Latency	\$/GB	
DRAM	100 ns	8.6	
NVM (soon)	300 ns	4.0	
SSD	10 us	0.25	
HDD	10 ms	0.02	
Large erasure blocks need to be sequentially written Random writes: 5~6x slowdown due to GC [FAST'15]			

A fast server on today's file system

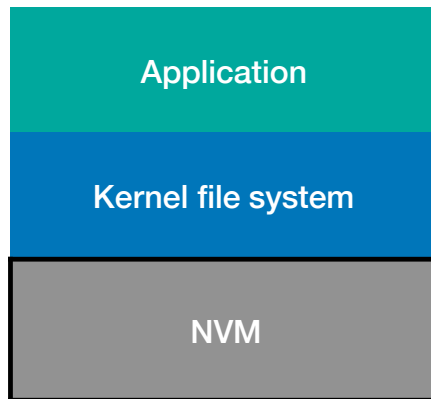
- ➔
- **Small updates (1 Kbytes) dominate**
 - Dataset scales up to 10TB
 - Updates must be crash consistent



Kernel file system:
NOVA [FAST 16, SOSP 17]

A fast server on today's file system

- ➔
- **Small updates (1 Kbytes) dominate**
 - Dataset scales up to 10TB
 - Updates must be crash consistent

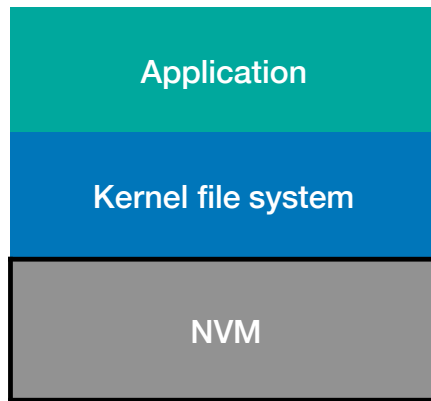


Small, random IO is slow!

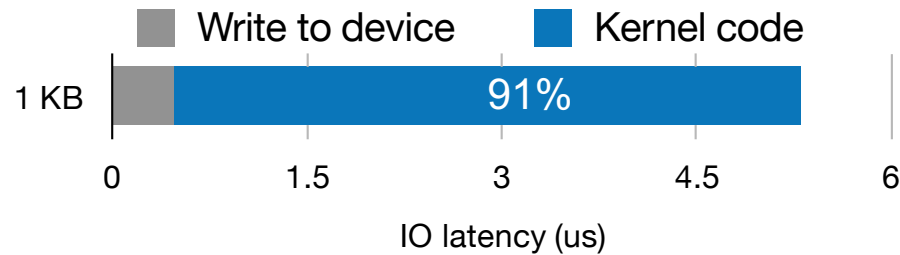
Kernel file system:
NOVA [FAST 16, SOSP 17]

A fast server on today's file system

- ➔
- **Small updates (1 Kbytes) dominate**
 - Dataset scales up to 10TB
 - Updates must be crash consistent



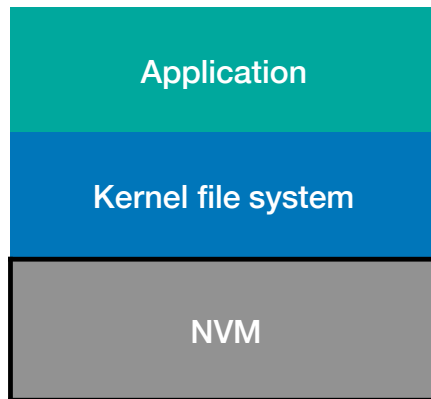
Small, random IO is slow!



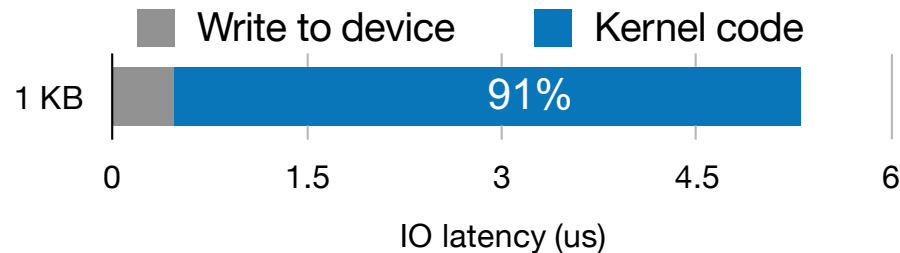
Kernel file system:
NOVA [FAST 16, SOSP 17]

A fast server on today's file system

- ➔
- **Small updates (1 Kbytes) dominate**
 - Dataset scales up to 10TB
 - Updates must be crash consistent



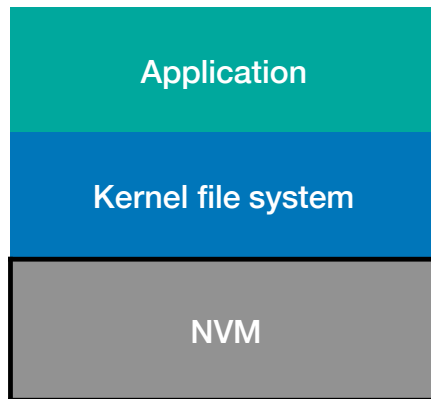
Small, random IO is slow!



NVM is so fast that kernel is the bottleneck

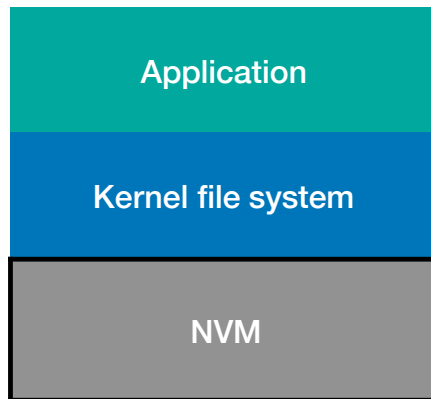
A fast server on today's file system

- Small updates (1 Kbytes) dominate
- ➔ • **Dataset scales up to 10TB**
- Updates must be crash consistent



A fast server on today's file system

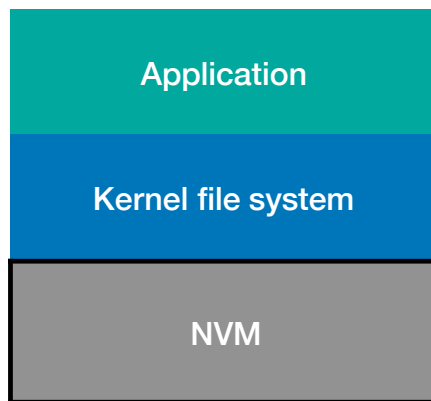
- Small updates (1 Kbytes) dominate
- • **Dataset scales up to 10TB**
- Updates must be crash consistent



**Need huge capacity, but
NVM alone is too expensive!
(\$40K for 10TB)**

A fast server on today's file system

- Small updates (1 Kbytes) dominate
- • **Dataset scales up to 10TB**
- Updates must be crash consistent

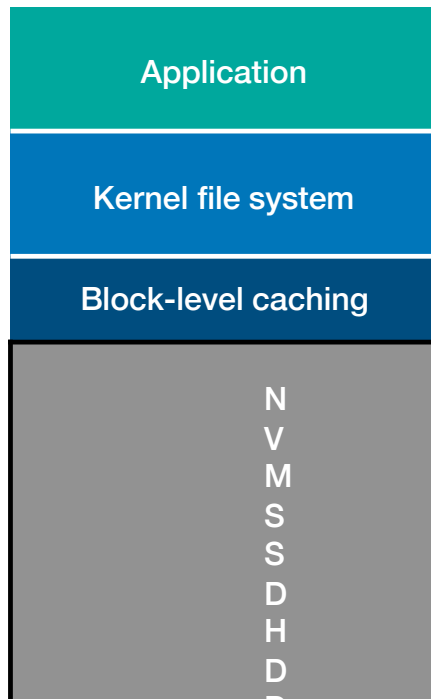


**Need huge capacity, but
NVM alone is too expensive!
(\$40K for 10TB)**

**For low-cost capacity with high performance,
must leverage multiple device types**

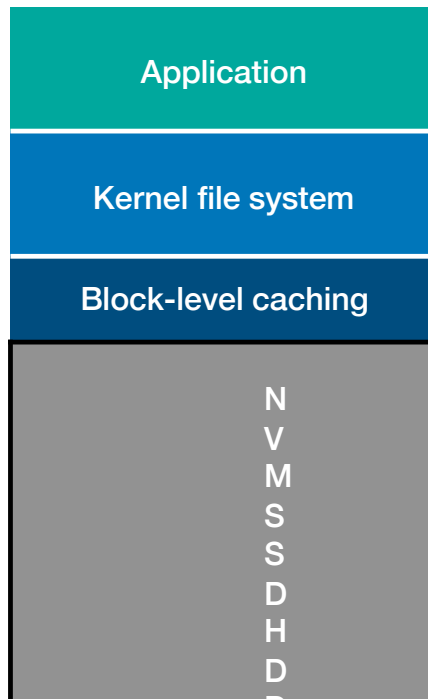
A fast server on today's file system

- Small updates (1 Kbytes) dominate
- ➔ • **Dataset scales up to 10TB**
- Updates must be crash consistent



A fast server on today's file system

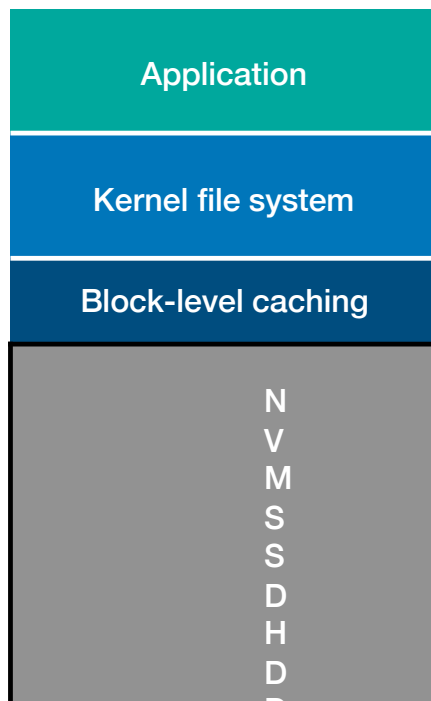
- Small updates (1 Kbytes) dominate
- ➔ • **Dataset scales up to 10TB**
- Updates must be crash consistent



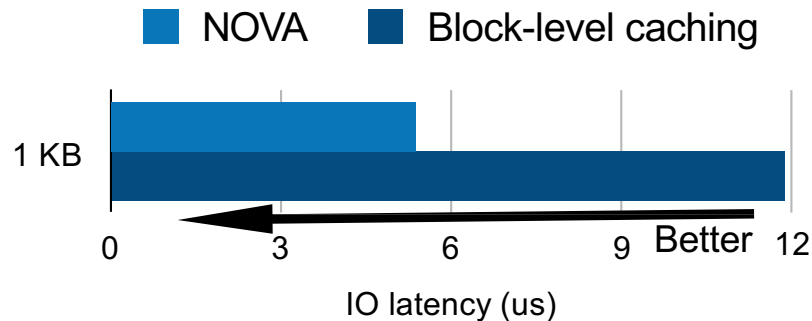
- **Block-level caching manages data in blocks, but NVM is byte-addressable**
- **Extra level of indirection**

A fast server on today's file system

- Small updates (1 Kbytes) dominate
- ➔ • **Dataset scales up to 10TB**
- Updates must be crash consistent

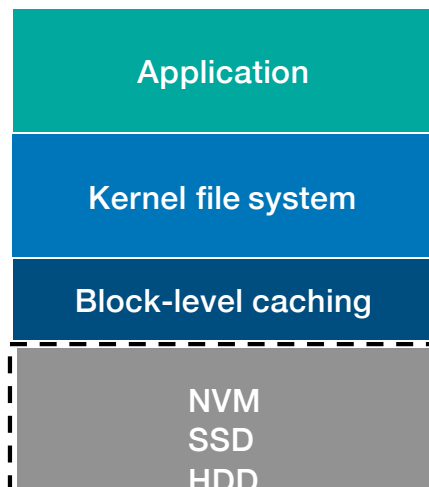


- **Block-level caching manages data in blocks, but NVM is byte-addressable**
- **Extra level of indirection**

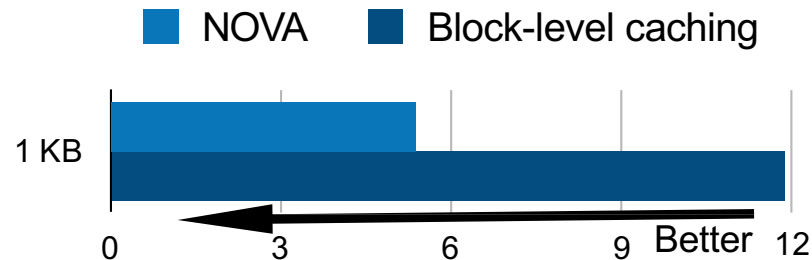


A fast server on today's file system

- Small updates (1 Kbytes) dominate
- ➔ • **Dataset scales up to 10TB**
- Updates must be crash consistent



- **Block-level caching manages data in blocks, but NVM is byte-addressable**
- **Extra level of indirection**



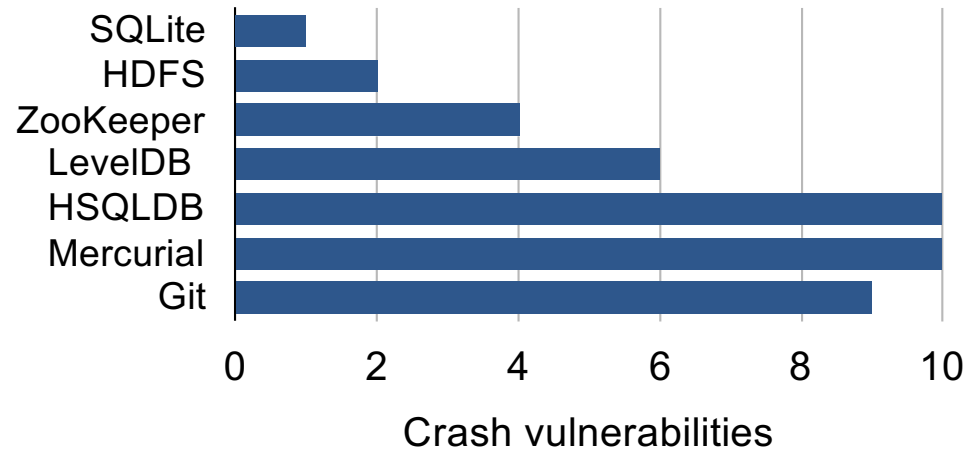
Block-level caching is too slow

A fast server on today's file system

- Small updates (1 Kbytes) dominate
- Dataset scales up to 10TB
- ➔ • **Updates must be crash consistent**

A fast server on today's file system

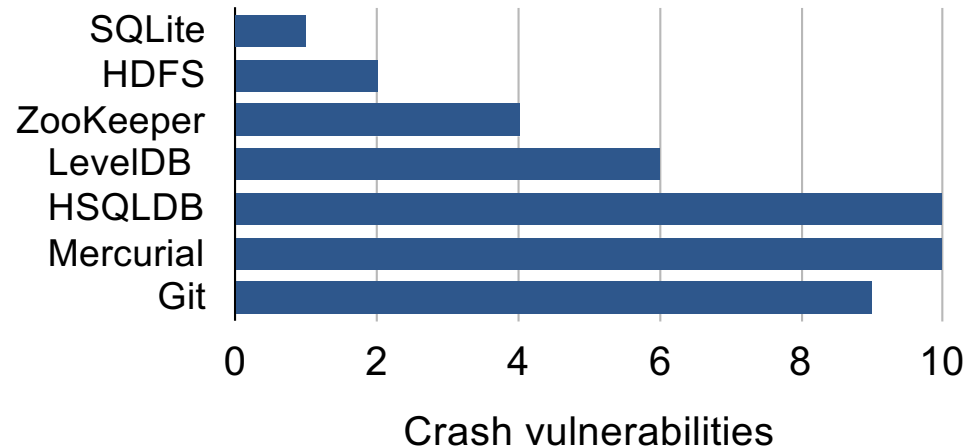
- Small updates (1 Kbytes) dominate
- Dataset scales up to 10TB
- ➔ • **Updates must be crash consistent**



Pillai et al., OSDI 2014

A fast server on today's file system

- Small updates (1 Kbytes) dominate
- Dataset scales up to 10TB
- ➔ • **Updates must be crash consistent**



Applications struggle for crash consistency

Problems in today's file systems

- Kernel mediates every operation
NVM is so fast that kernel is the bottleneck
- Tied to a single type of device
**For low-cost capacity with high performance,
must leverage multiple device types
NVM (soon), SSD, HDD**
- Aggressive caching in DRAM,
write to device only when you must (fsync)
Applications struggle for crash consistency

Strata:

A Cross Media File System

Performance: especially small, random IO

- Fast user-level device access

Low-cost capacity: leverage NVM, SSD & HDD

- Transparent data migration across different storage media
- Efficiently handle device IO properties

Simplicity: intuitive crash consistency model

- In-order, synchronous IO
- No fsync() required

Strata: main design principle

Log operations to NVM at user-level

Digest and migrate data in kernel

Strata: main design principle

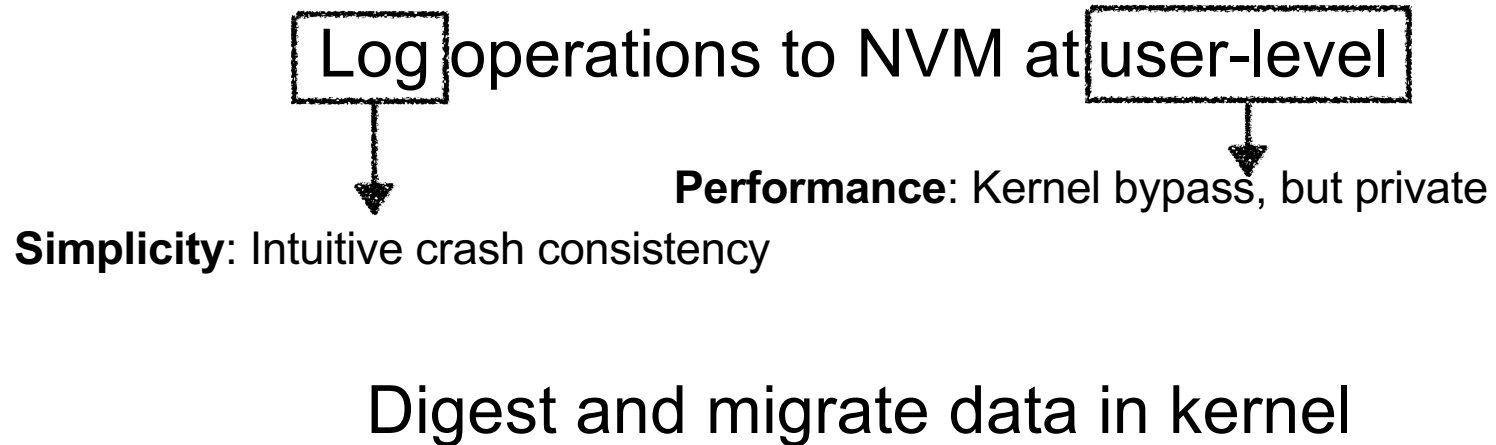
Log operations to NVM at user-level



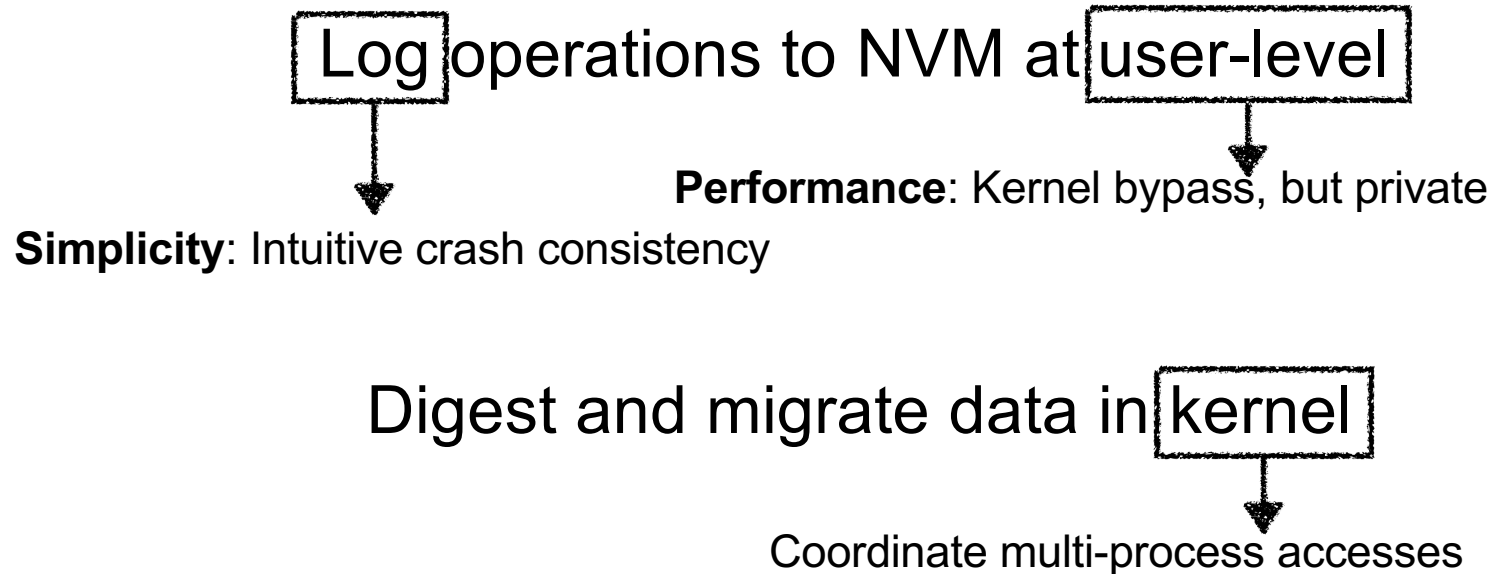
Performance: Kernel bypass, but private

Digest and migrate data in kernel

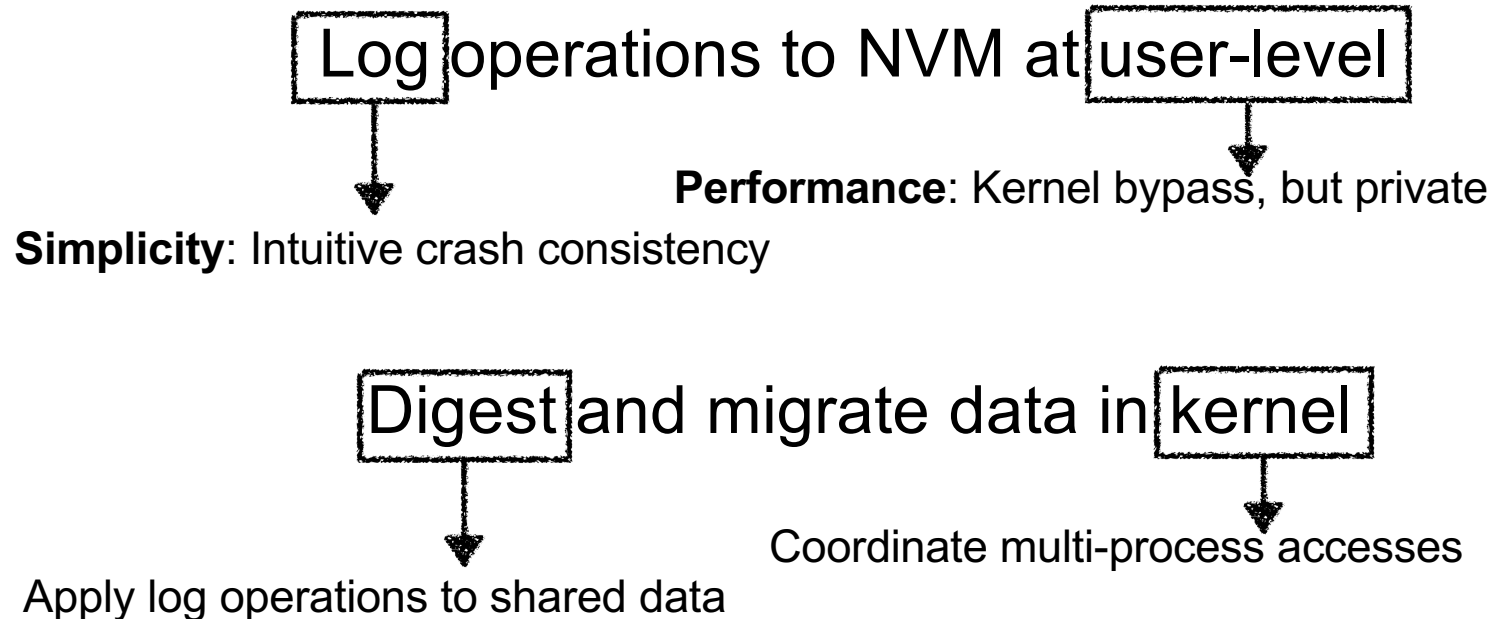
Strata: main design principle



Strata: main design principle



Strata: main design principle



Strata: main design principle

LibFS

Log operations to NVM at user-level



Performance: Kernel bypass, but private

Simplicity: Intuitive crash consistency

KernelFS

Digest and migrate data in kernel



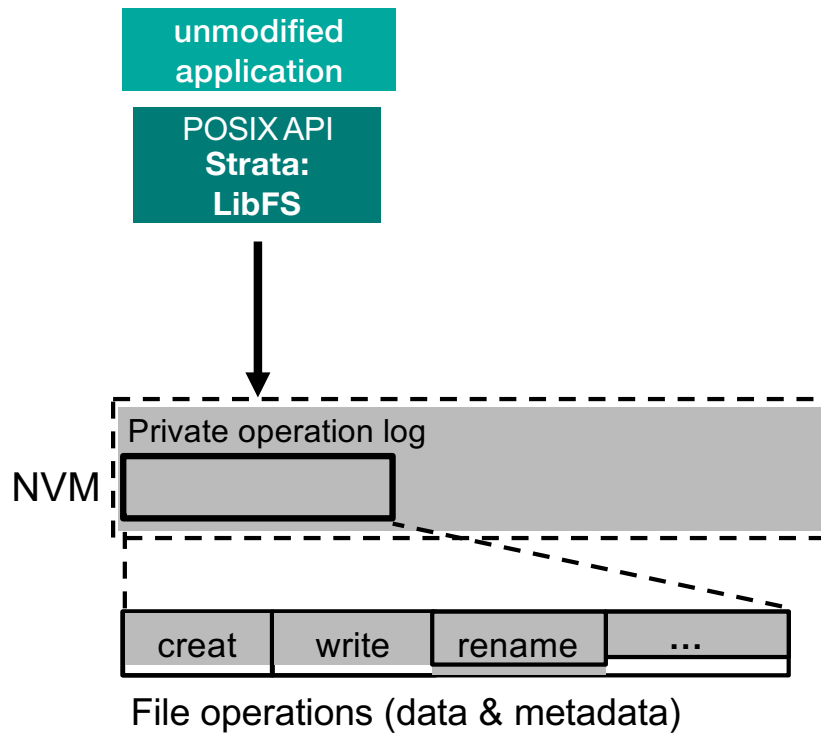
Coordinate multi-process accesses

Apply log operations to shared data

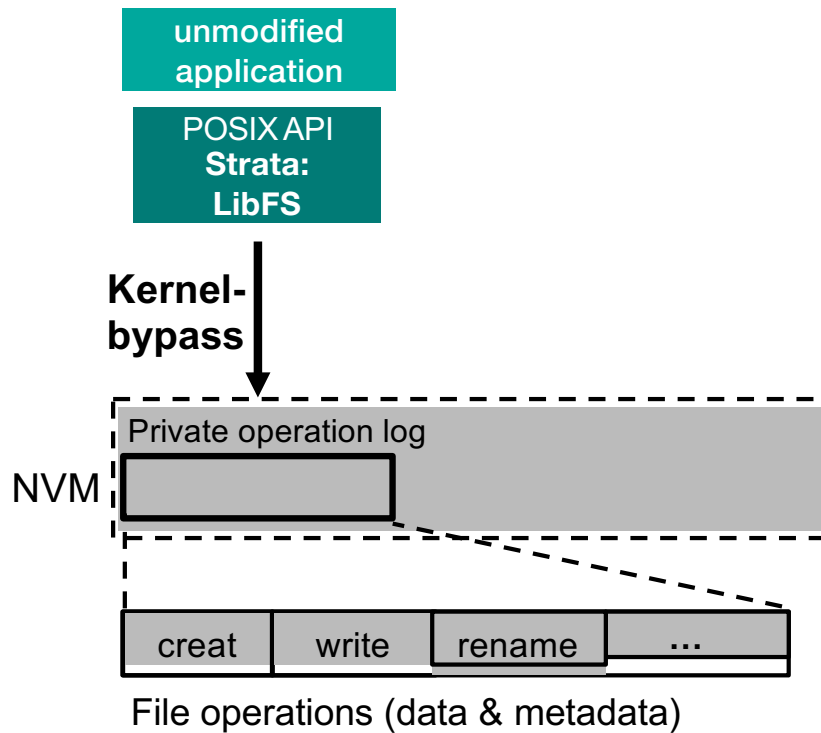
Outline

- **LibFS:** Log operations to NVM at user-level
 - Fast user-level access
 - In-order, synchronous IO
- **KernelFS:** Digest and migrate data in kernel
 - Asynchronous digest
 - Transparent data migration
 - Shared file access
- **Evaluation**

Log operations to NVM at user-level

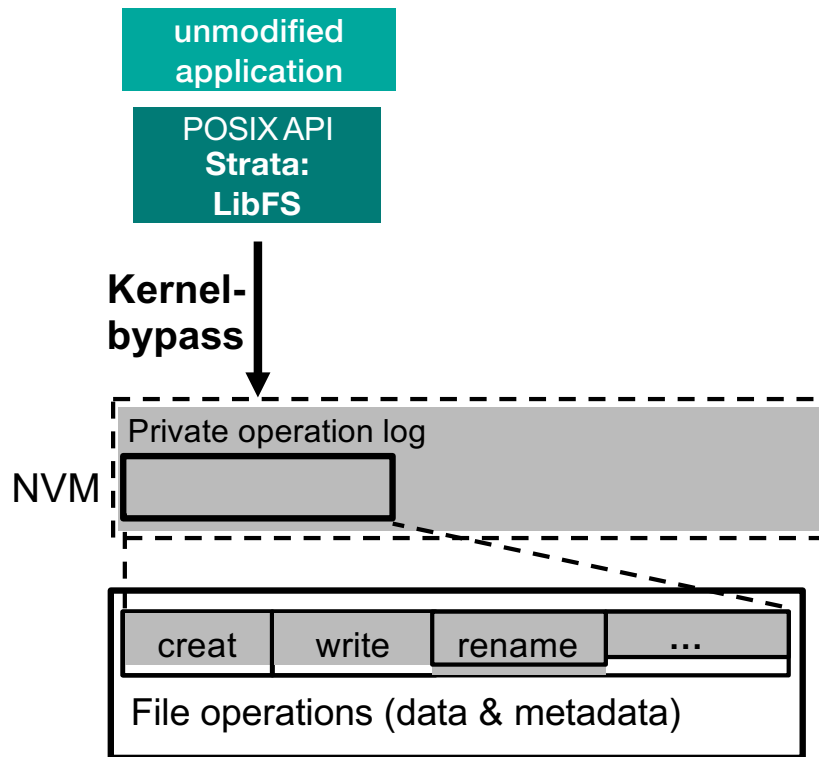


Log operations to NVM at user-level



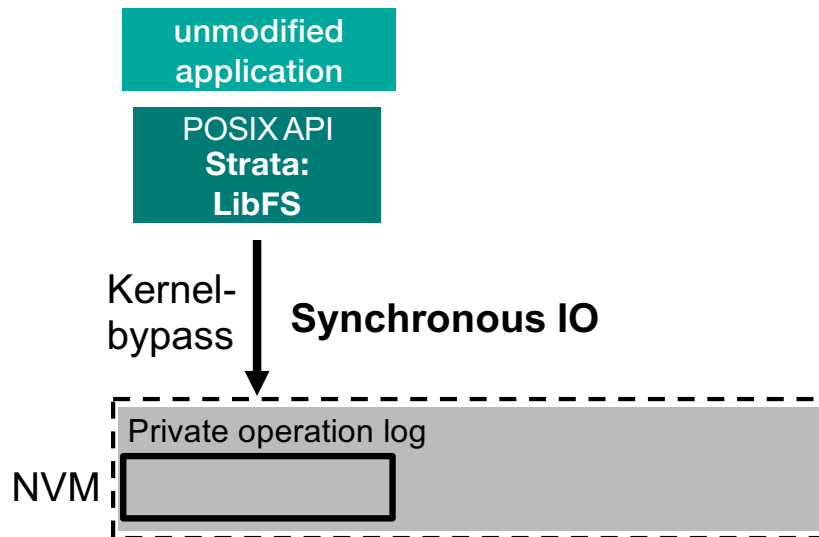
- Fast writes
 - Directly access fast NVM
 - Sequentially append data
 - Cache-line granularity
 - Blind writes

Log operations to NVM at user-level

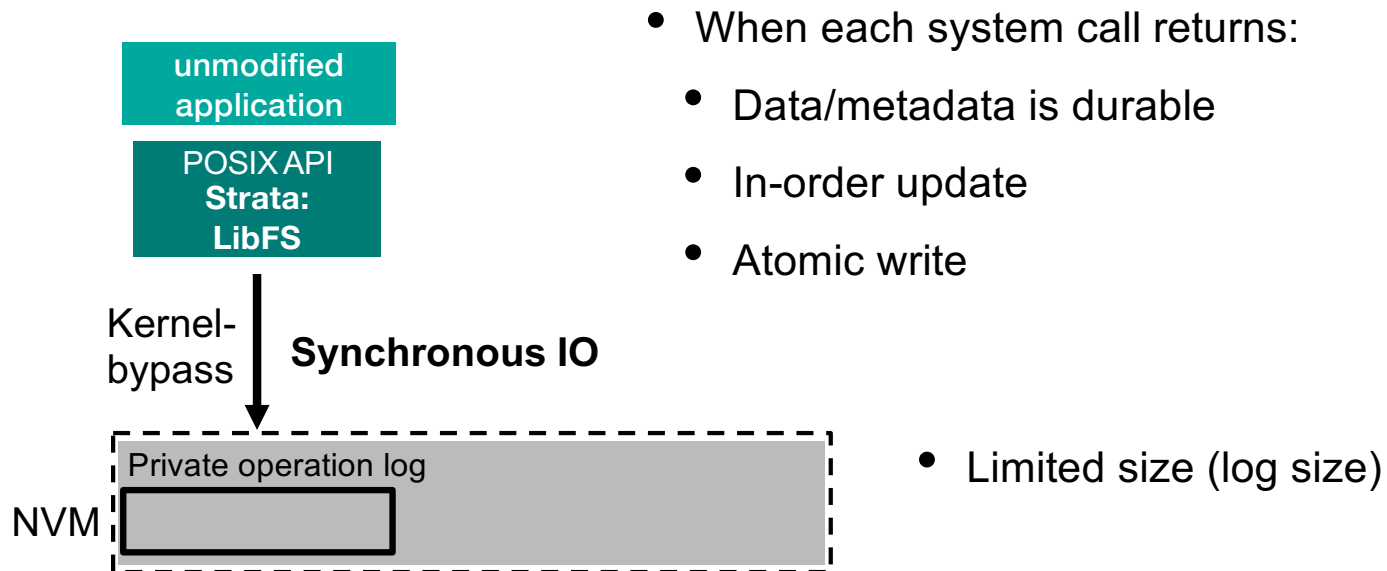


- Fast writes
 - Directly access fast NVM
 - Sequentially append data
 - Cache-line granularity
 - Blind writes
- Crash consistency
 - On crash, kernel replays log

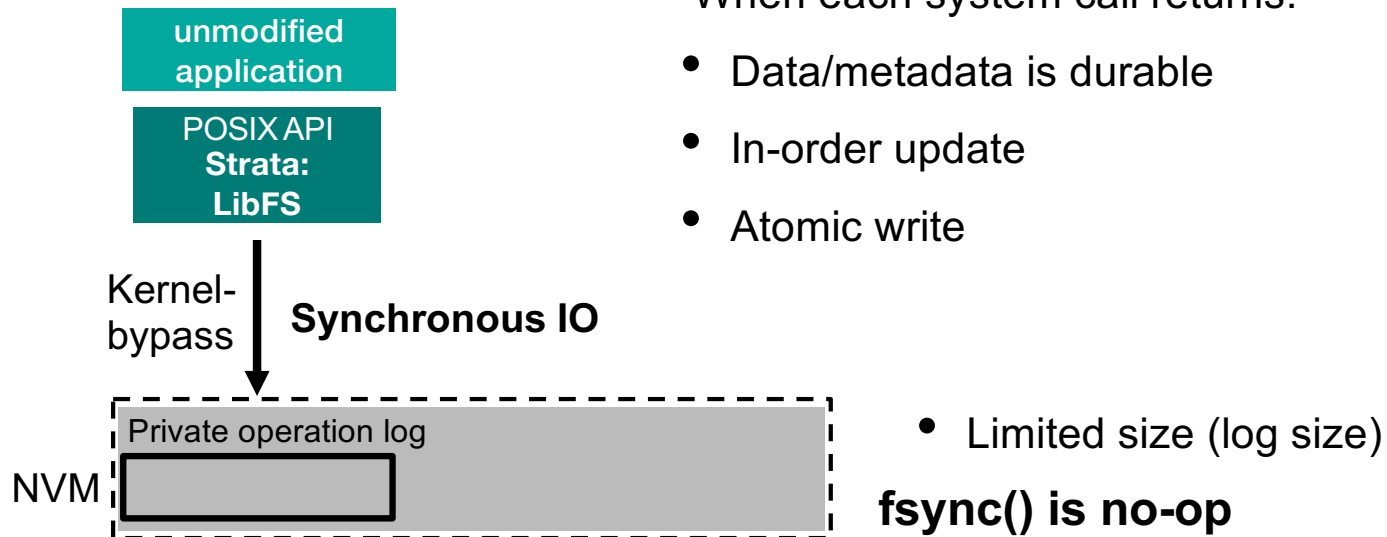
Intuitive crash consistency



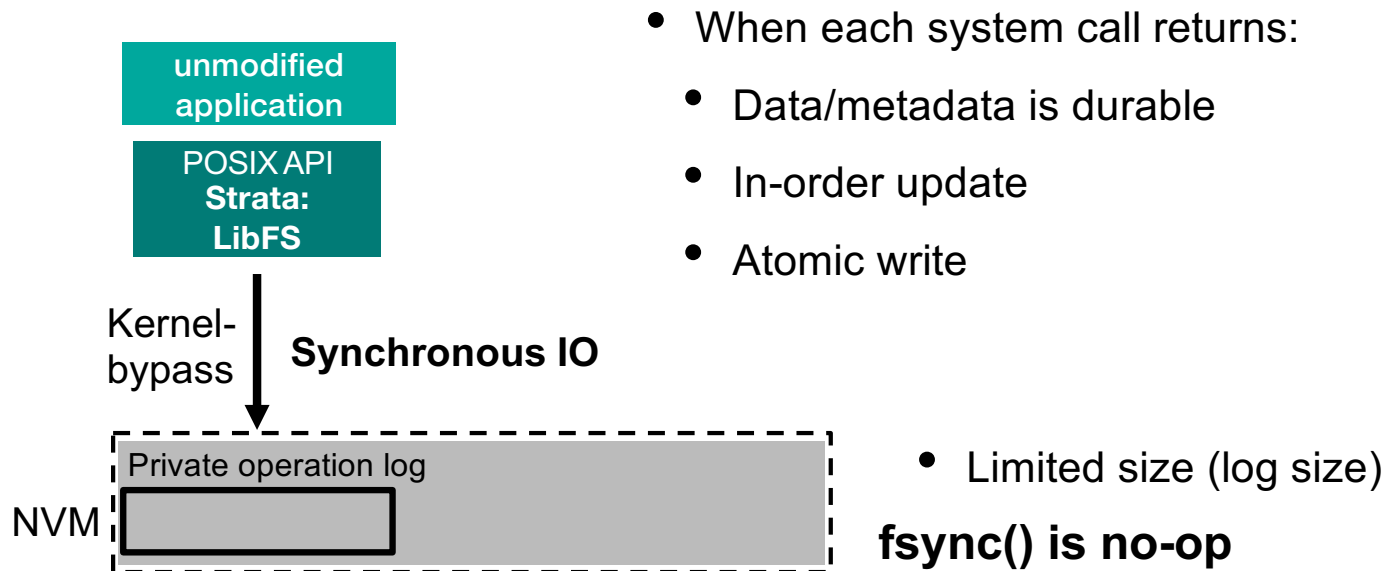
Intuitive crash consistency



Intuitive crash consistency



Intuitive crash consistency

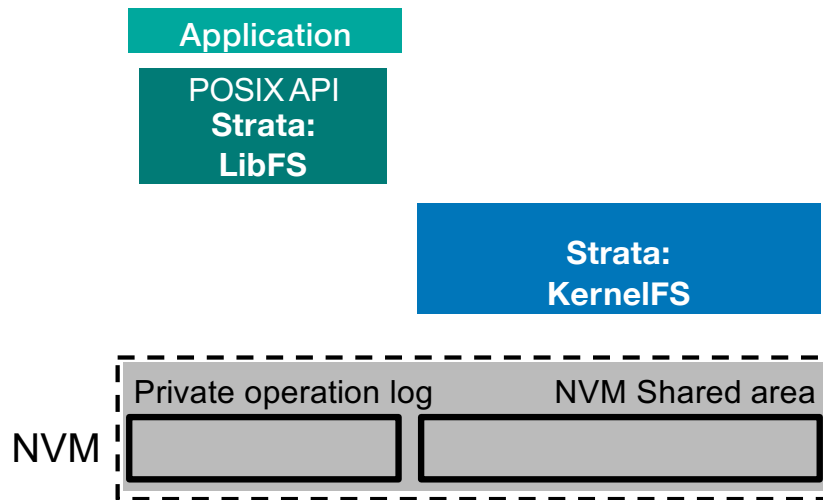


Fast synchronous IO: NVM and kernel-bypass

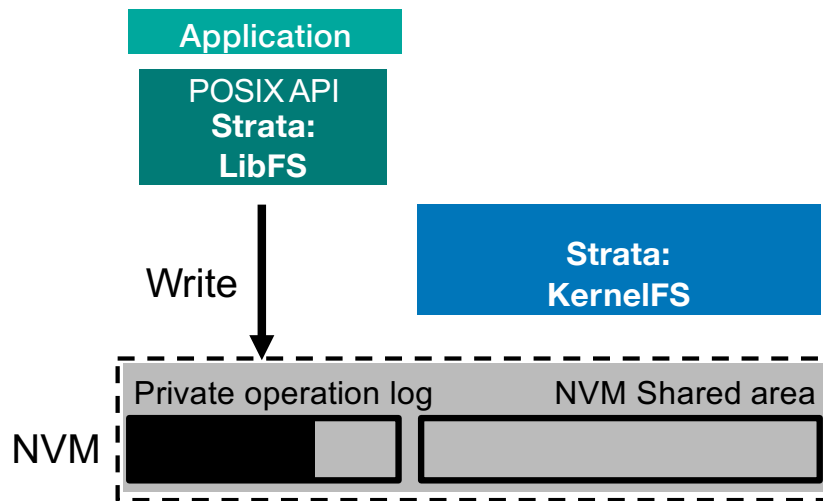
Outline

- **LibFS**: Log operations to NVM at user-level
 - Fast user-level access
 - In-order, synchronous IO
- **KernelFS**: Digest and migrate data in kernel
 - Asynchronous digest
 - Transparent data migration
 - Shared file access
- **Evaluation**

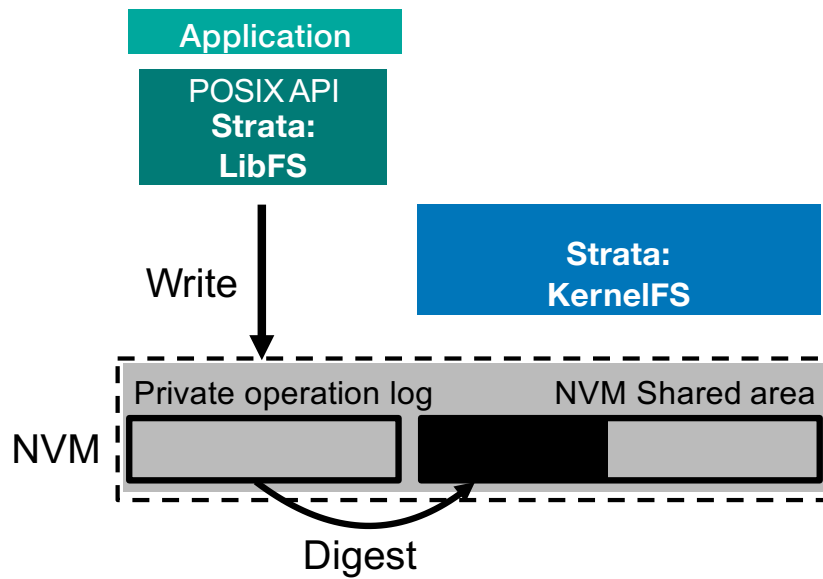
Digest data in kernel



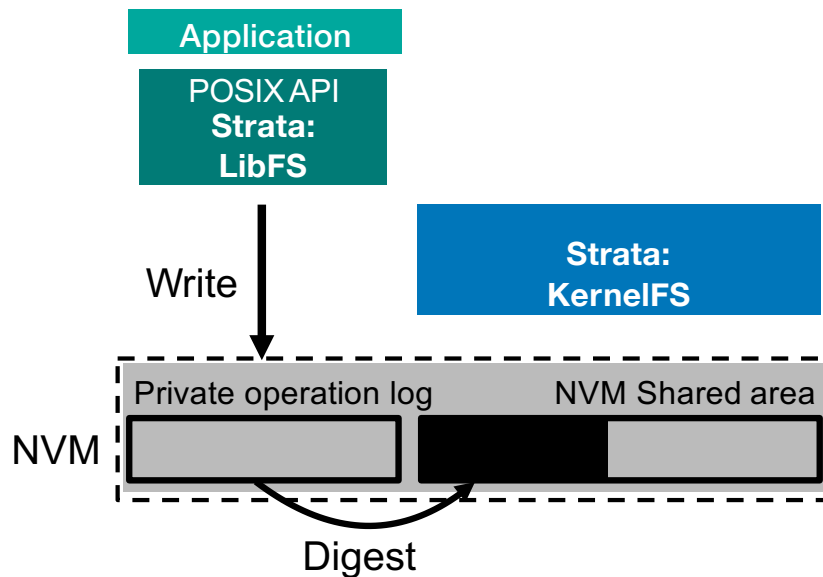
Digest data in kernel



Digest data in kernel



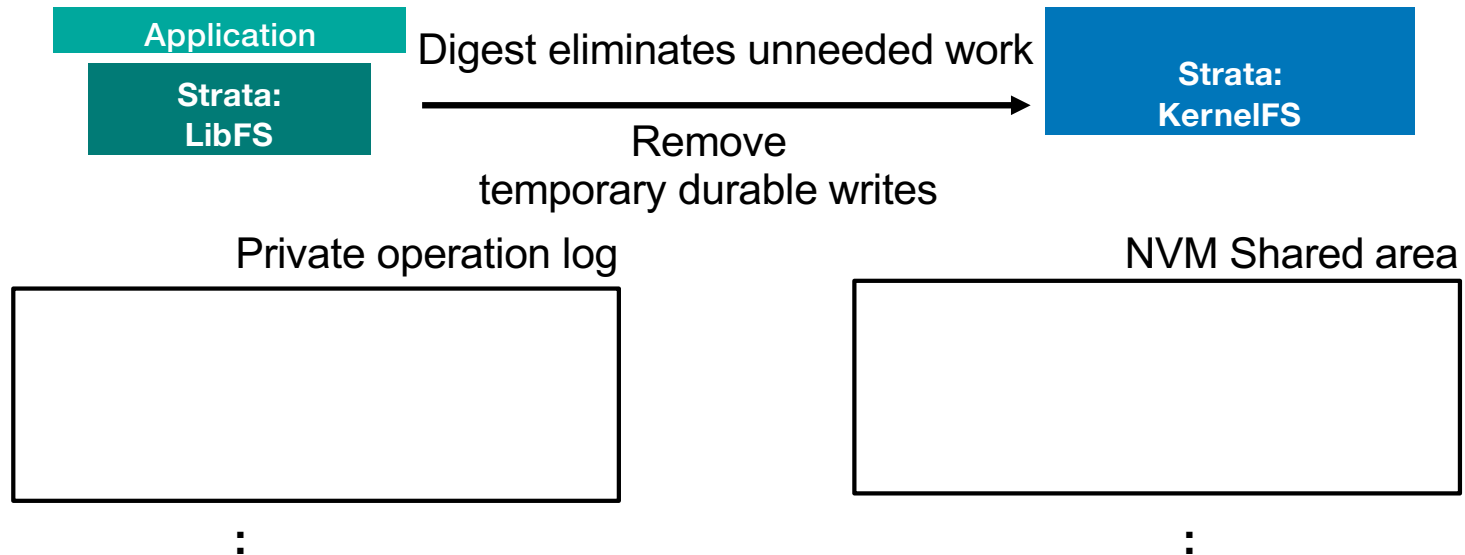
Digest data in kernel



- **Visibility:**
make private log visible to other applications
- **Data layout:**
turn write-optimized to read-optimized format (extent tree)
- Large, batched IO
 - Coalesce log

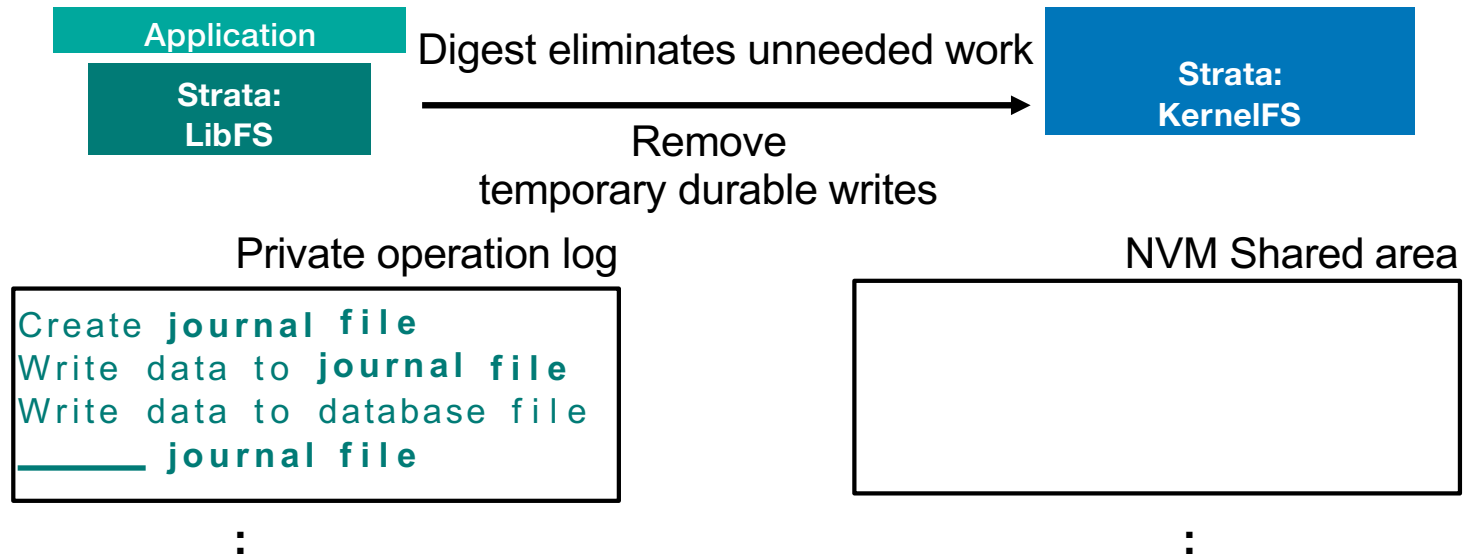
Digest optimization: Log coalescing

SQLite, Mail server: crash consistent update using write ahead logging



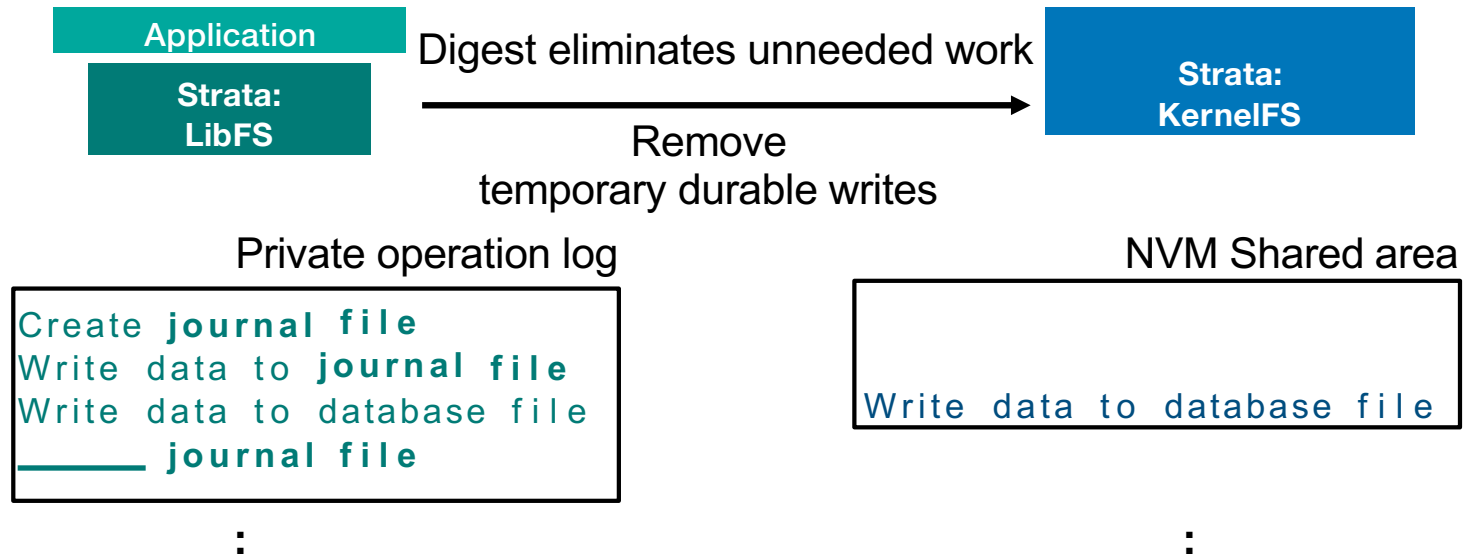
Digest optimization: Log coalescing

SQLite, Mail server: crash consistent update using write ahead logging



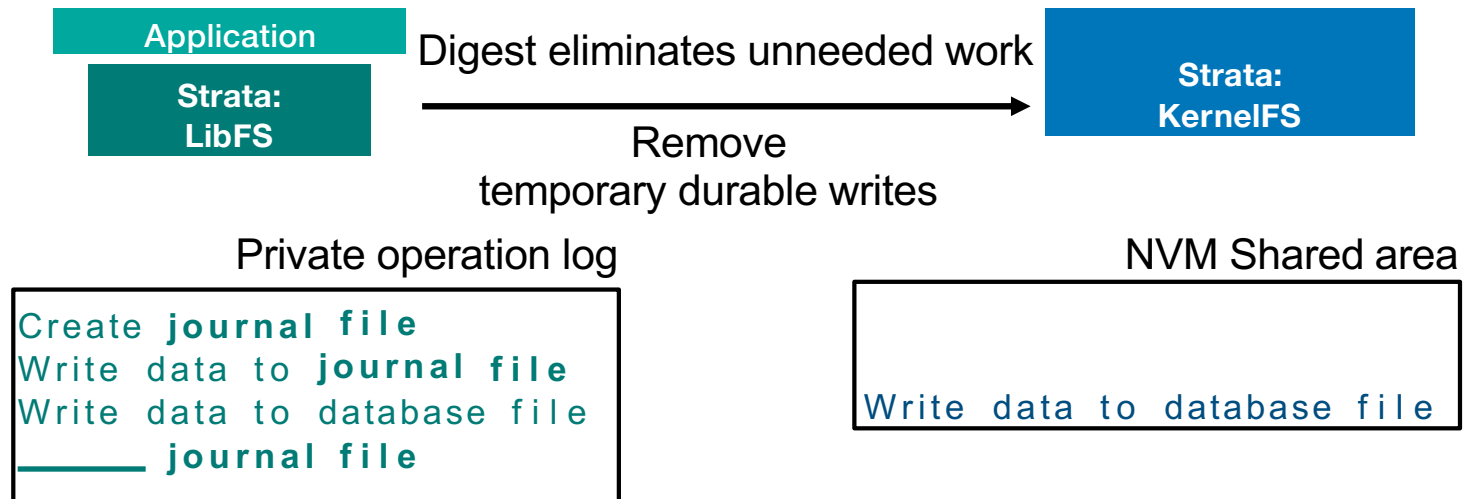
Digest optimization: Log coalescing

SQLite, Mail server: crash consistent update using write ahead logging



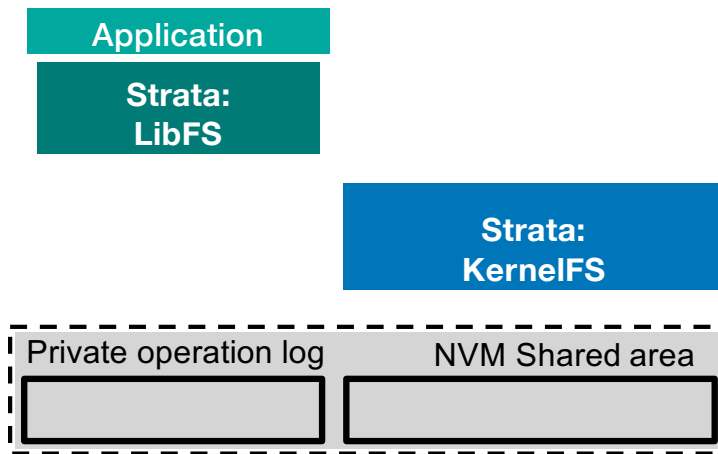
Digest optimization: Log coalescing

SQLite, Mail server: crash consistent update using write ahead logging

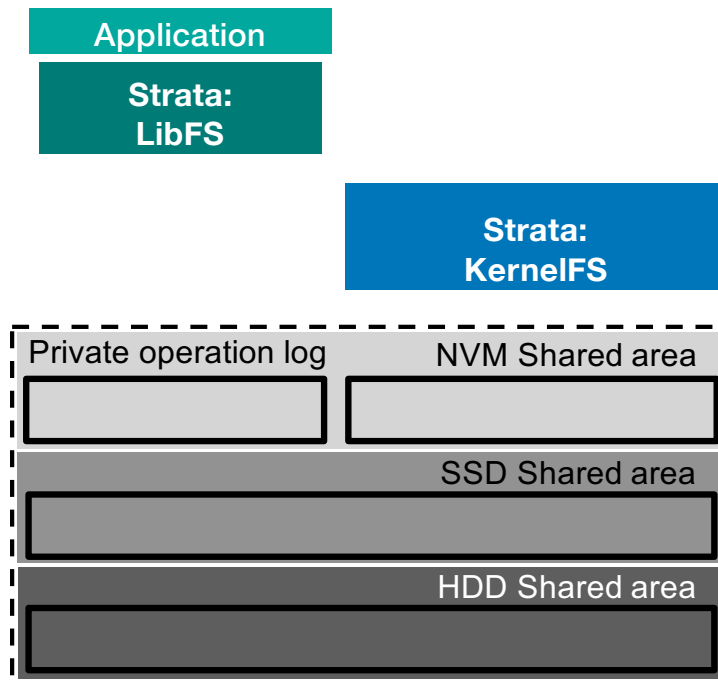


**Throughput optimization:
Log coalescing saves IO while digesting**

Digest and migrate data in kernel

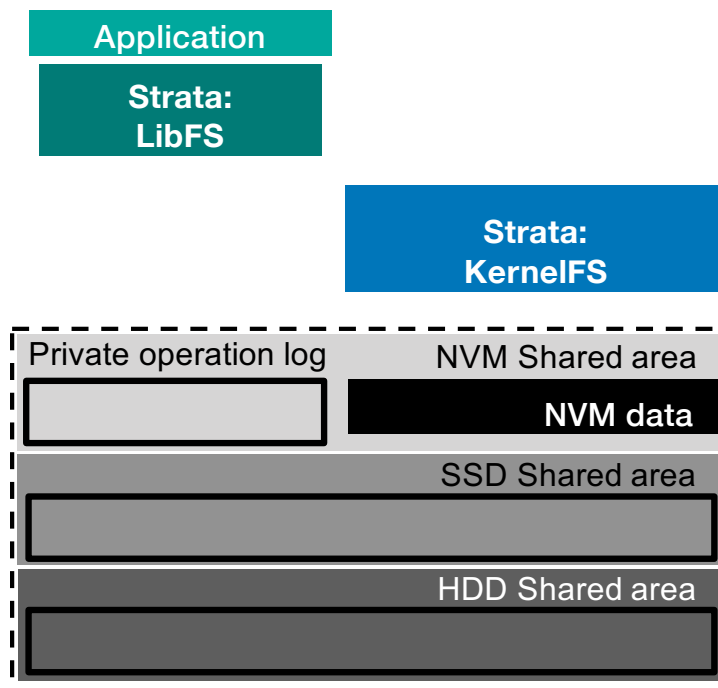


Digest and migrate data in kernel



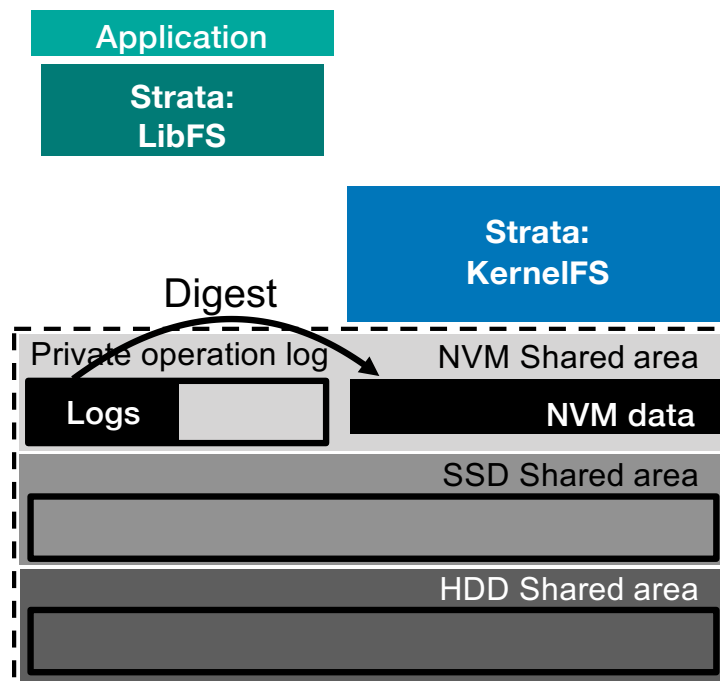
- Low-cost capacity
- KernelFS migrates cold data to lower layers

Digest and migrate data in kernel



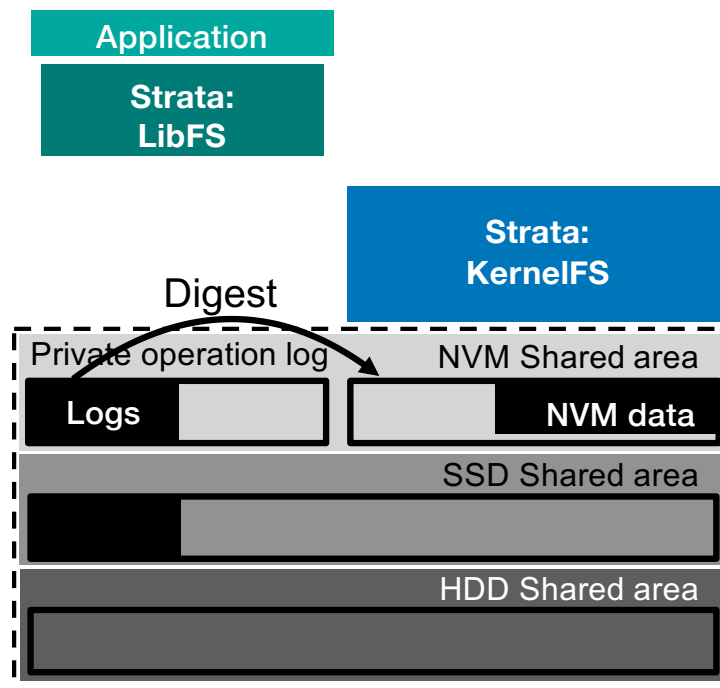
- Low-cost capacity
- KernelFS migrates cold data to lower layers

Digest and migrate data in kernel



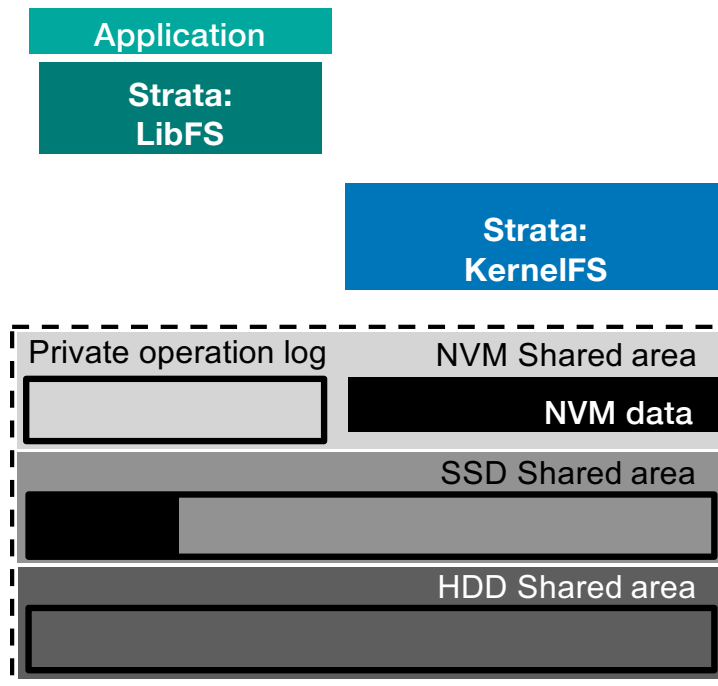
- Low-cost capacity
- KernelFS migrates cold data to lower layers

Digest and migrate data in kernel



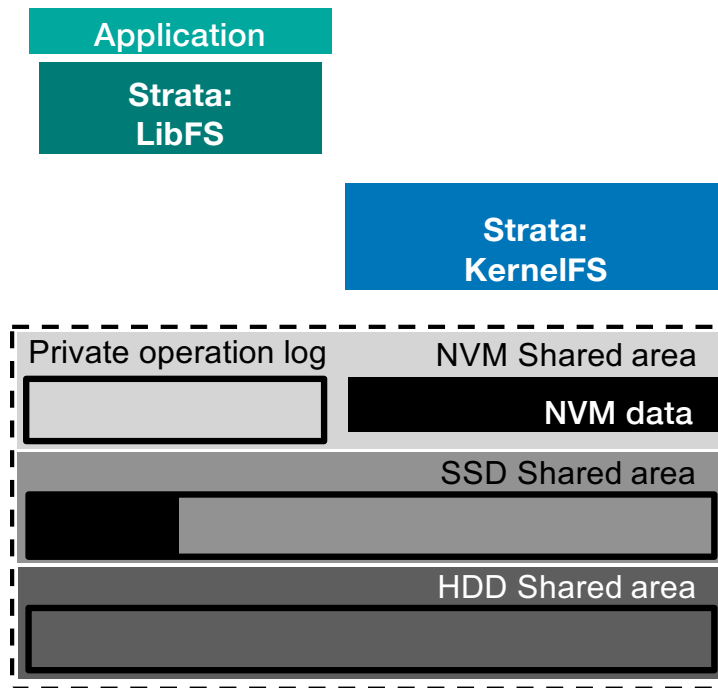
- Low-cost capacity
- KernelFS migrates cold data to lower layers

Digest and migrate data in kernel



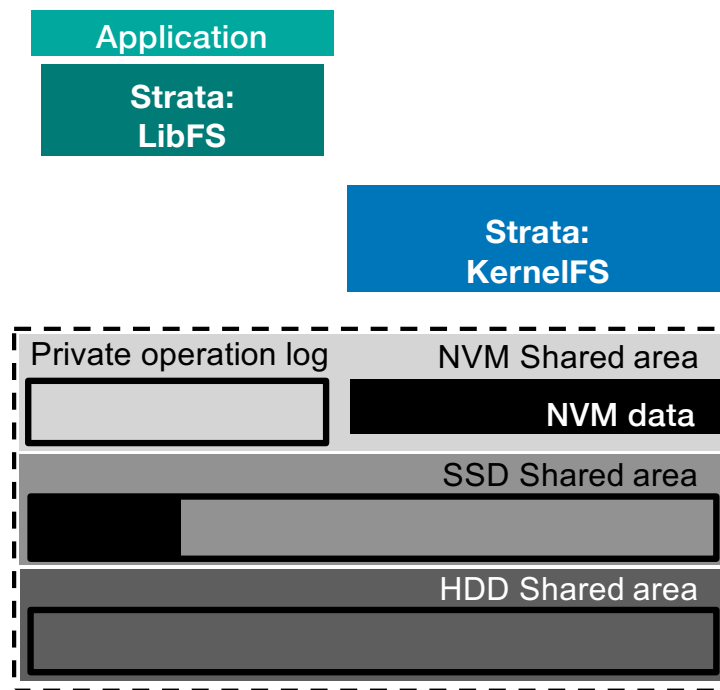
- Low-cost capacity
- KernelFS migrates cold data to lower layers

Digest and migrate data in kernel



- Low-cost capacity
 - KernelFS migrates cold data to lower layers
- Handle device IO properties
 - Migrate 1 GB blocks
 - Avoid SSD garbage collection overhead

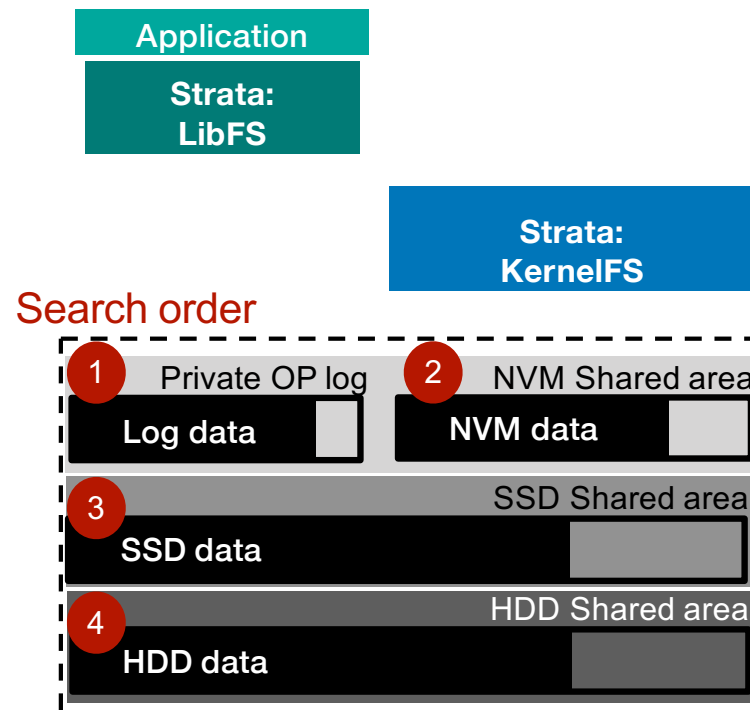
Digest and migrate data in kernel



- Low-cost capacity
 - KernelFS migrates cold data to lower layers
- Handle device IO properties
 - Migrate 1 GB blocks
 - Avoid SSD garbage collection overhead

Resembles log-structured merge (LSM) tree

Read: hierarchical search



Shared file access

- **Leases** grant access rights to applications [SOSP'89]
 - Required for files and directories
 - Function like lock, but revocable
 - Exclusive writer, shared readers

Shared file access

- **Leases** grant access rights to applications [SOSP'89]
 - Required for files and directories
 - Function like lock, but revocable
 - Exclusive writer, shared readers
- On revocation, LibFS digests leased data
 - Private data made public before losing lease
- **Leases serialize concurrent updates**

Outline

- **LibFS:** Log operations to NVM at user-level
 - Fast user-level access
 - In-order, synchronous IO
- **KernelFS:** Digest and migrate data in kernel
 - Asynchronous digest
 - Transparent data migration
 - Shared file access
- **Evaluation**

Experimental setup

- 2x Intel Xeon E5-2640 CPU, 64 GB DRAM
 - 400 GB NVMe SSD, 1 TB HDD
- Ubuntu 16.04 LTS, Linux kernel 4.8.12
- Emulated NVM
 - Use 40 GB of DRAM
 - Performance model [Y. Zhang et al. MSST 2015]
 - Throttle latency & throughput in software

Evaluation questions

- **Latency:**
 - Does Strata efficiently support small, random writes?
 - Does asynchronous digest have an impact on latency?
- **Throughput:**
 - Strata writes data twice (logging and digesting).
Can Strata sustain high throughput?
 - How well does Strata perform
when managing data across storage layers?

Related work

- **NVM file systems**

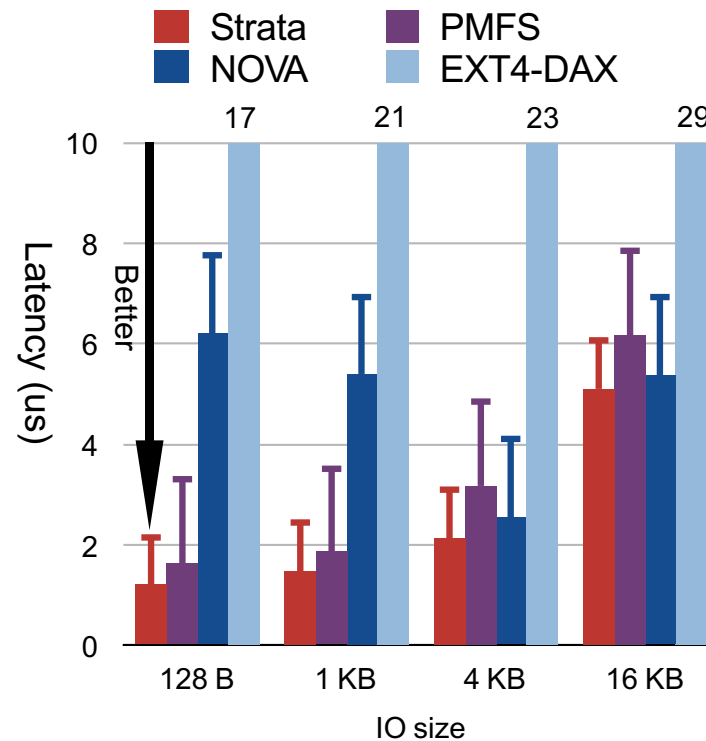
-  PMFS[EuroSys 14]: In-place update file system
-  NOVA[FAST 16]: log-structured file system
-  EXT4-DAX: NVM support for EXT4

- **SSD file system**

-  F2FS[FAST 15]: log-structured file system

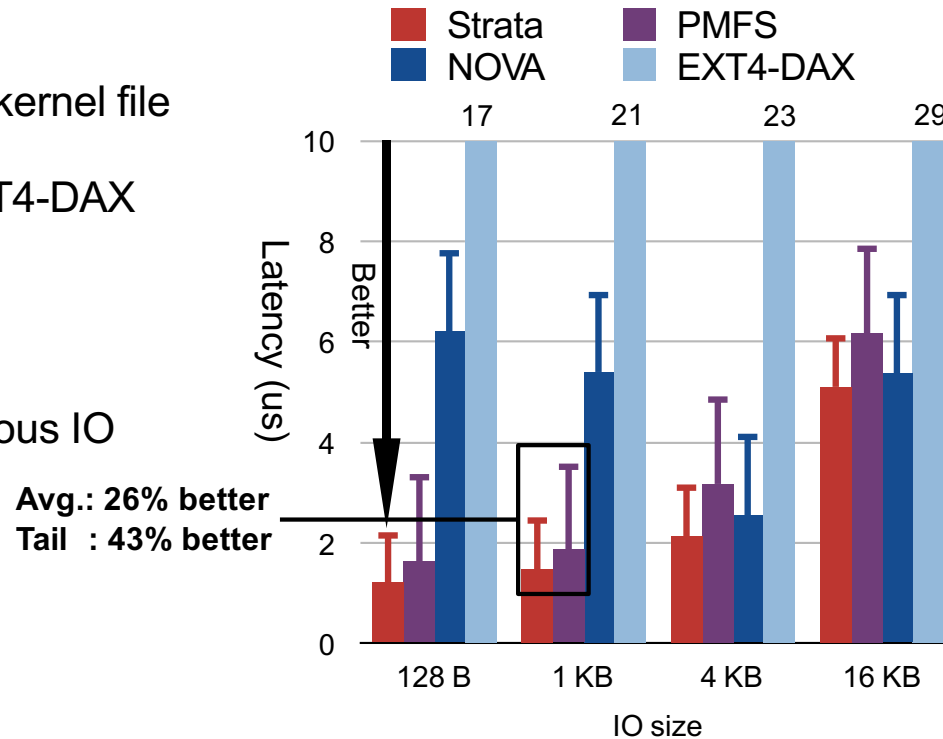
Microbenchmark: write latency

- Strata logs to NVM
 - Compare to NVM kernel file systems:
PMFS, NOVA, EXT4-DAX
- Strata, NOVA
 - In-order, synchronous IO
 - Atomic write
- PMFS, EXT4-DAX
 - No atomic write



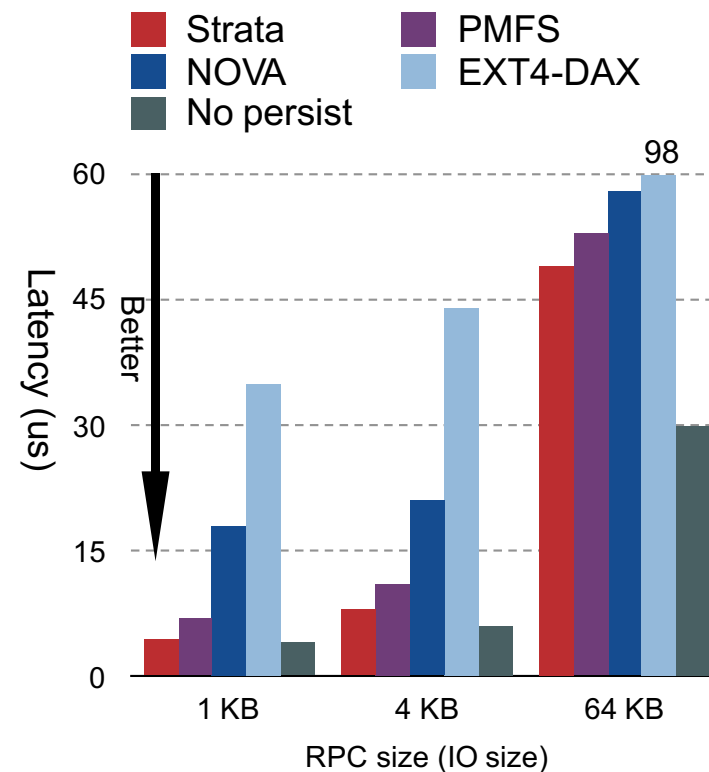
Microbenchmark: write latency

- Strata logs to NVM
 - Compare to NVM kernel file systems:
PMFS, NOVA, EXT4-DAX
- Strata, NOVA
 - In-order, synchronous IO
 - Atomic write
- PMFS, EXT4-DAX
 - No atomic write



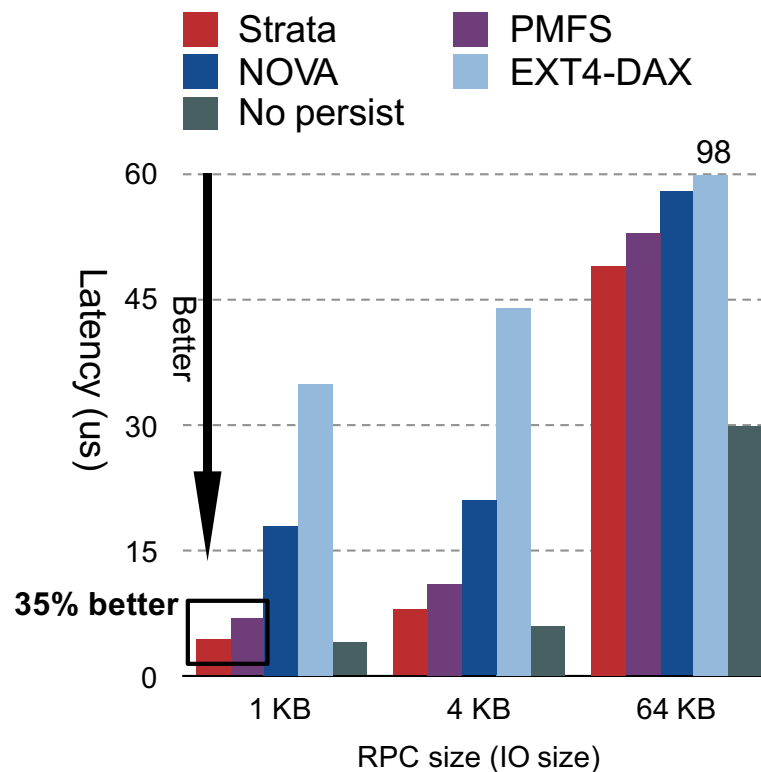
Latency: persistent RPC

- Foundation of most servers
 - Persist RPC data before sending ACK to client
- RPC over RDMA
 - 40 Gb/s Infiniband NIC
- For small IO (1 KB)
 - 25% slower than No persist
 - 35% faster than PMFS
 - 7x faster than EXT4-DAX



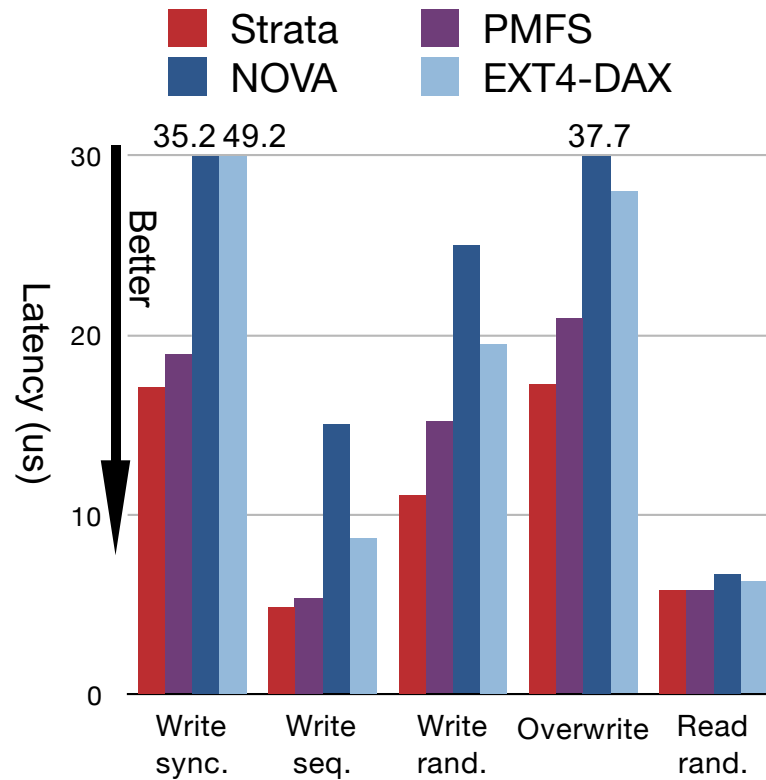
Latency: persistent RPC

- Foundation of most servers
 - Persist RPC data before sending ACK to client
- RPC over RDMA
 - 40 Gb/s Infiniband NIC
- For small IO (1 KB)
 - 25% slower than No persist
 - 35% faster than PMFS
 - 7x faster than EXT4-DAX



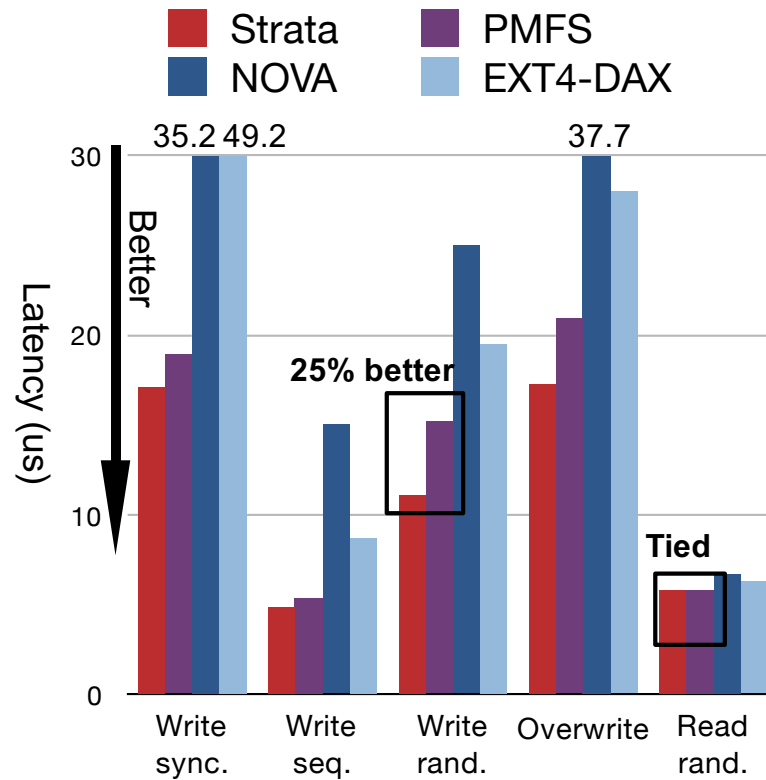
Latency: LevelDB

- LevelDB (NVM)
 - Key size: 16 B
 - Value size: 1 KB
 - 300,000 objects
- **Workload causes asynchronous digests**
- Fast user-level logging
 - Random write
 - 25% better than PMFS
 - Random read
 - Tied with PMFS



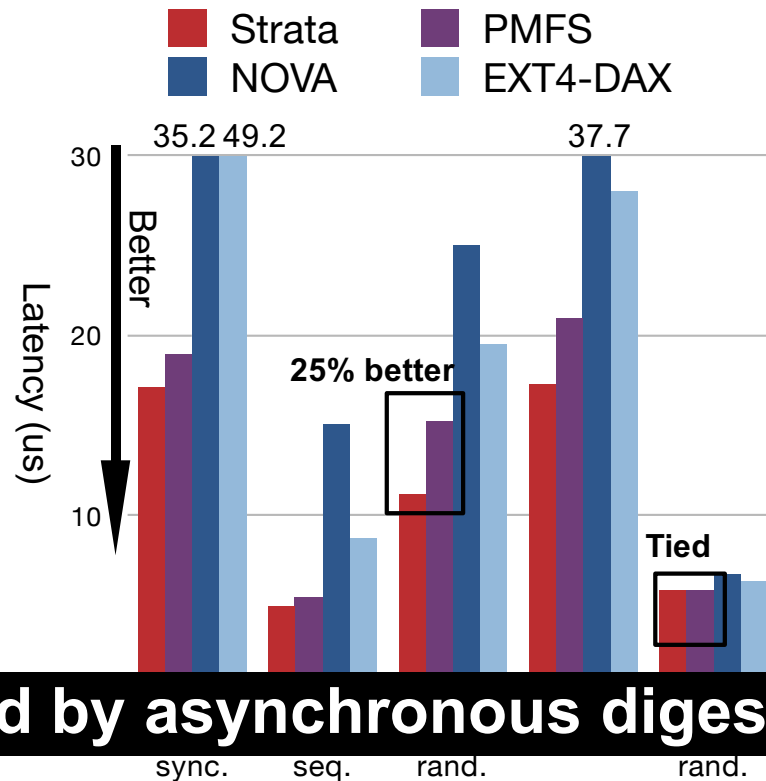
Latency: LevelDB

- LevelDB (NVM)
 - Key size: 16 B
 - Value size: 1 KB
 - 300,000 objects
- **Workload causes asynchronous digests**
- Fast user-level logging
 - Random write
 - 25% better than PMFS
 - Random read
 - Tied with PMFS



Latency: LevelDB

- LevelDB (NVM)
 - Key size: 16 B
 - Value size: 1 KB
 - 300,000 objects
- **Workload causes asynchronous digests**
- Fast user-level logging
 - Random write
 - 25% better than PMFS
- Tied with PMFS



IO latency not impacted by asynchronous digest

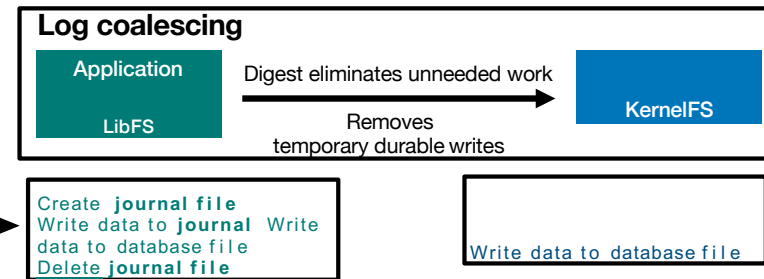
Evaluation questions

- **Latency:**
 - Does Strata efficiently support small, random writes?
 - Does asynchronous digest have an impact on latency?
- **Throughput:**
 - Strata writes data twice (logging and digesting).
Can Strata sustain high throughput?
 - How well does Strata perform
when managing data across storage layers?

Throughput: Varmail

Mail server workload from Filebench

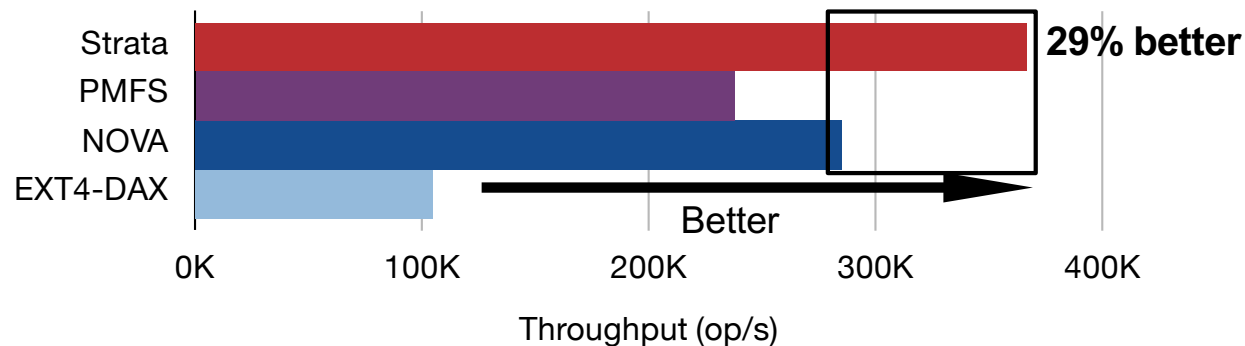
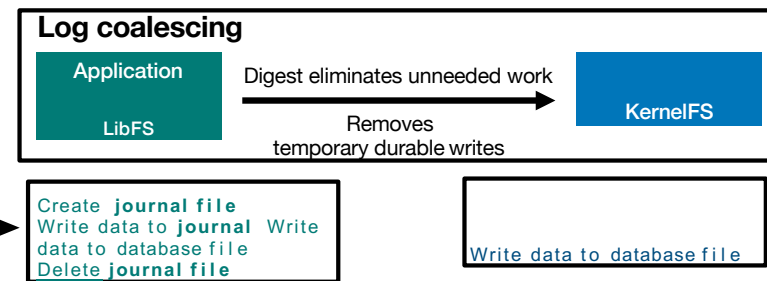
- Using only NVM
- 10000 files
- Read/Write ratio is 1:1
- **Write-ahead logging** →



Throughput: Varmail

Mail server workload from Filebench

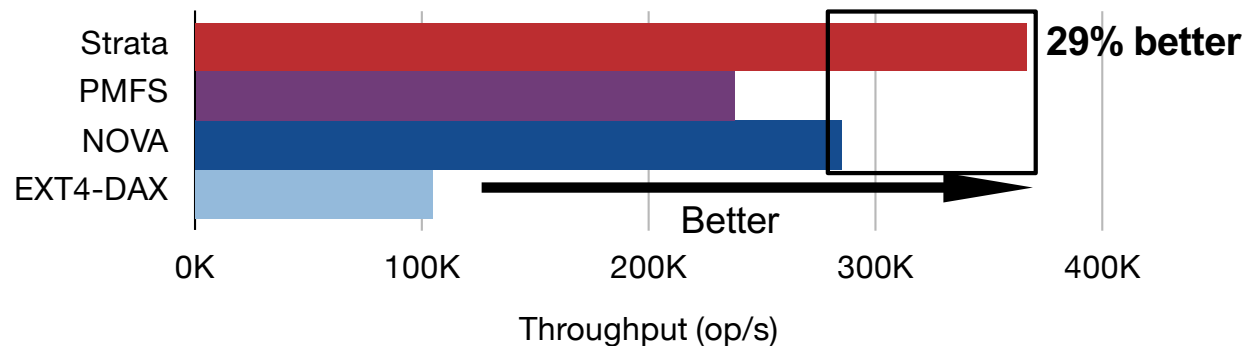
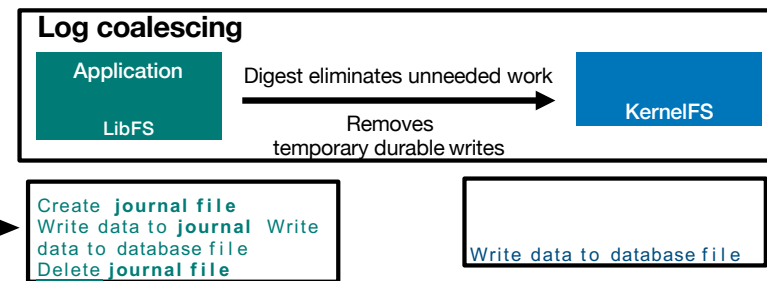
- Using only NVM
- 10000 files
- Read/Write ratio is 1:1
- **Write-ahead logging** →



Throughput: Varmail

Mail server workload from Filebench

- Using only NVM
- 10000 files
- Read/Write ratio is 1:1
- **Write-ahead logging** →



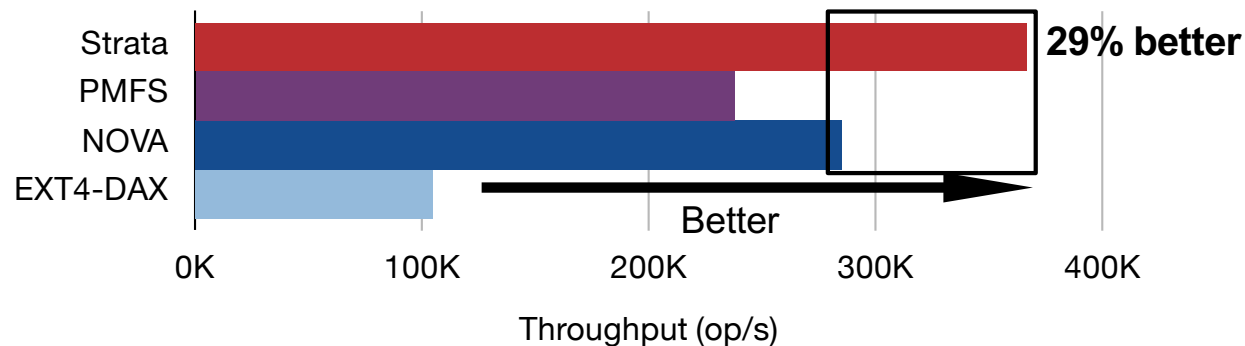
Log coalescing eliminates 86% of log entries, saving 14 GB of IO

Throughput: Varmail

No kernel file system has both low latency and high throughput:

- PMFS: better latency
- NOVA: better throughput

Strata achieves both low latency and high throughput



Log coalescing eliminates 86% of log entries, saving 14 GB of IO

Throughput: data migration

File server workload from Filebench

- Working set starts at NVM, grows to SSD, HDD
- Read/Write ratio is 1:2

Throughput: data migration

File server workload from Filebench

- Working set starts at NVM, grows to SSD, HDD
- Read/Write ratio is 1:2

User-level migration

- LRU: whole file granularity
- Treat each file system as a black-box
- NVM: NOVA, SSD: F2FS, HDD: EXT4

Throughput: data migration

File server workload from Filebench

- Working set starts at NVM, grows to SSD, HDD
- Read/Write ratio is 1:2

User-level migration

- LRU: whole file granularity
- Treat each file system as a black-box
- NVM: NOVA, SSD: F2FS, HDD: EXT4

Block-level caching

- Linux LVM cache, formatted with F2FS

Throughput: data migration

File server workload from Filebench

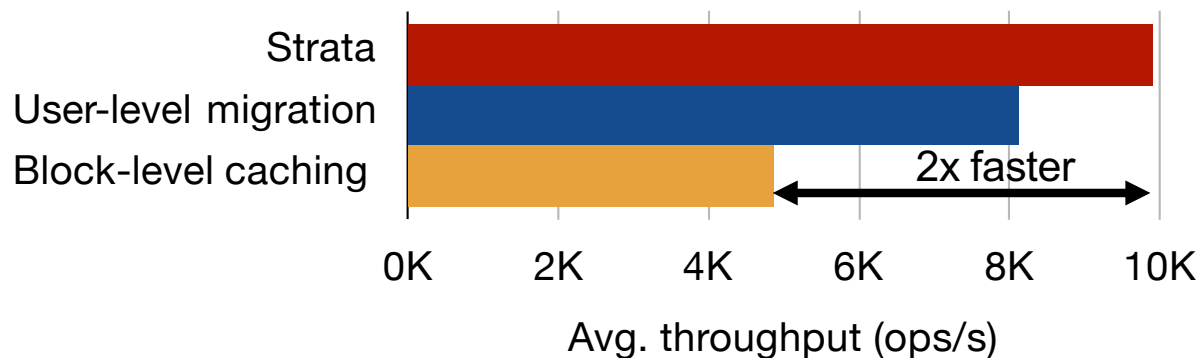
- Working set starts at NVM, grows to SSD, HDD
- Read/Write ratio is 1:2

User-level migration

- LRU: whole file granularity
- Treat each file system as a black-box
- NVM: NOVA, SSD: F2FS, HDD: EXT4

Block-level caching

- Linux LVM cache, formatted with F2FS



Throughput: data migration

File server workload from Filebench

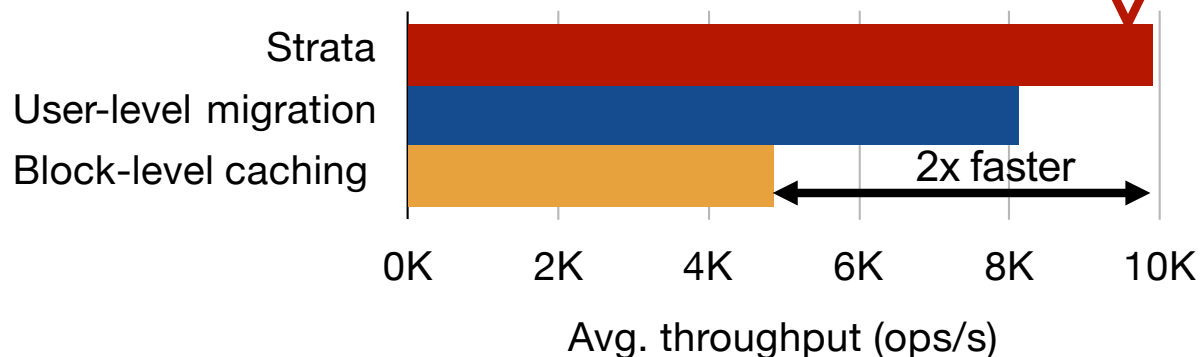
- Working set starts at NVM, grows to SSD, HDD
- Read/Write ratio is 1:2

User-level migration

- LRU: whole file granularity
- Treat each file system as a black-box
- NVM: NOVA, SSD: F2FS, HDD: EXT4

Block-level caching

- Linux LVM cache, formatted with F2FS



Conclusion

Server applications need fast, small random IO on vast datasets with intuitive crash consistency

Strata, a cross media file system, addresses these concerns

Performance: low latency, high throughput

- Novel split of LibFS, KernelFS
- Fast user-level access

Low-cost capacity: leverage NVM, SSD & HDD

- Asynchronous digest
- Transparent data migration with large, sequential IO

Simplicity: intuitive crash consistency model

- In-order, synchronous IO

Source code is available at
<https://github.com/ut-osa/strata>

Table of contents

- A file system is no longer tied to one storage device
 - **Strata**: A Cross Media File System, **SOSP'17**
- The file system as a distributed service:
 - **Assise**: Performance and Availability via Client local NVM in a Distributed File System, **OSDI'20**
- Persistence is now also about recovery speed, not just durability
 - **Gemini**: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints, **SOSP'23**

Assise:

Performance and Availability via Client-local NVM in a Distributed File System

- Thomas Anderson, Marco Canini, Jongyul Kim, Dejan Kostić, Youngjin Kwon, Simon Peter, **Waleed Reda**, Henry N. Schuh, and Emmett Witchel



Commercial adoption of NVM

Non-Volatile Memory

- Fast IO with byte-addressability
- Only 40% cost of DRAM
- Persistent across failures

Application

Memory

intel. OPTANE™»»

	DRAM		NVM
Write Latency	82 ns	~	94 ns
Cost (\$/GB)	9.77	>>	3.83

Commercial adoption of NVM

intel. OPTANE™»

Non-Volatile Memory

- Fast IO with byte-addressability
- Only 40% cost of DRAM
- Persistent across failures



Write Latency

DRAM

NVM

82 ns

~

94 ns

Cost (\$/GB)

9.77

>>

3.83

Application

Memory

Commercial adoption of NVM

intel. OPTANE™»

Non-Volatile Memory

- Fast IO with byte-addressability
- Only 40% cost of DRAM
- Persistent across failures



Write Latency

DRAM

82 ns

~

NVM

94 ns

Cost (\$/GB)

9.77

>>

3.83

Application

Memory

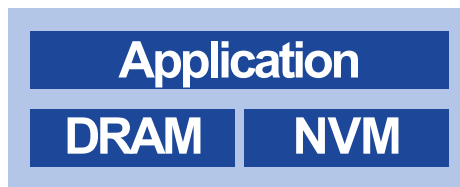
Commercial adoption of NVM

Non-Volatile Memory

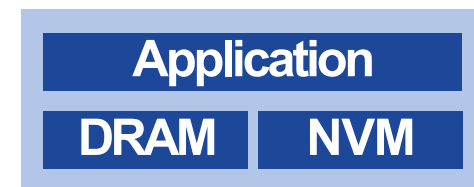
- Fast IO with byte-addressability
- Only 40% cost of DRAM
- Persistent across failures

intel. OPTANE™»»

	DRAM		NVM
Write Latency	82 ns	~	94 ns
Cost (\$/GB)	9.77	>>	3.83

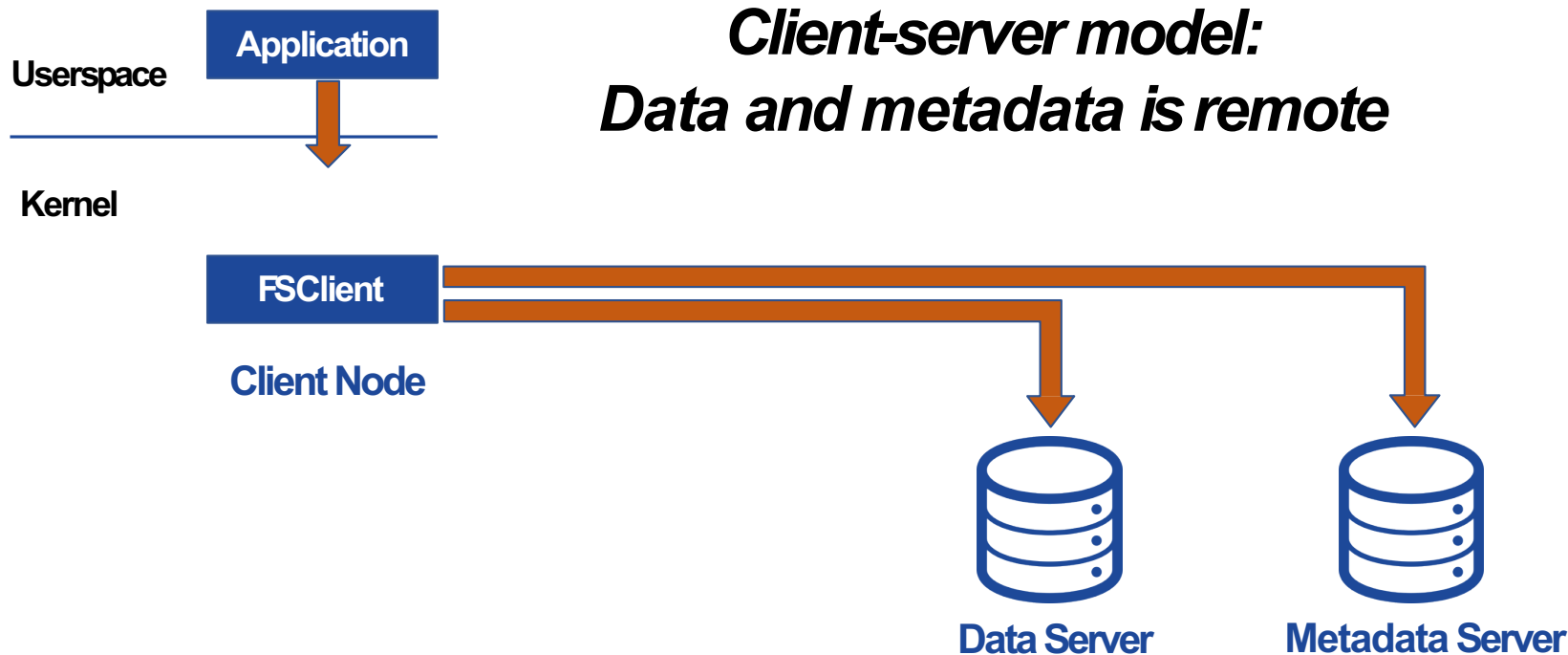


NVM adoption at scale



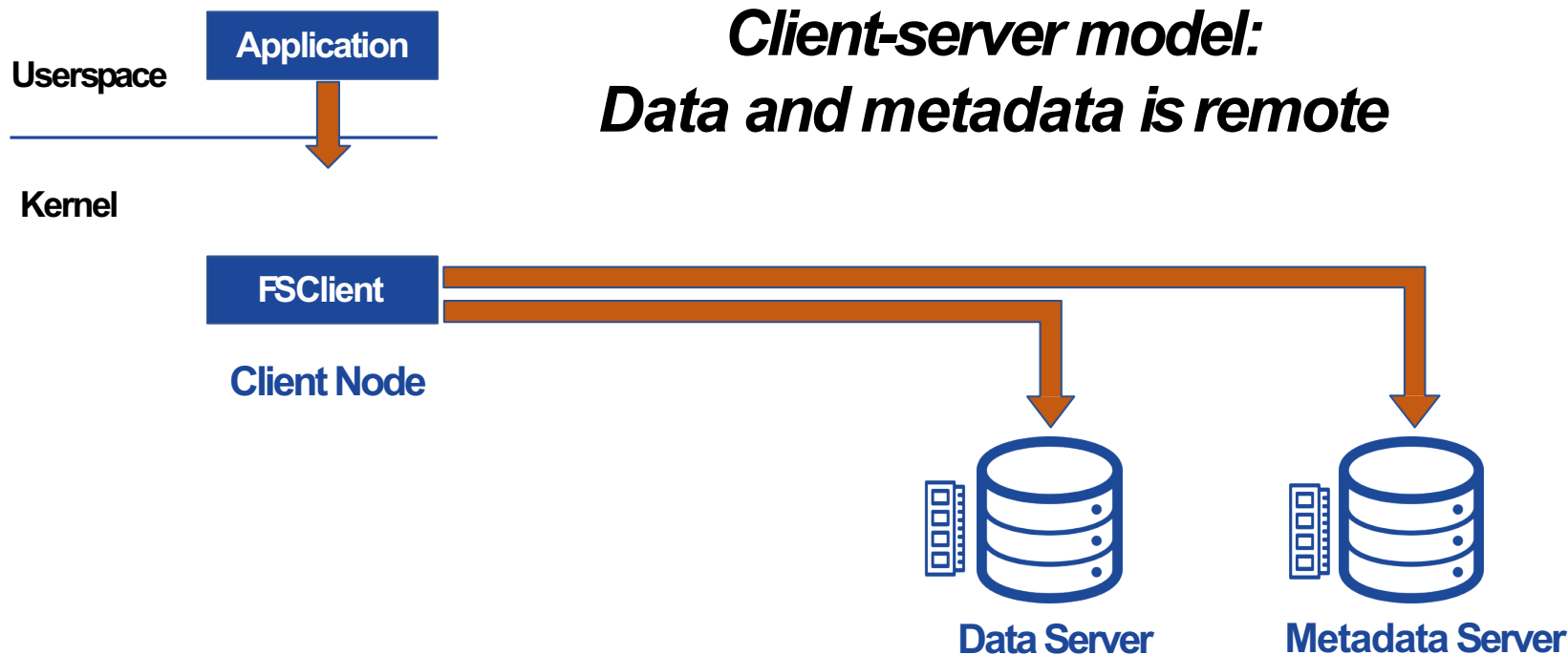
Distributed filesystems are integrating NVM

e.g. Ceph, NFS, ...



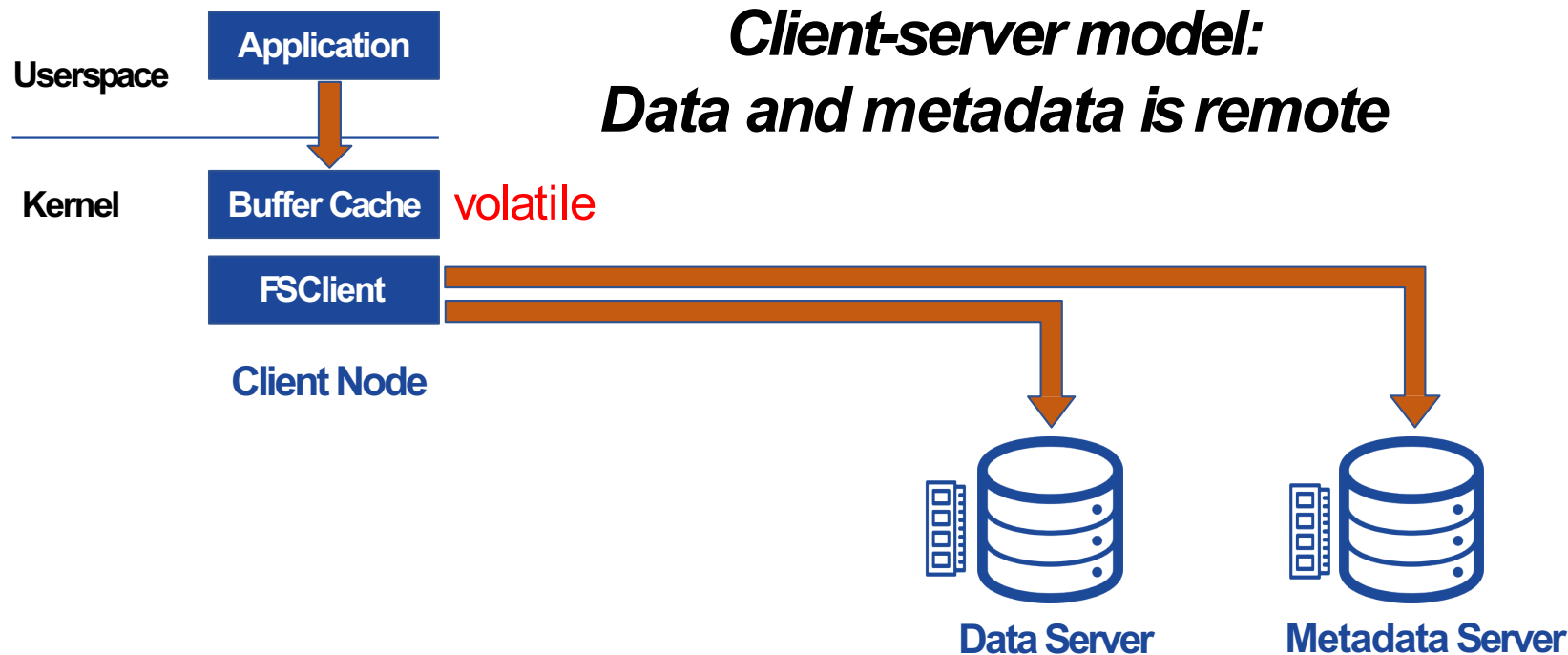
Distributed filesystems are integrating NVM

e.g. Ceph, NFS, ...



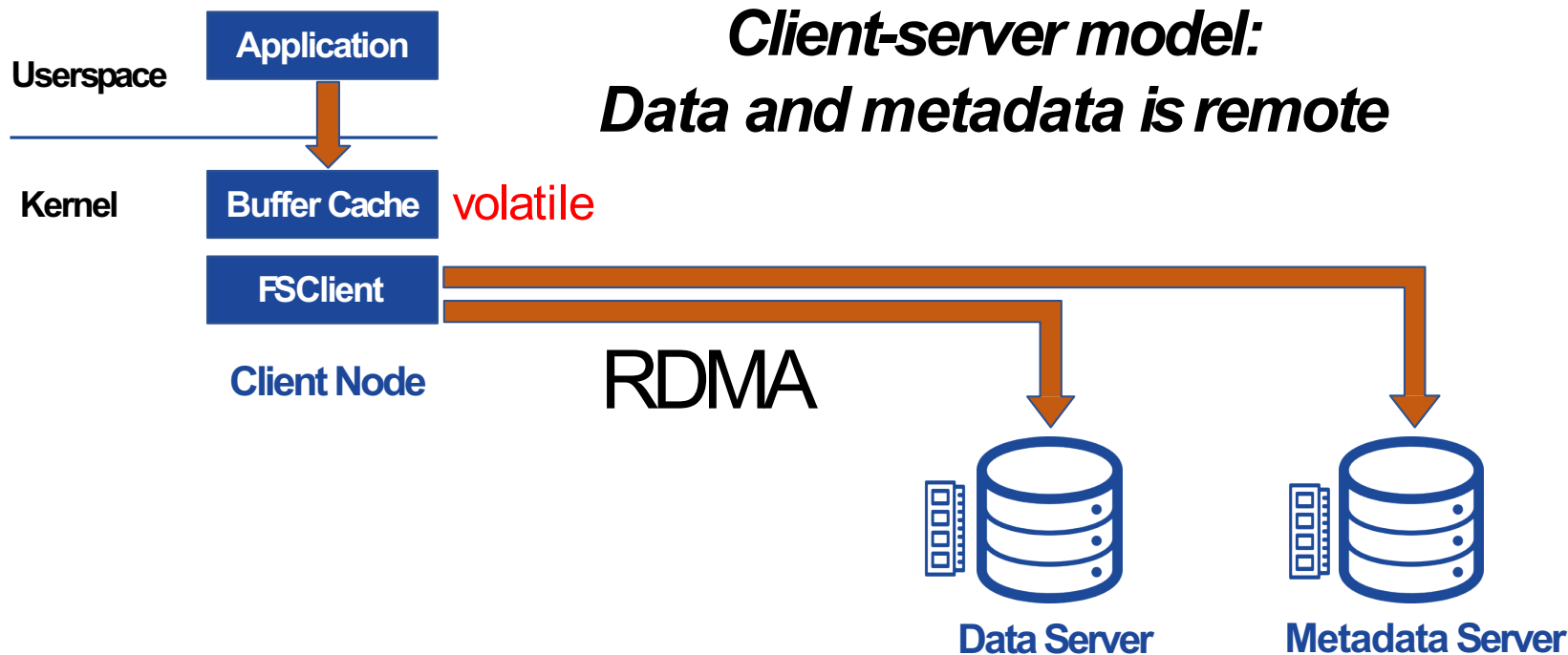
Distributed filesystems are integrating NVM

e.g. Ceph, NFS, ...



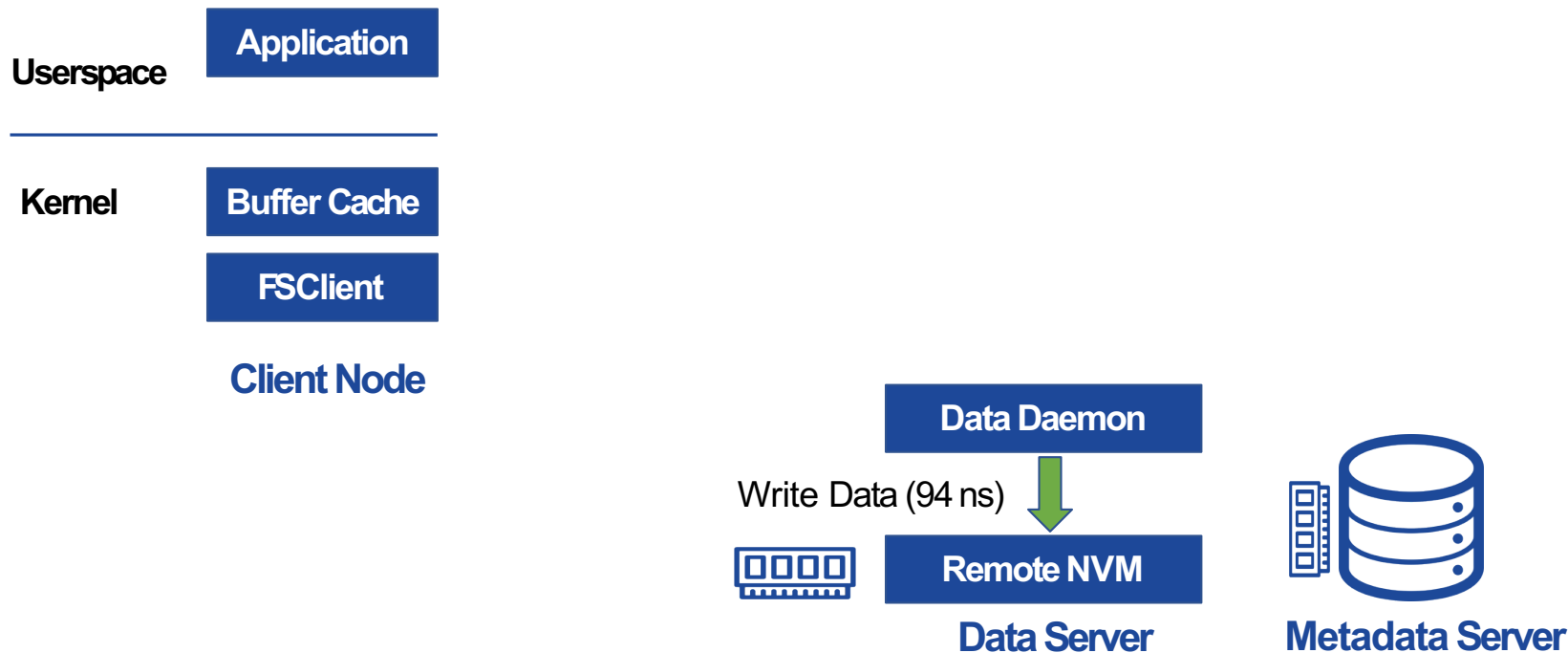
Distributed filesystems are integrating NVM

e.g. Ceph, NFS, ...



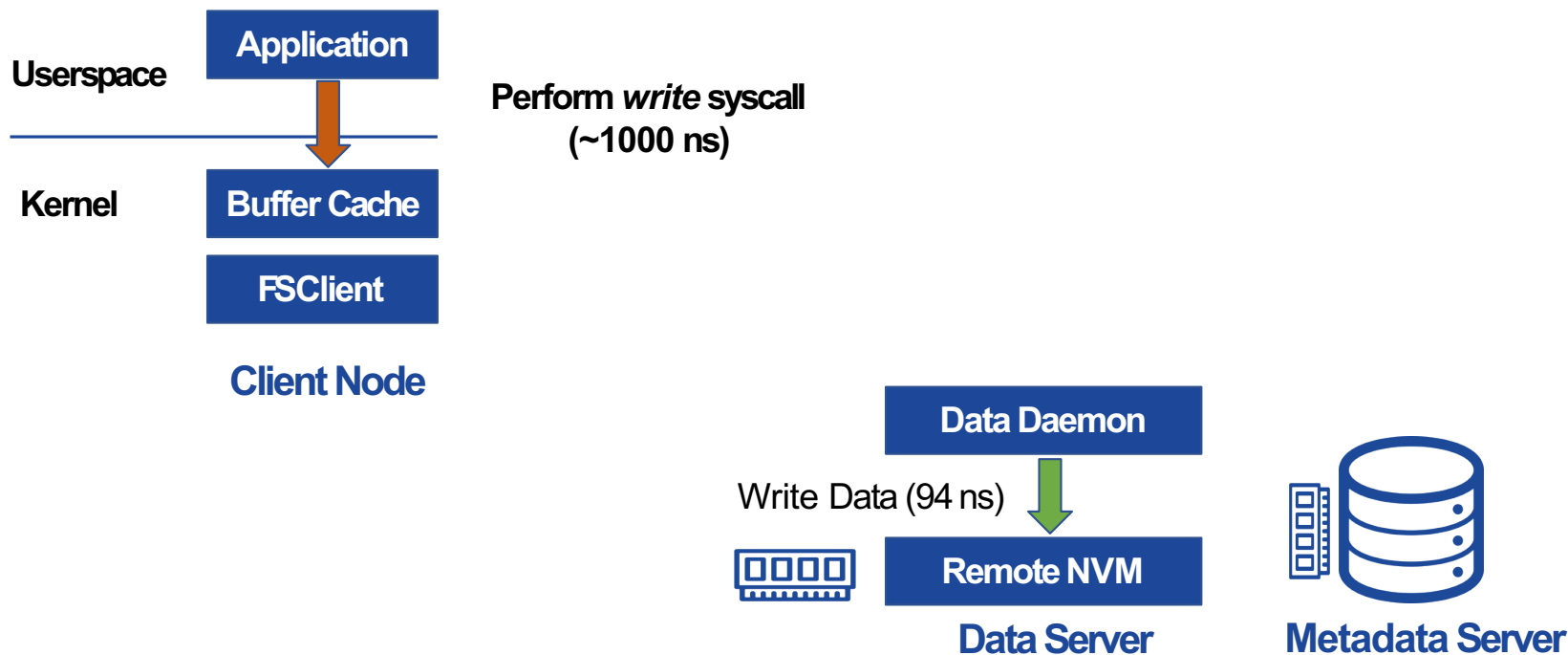
Example: Write 256B durably to a file

Challenge 1: Network/kernel latencies dominate NVM IO latency



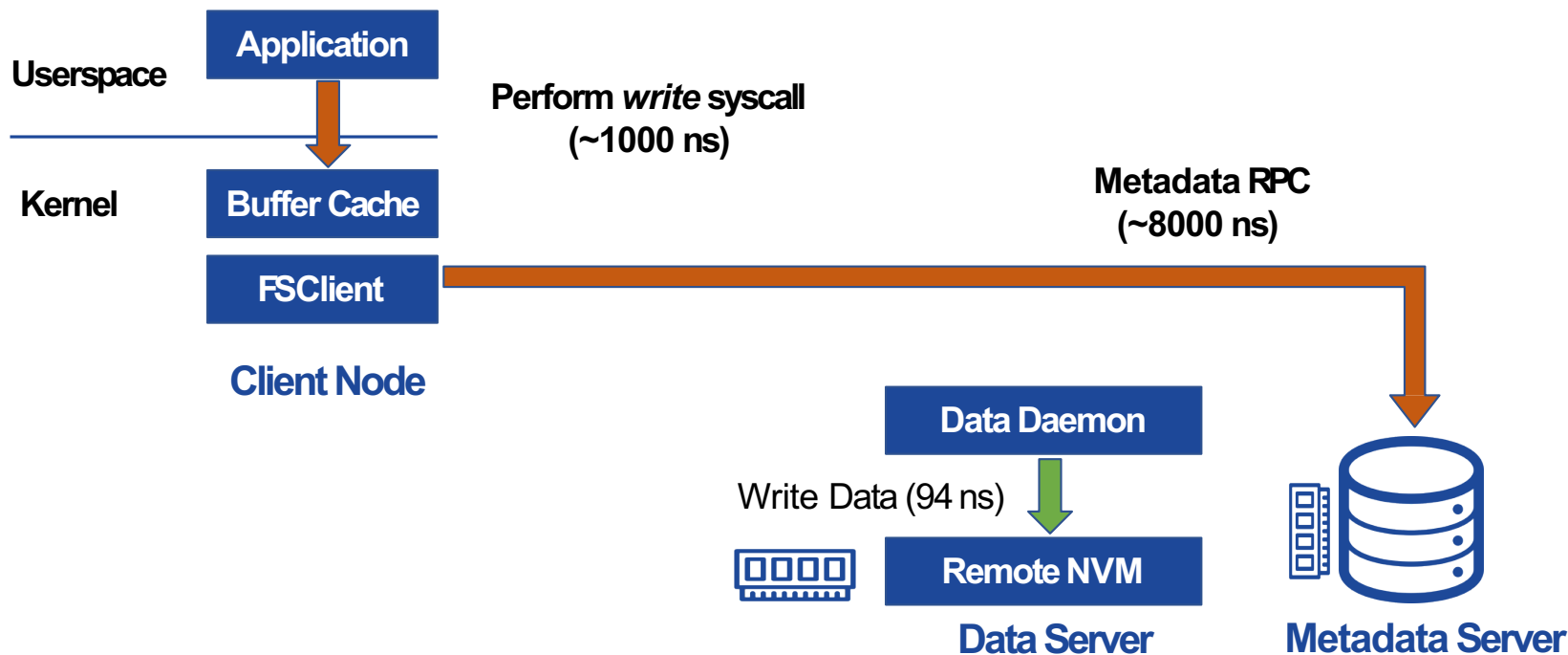
Example: Write 256B durably to a file

Challenge 1: Network/kernel latencies dominate NVM IO latency



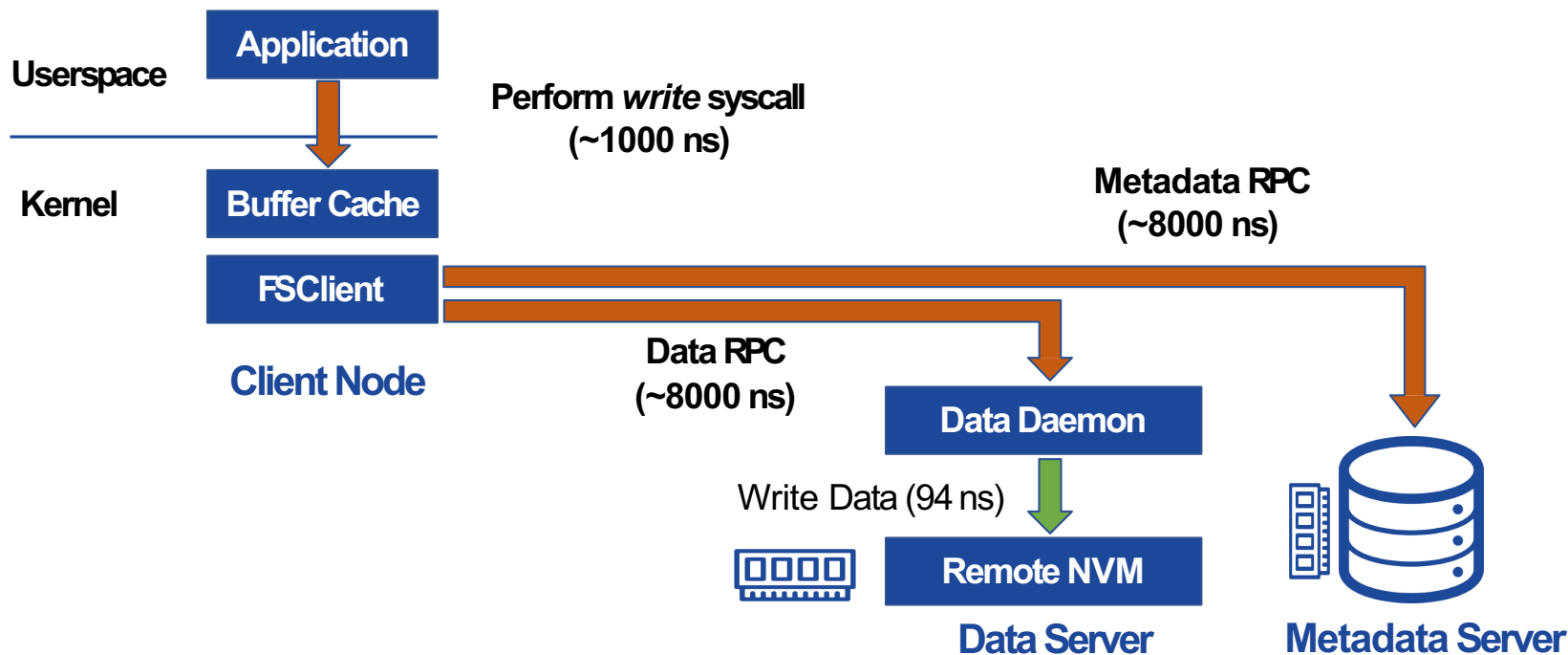
Example: Write 256B durably to a file

Challenge 1: Network/kernel latencies dominate NVM IO latency



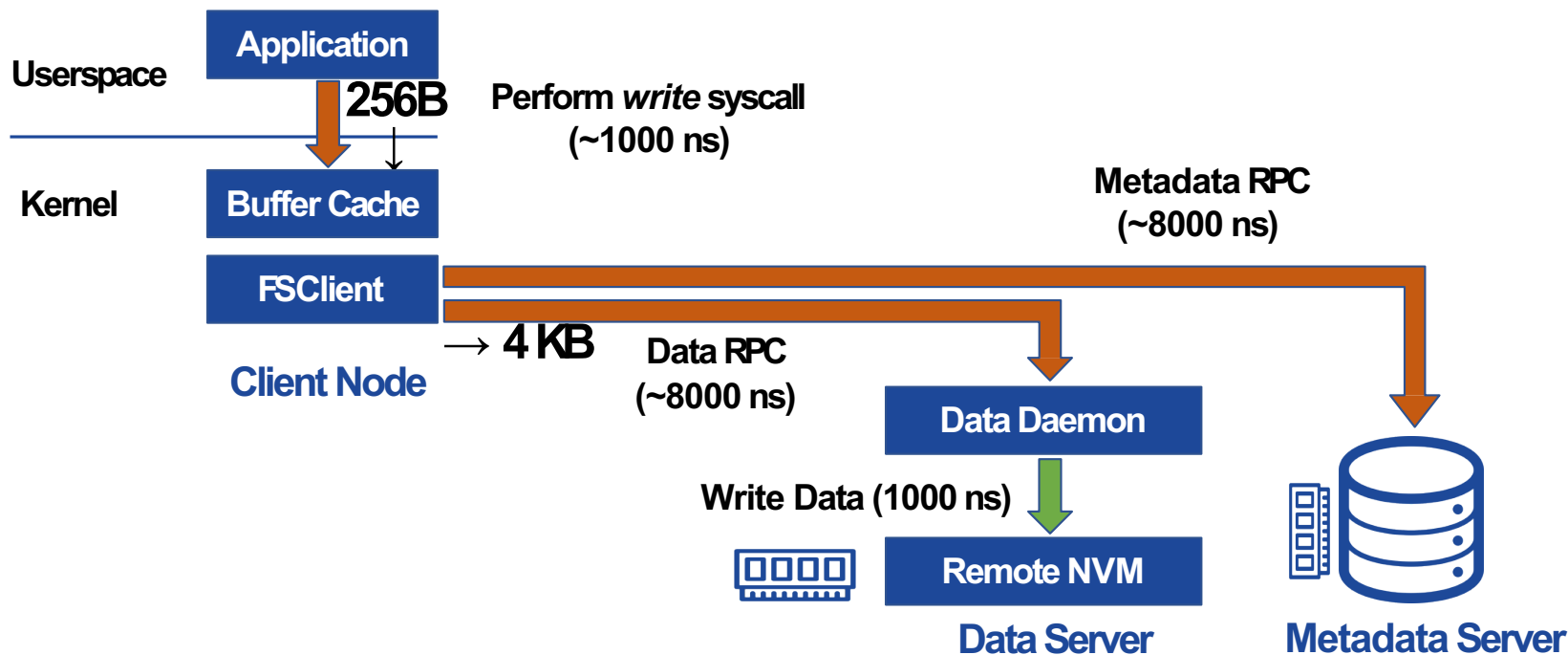
Example: Write 256B durably to a file

Challenge 1: Network/kernel latencies dominate NVM IO latency



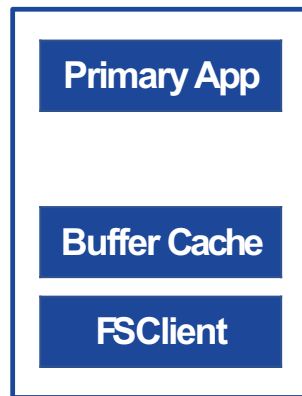
Example: Write 256B durably to a file

Challenge 2: Block-sized IO exacerbates network & storage latencies



Client fail-over & recovery

Challenge 3: File system must rebuild caches of failed clients



Client 1



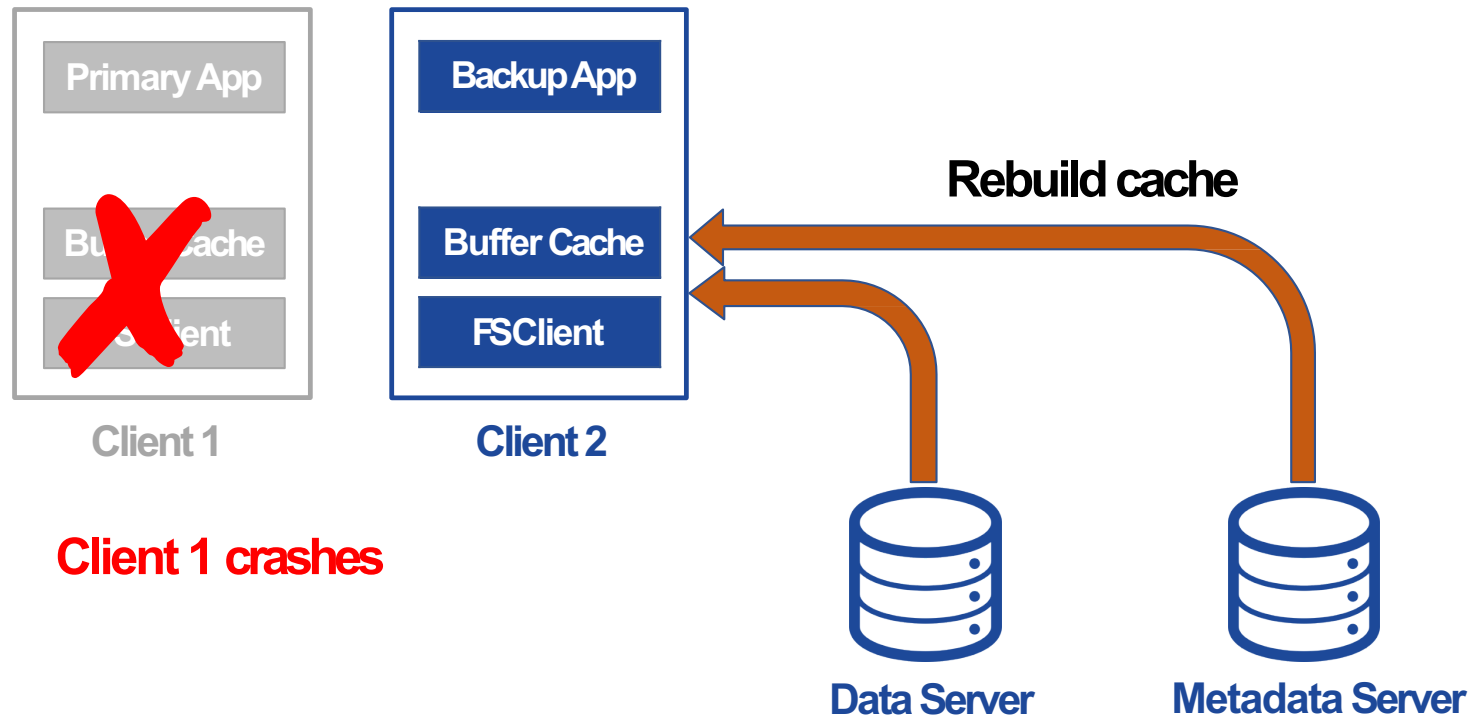
Data Server



Metadata Server

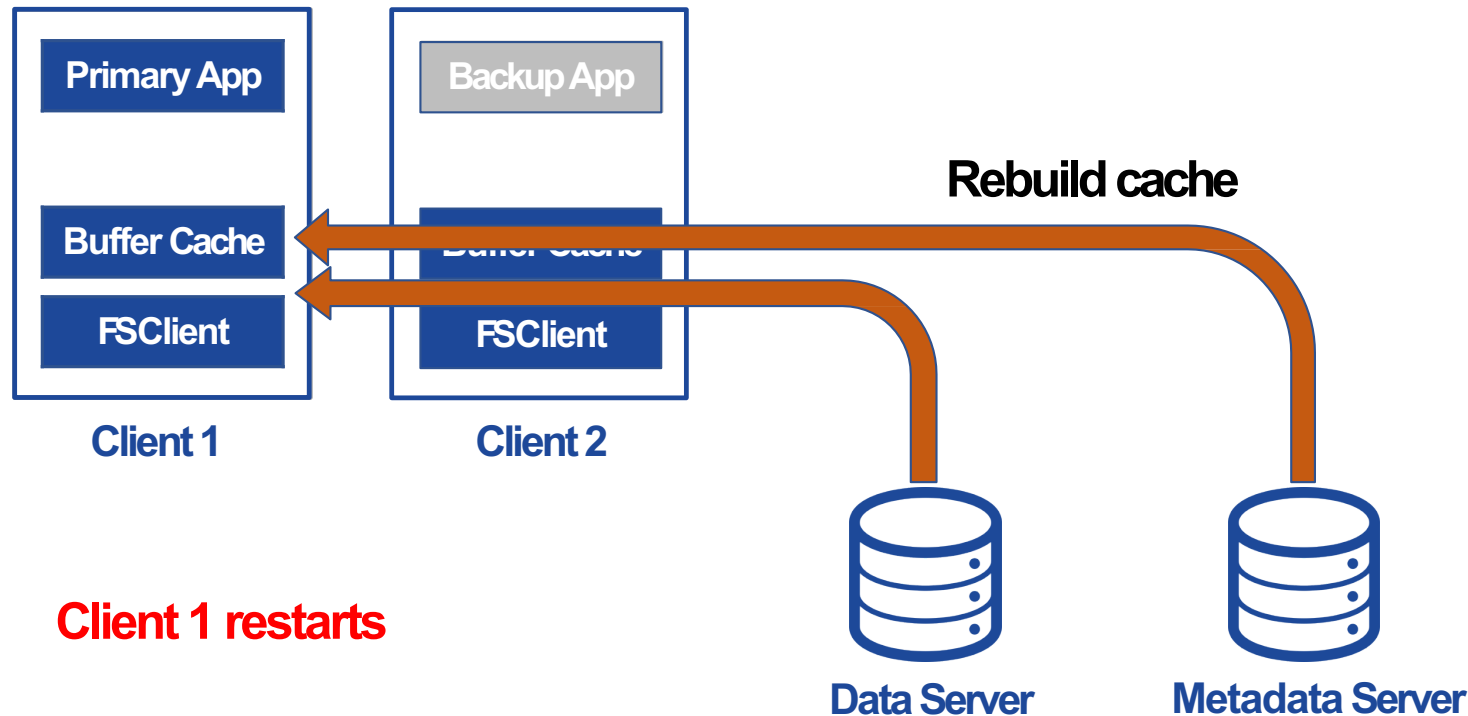
Client fail-over & recovery

Challenge 3: File system must rebuild caches of failed clients



Client fail-over & recovery

Challenge 3: File system must rebuild caches of failed clients



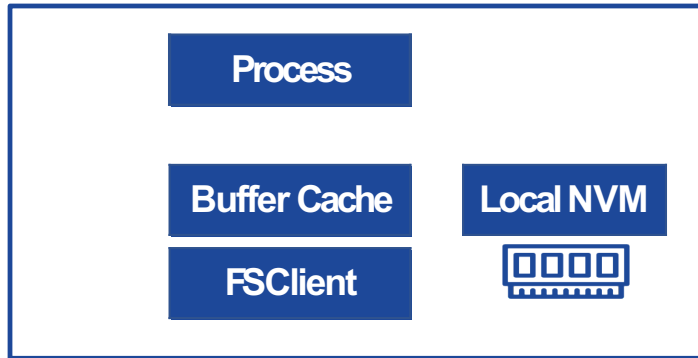
Assise: High performance & availability

Main principle: **Maximize use of client-local NVM**

1. **Low latency IO**: Perform process-local & client-local IO
2. **Linearizability and data crash consistency**
CC-NVM distributed client-side NVM coherence protocol
3. **High availability**
Fail-over to cache-hot client replicas, recover client NVMcaches

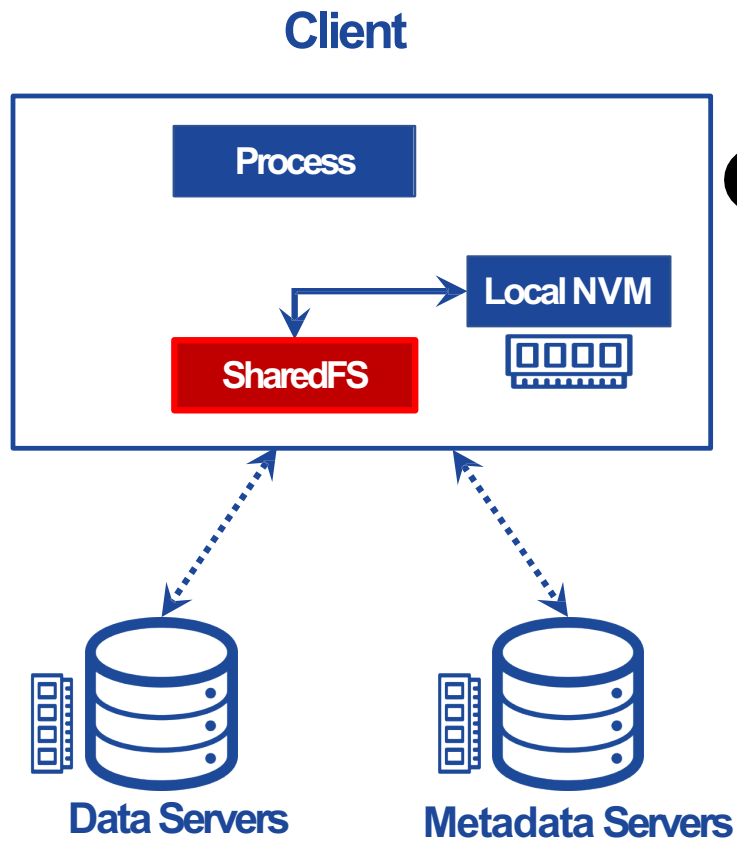
Client-local IO with Assise

Client



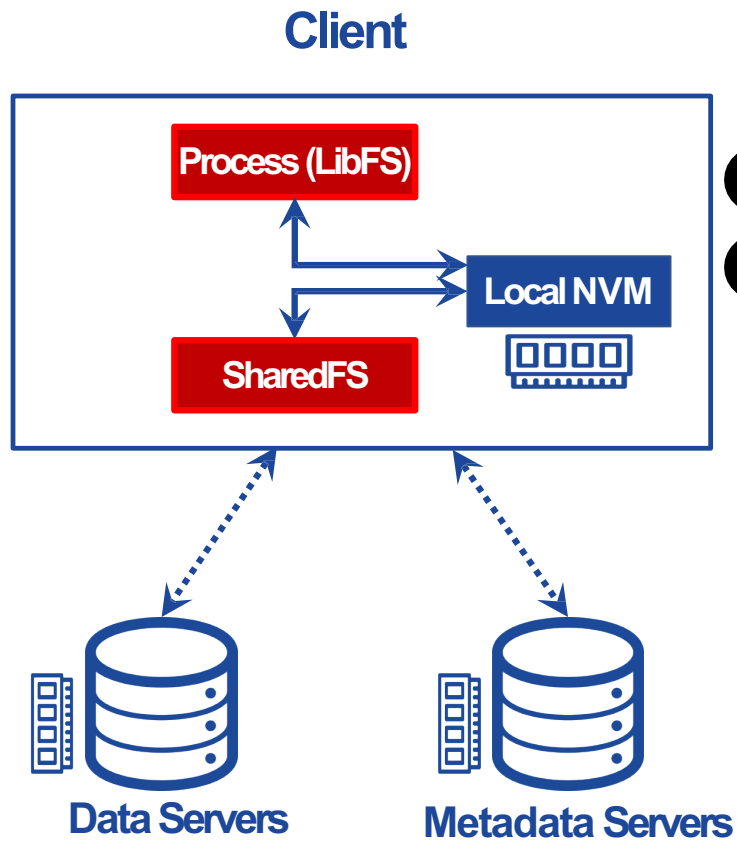
1 Store (meta-)data in client-local NVM

Client-local IO with Assise



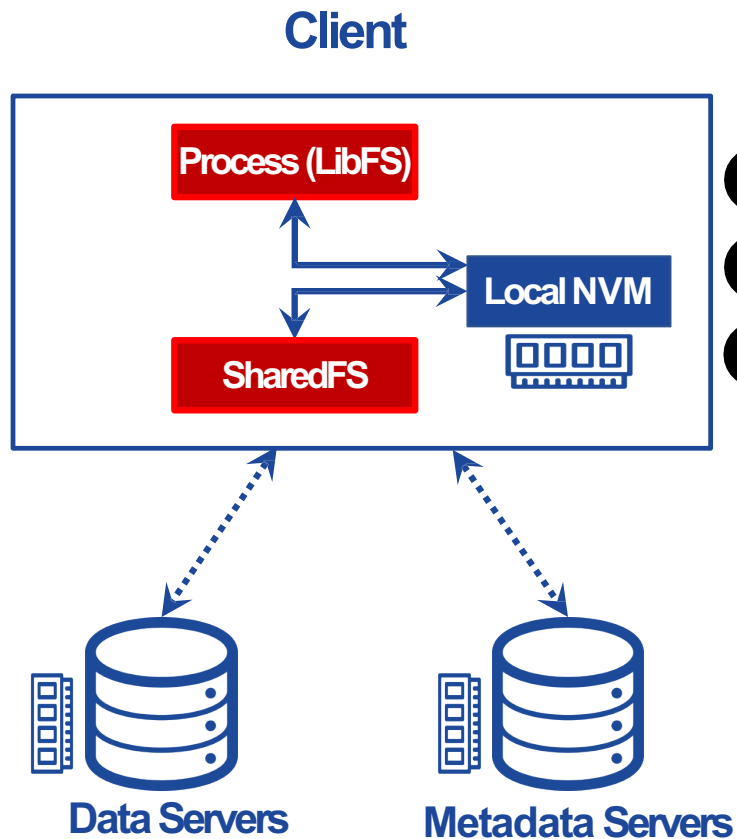
1 Store (meta-)data in client-local NVM

Client-local IO with Assise



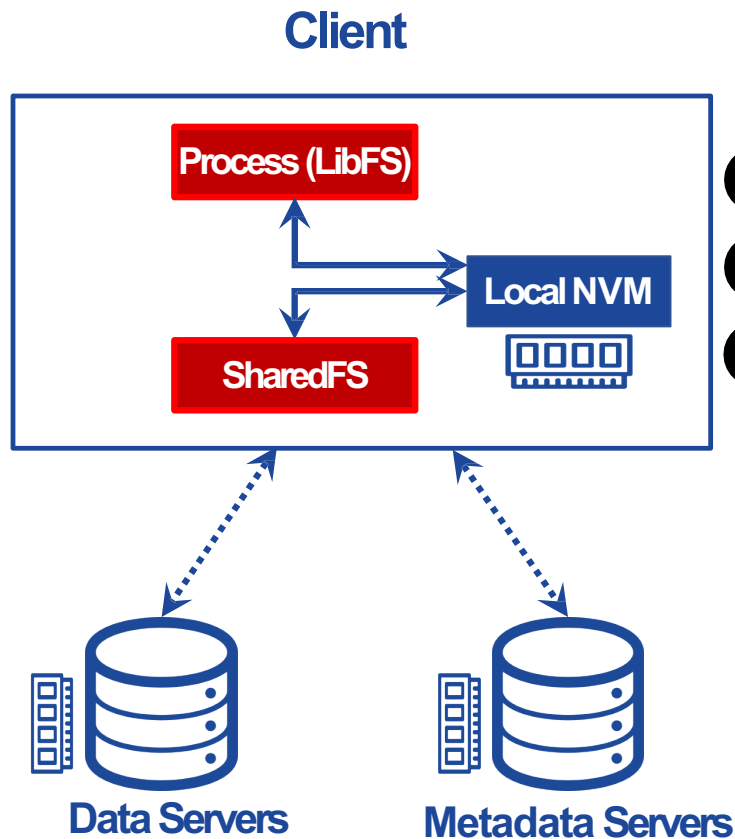
- 1 Store (meta-)data in client-local NVM
- 2 Userspace access via LibFS

Client-local IO with Assise



- ❶ Store (meta-)data in client-local NVM
- ❷ Userspace access via LibFS
- ❸ Logwrites at operation granularity

Client-local IO with Assise



- ❶ Store (meta-)data in client-local NVM
- ❷ Userspace access via LibFS
- ❸ Logwrites at operation granularity

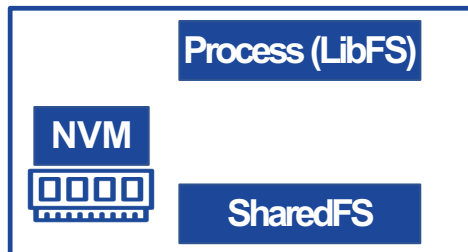
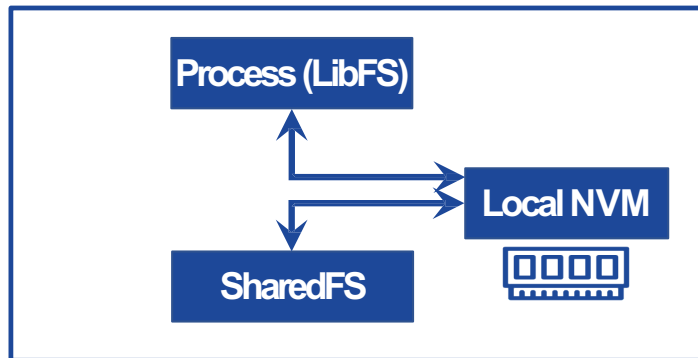
Local userspace data access (= low latency IO)

No block amplification (= efficient IO)

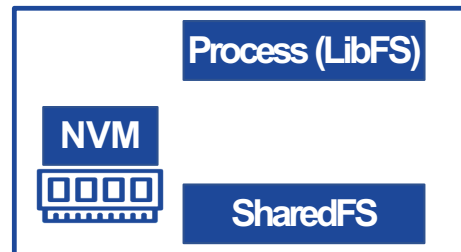
Local durability (= fast, local recovery)

Remote IO with Assise

Client 1

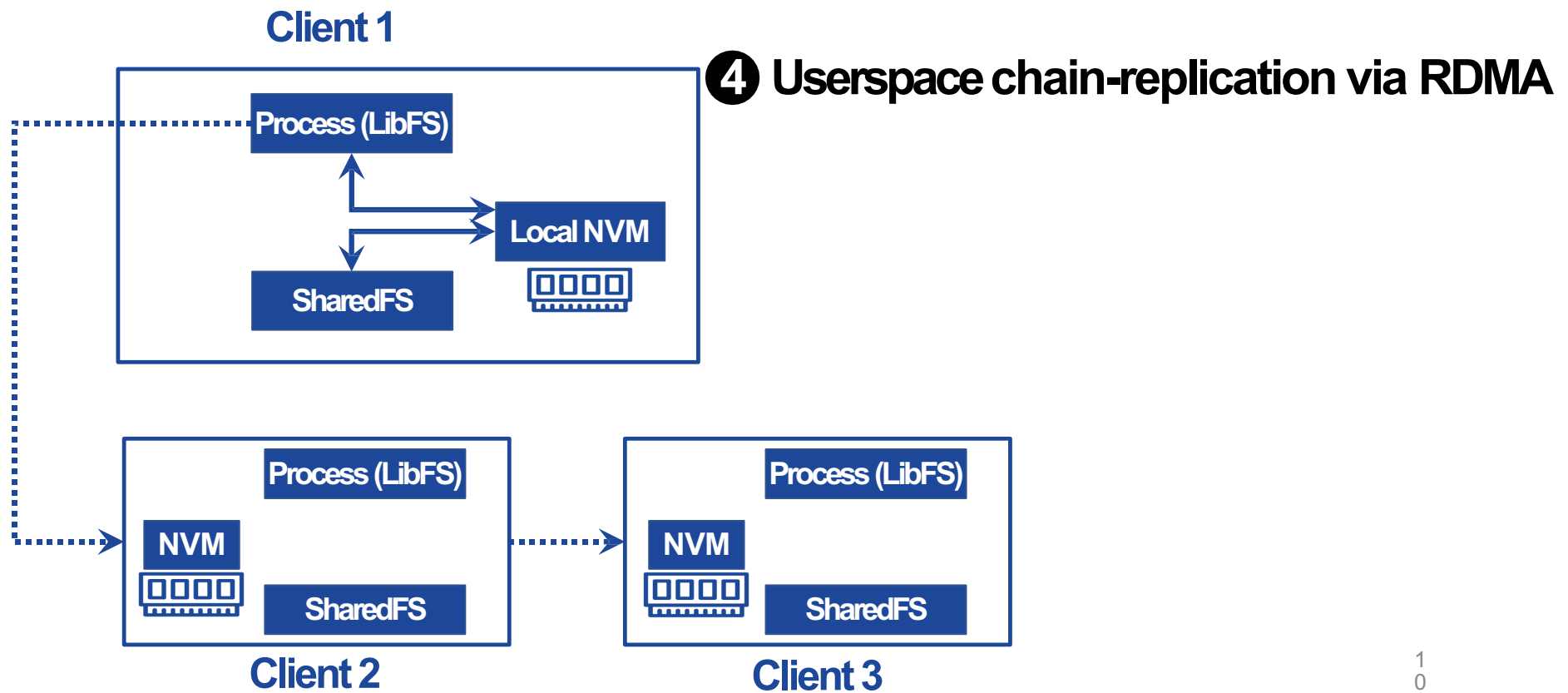


Client 2

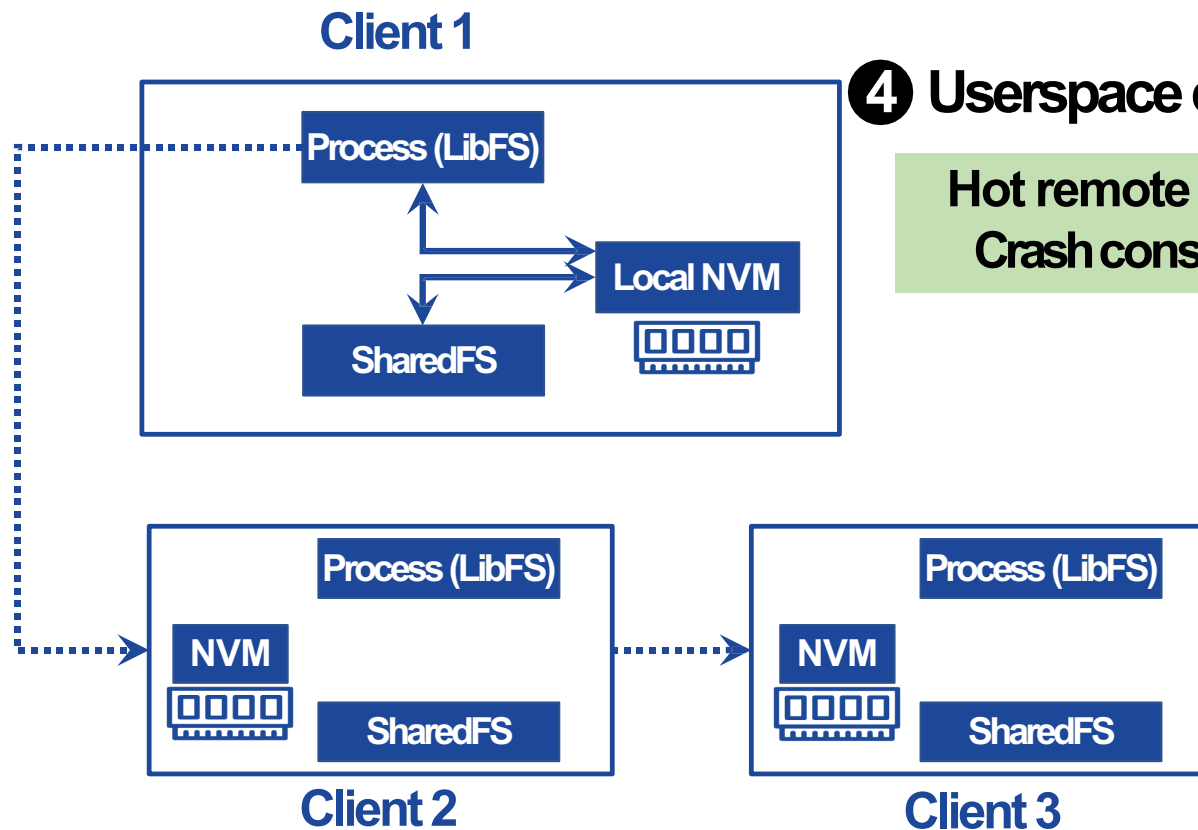


Client 3

Remote IO with Assise



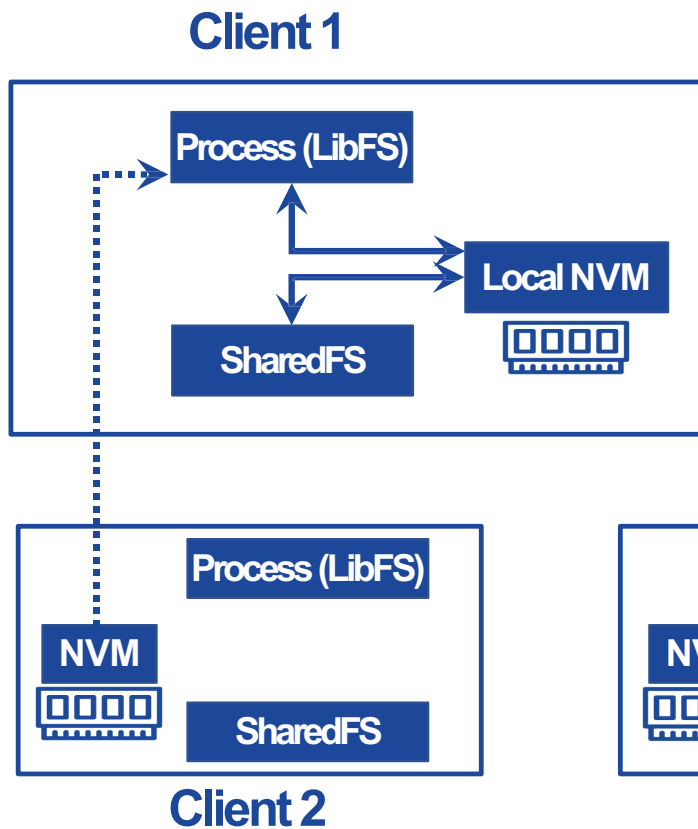
Remote IO with Assise



④ Userspace chain-replication via RDMA

Hot remote client caches (= fast fail-over)
Crash consistent via RDMA write order

Remote IO with Assise



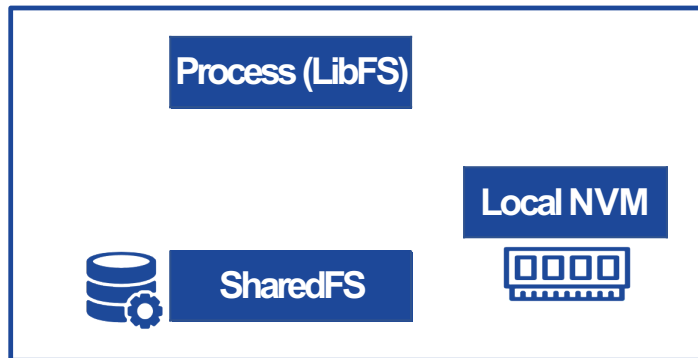
④ Userspace chain-replication via RDMA

Hot remote client caches (= fast fail-over)
Crash consistent via RDMA write order

⑤ RPC via RDMA for read misses in client-local caches

CC-NVM: Linearizability with client-local IO

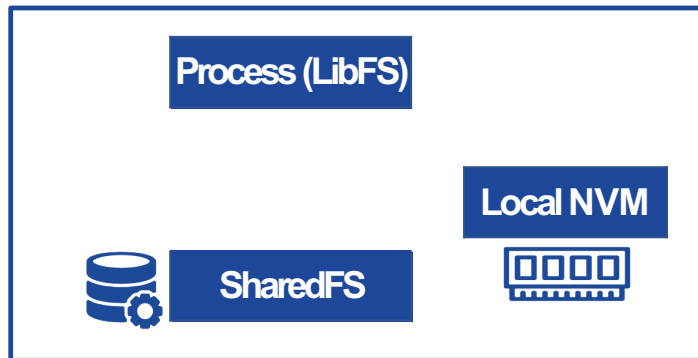
Client 1



- ⑥ CC-NVM provides linearizability by granting leases to processes

CC-NVM: Linearizability with client-local IO

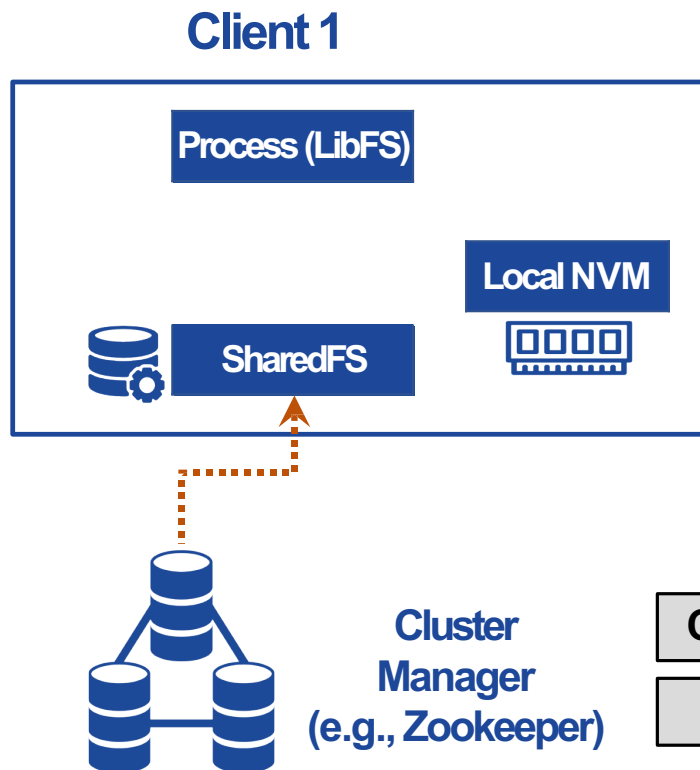
Client 1



⑥ CC-NVM provides linearizability by granting leases to processes

Example: Create file **/home/alice/mail1**

CC-NVM: Linearizability with client-local IO



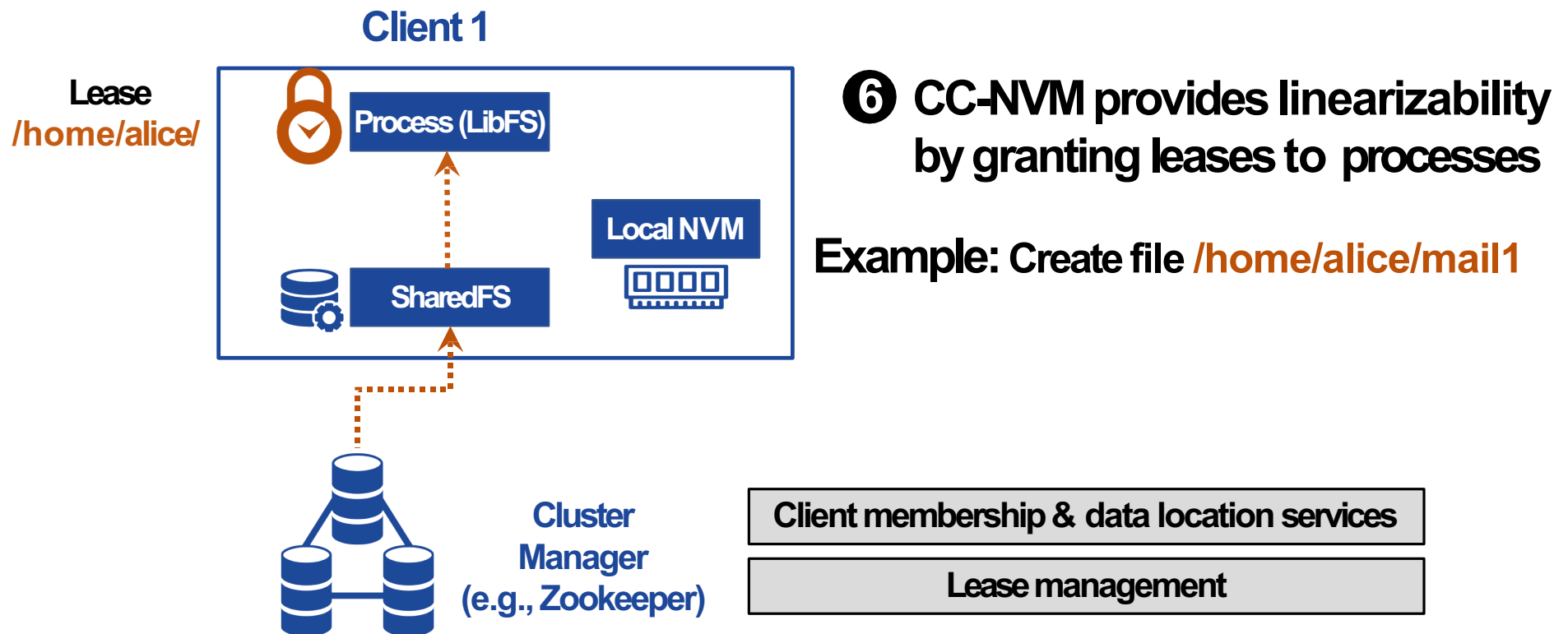
⑥ CC-NVM provides linearizability by granting leases to processes

Example: Create file **/home/alice/mail1**

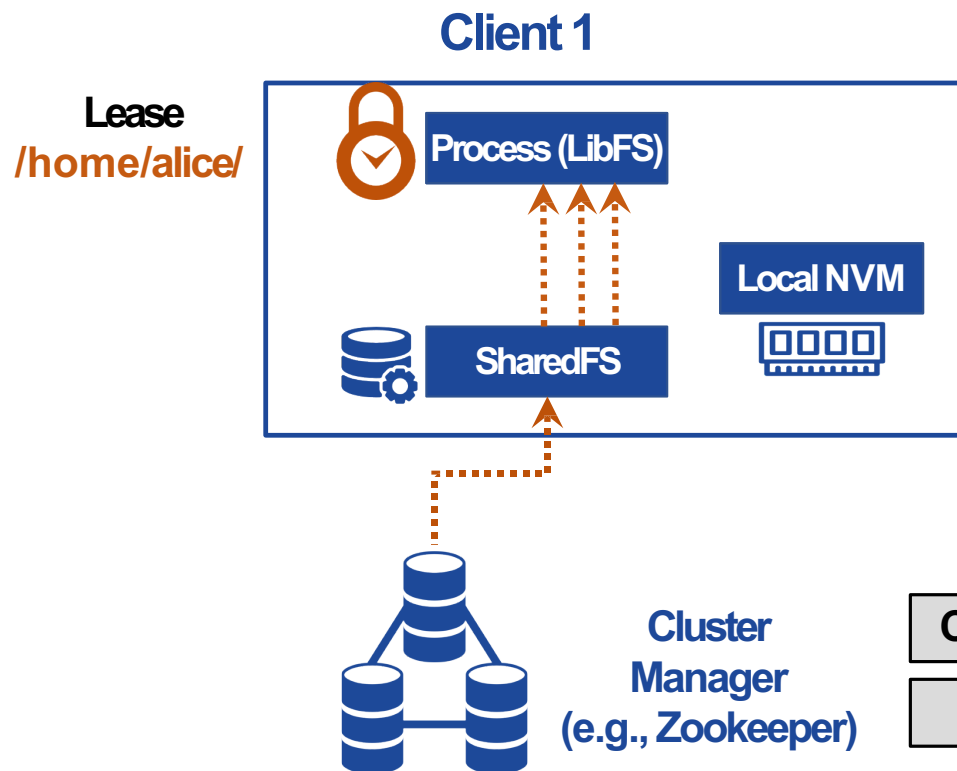
Client membership & data location services

Lease management

CC-NVM: Linearizability with client-local IO



CC-NVM: Linearizability with client-local IO



⑥ CC-NVM provides linearizability by granting leases to processes

Example: Create file **/home/alice/mail1**
Create file **/home/alice/mail2**
Create file **/home/alice/mail3**

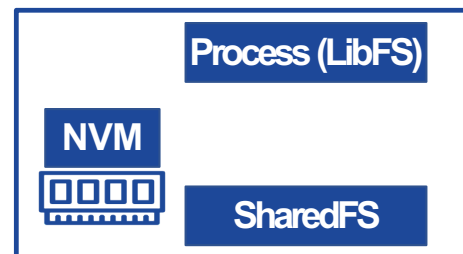
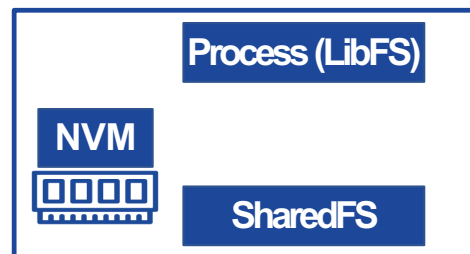
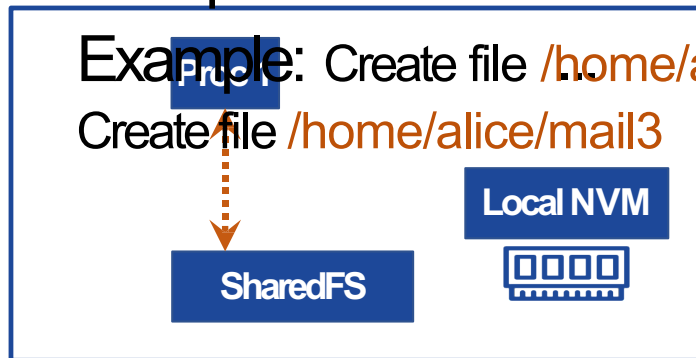
Client membership & data location services

Lease management

CC-NVM: Scalable shared file system access

- CC-NVM provides scalable access via hierarchical lease delegation

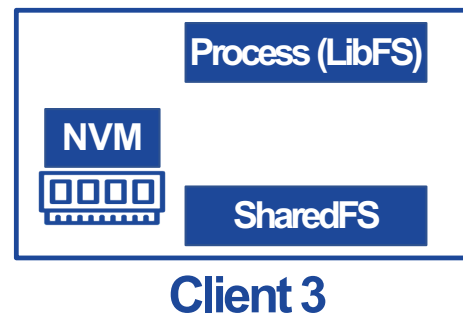
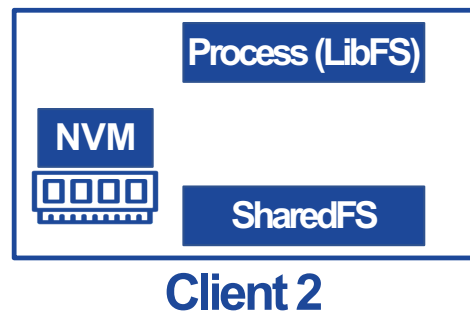
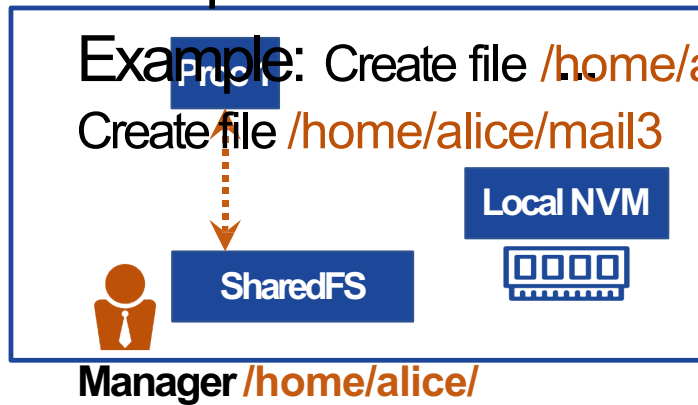
Example: Create file `/home/alice/mail1` Create file `/home/alice/mail2`
Create file `/home/alice/mail3`



CC-NVM: Scalable shared file system access

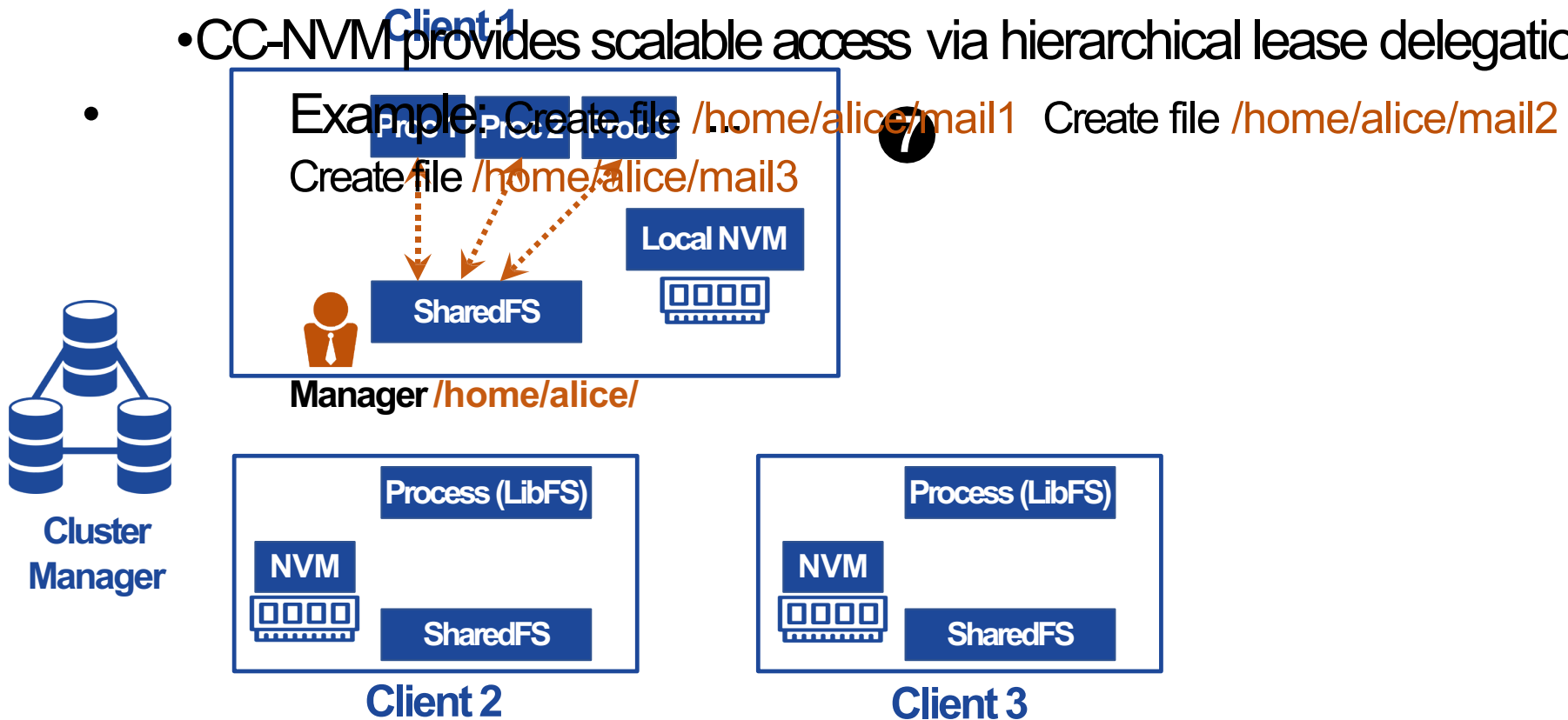
- CC-NVM provides scalable access via hierarchical lease delegation

Example: Create file `/home/alice/mail1` Create file `/home/alice/mail2`
Create file `/home/alice/mail3`



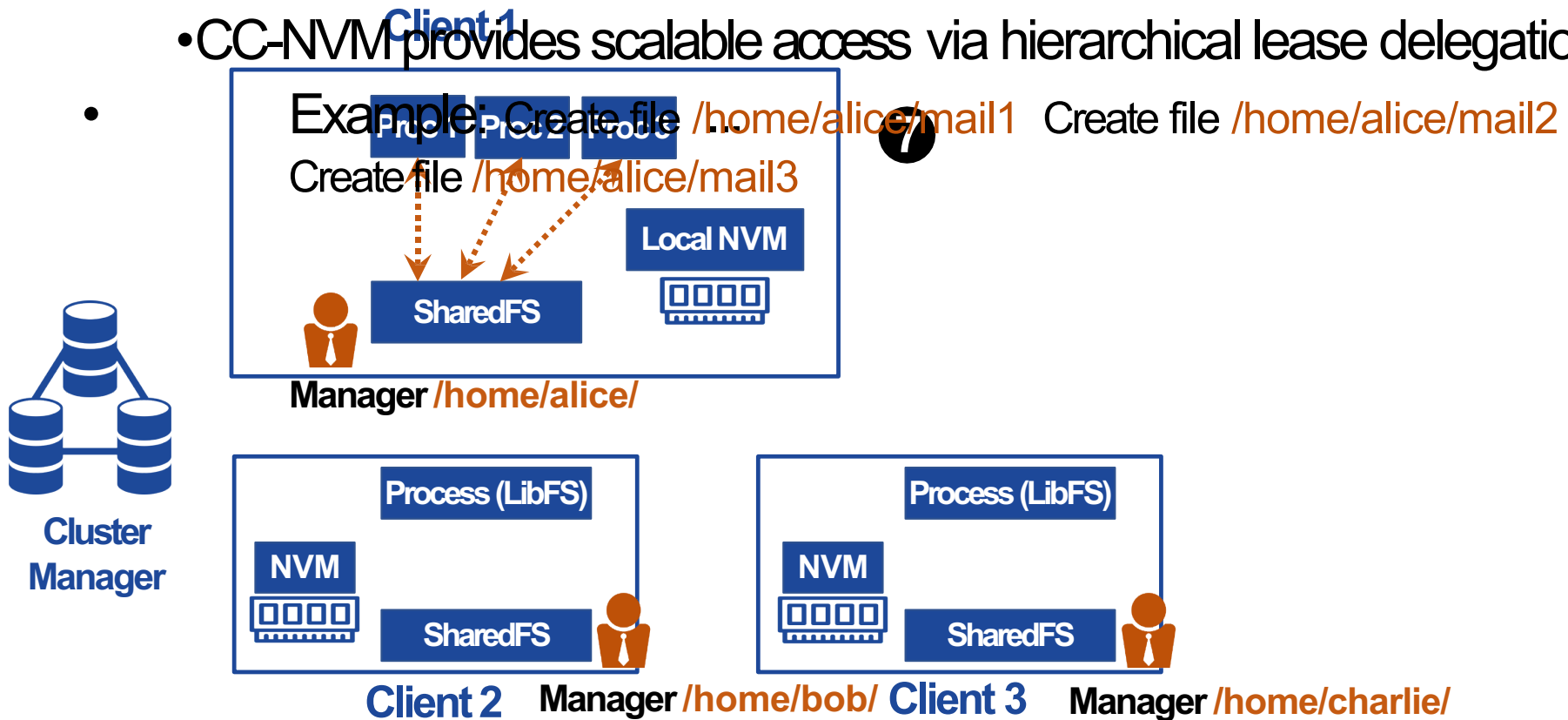
CC-NVM: Scalable shared file system access

- CC-NVM provides scalable access via hierarchical lease delegation



CC-NVM: Scalable shared file system access

- CC-NVM provides scalable access via hierarchical lease delegation



Assise: Design summary

Main principle: **Maximize use of client-local NVM**

1. **Low latency IO**: Perform process-local & client-local IO
2. **Linearizability and data crash consistency**
CC-NVM distributed client-side NVM coherence protocol
3. **High availability**
Fail-over to cache-hot client replicas, recover client NVMcaches
4. Exploit NUMA locality
5. Efficient NVM and network bandwidth use
6. Low-cost cold storage

Assise: Design summary

Main principle: **Maximize use of client-local NVM**

1. **Low latency IO**: Perform process-local & client-local IO
2. **Linearizability and data crash consistency**
CC-NVM distributed client-side NVM coherence protocol
3. **High availability**
Fail-over to cache-hot client replicas, recover client NVMcaches
4. **Exploit NUMA locality**
5. **Efficient NVM and network bandwidth use**
6. **Low-cost cold storage**

Evaluation

Evaluation questions

- **Latency:** How close is Assise to raw, replicated NVM IO latency? How much faster is Assise than the state-of-the-art?
- **Scalability:** By how much can CC-NVM improve multi-process and multi-node scalability?
- **Availability:** How quickly can Assise fail-over?
- More results in the paper
 - Large-scale distributed storage, cloud application benchmarks, re-establishing the replication factor, ...

Experimental Setup

- **5× dual-socket Cascade Lake-SP servers with 48 cores @ 2.2GHz**
 - All nodes connect to an InfiniBand switch via **40 Gbps ConnectX-3 NICs**
- Each node has **6 TB of Optane DCNVM** and **384 GB of DRAM**
 - 6x DIMMs of NVM and DRAM per NUMA node

Experimental baselines – Properties

Feature	Assise	CephFS	NFS	ATC'17	FAST'19
				Octopus	Orion
Local consistency	✓				
Kernel bypass	✓				
Linearizability	✓				✓
Byte-oriented	✓			✓	✓
Replication	✓	✓			✓
RDMA	✓	✓	✓	✓	✓

Experimental baselines – Properties

Feature	Assise	CephFS	NFS	ATC'17	FAST'19
				Octopus	Orion
Local consistency	✓				
Kernel bypass	✓				
Linearizability	✓				✓
Byte-oriented	✓			✓	✓
Replication	✓	✓			✓
RDMA	✓	✓	✓	✓	✓

Experimental baselines – Properties

Feature	Assise	CephFS	NFS	ATC'17	FAST'19
				Octopus	Orion
Local consistency	✓				
Kernel bypass	✓				
Linearizability	✓				✓
Byte-oriented	✓			✓	✓
Replication	✓	✓			✓
RDMA	✓	✓	✓	✓	✓

Experimental baselines – Properties

Feature	Assise	CephFS	NFS	ATC'17	FAST'19
				Octopus	Orion
Local consistency	✓				
Kernel bypass	✓				
Linearizability	✓				✓
Byte-oriented	✓			✓	✓
Replication	✓	✓			✓
RDMA	✓	✓	✓	✓	✓

Experimental baselines – Properties

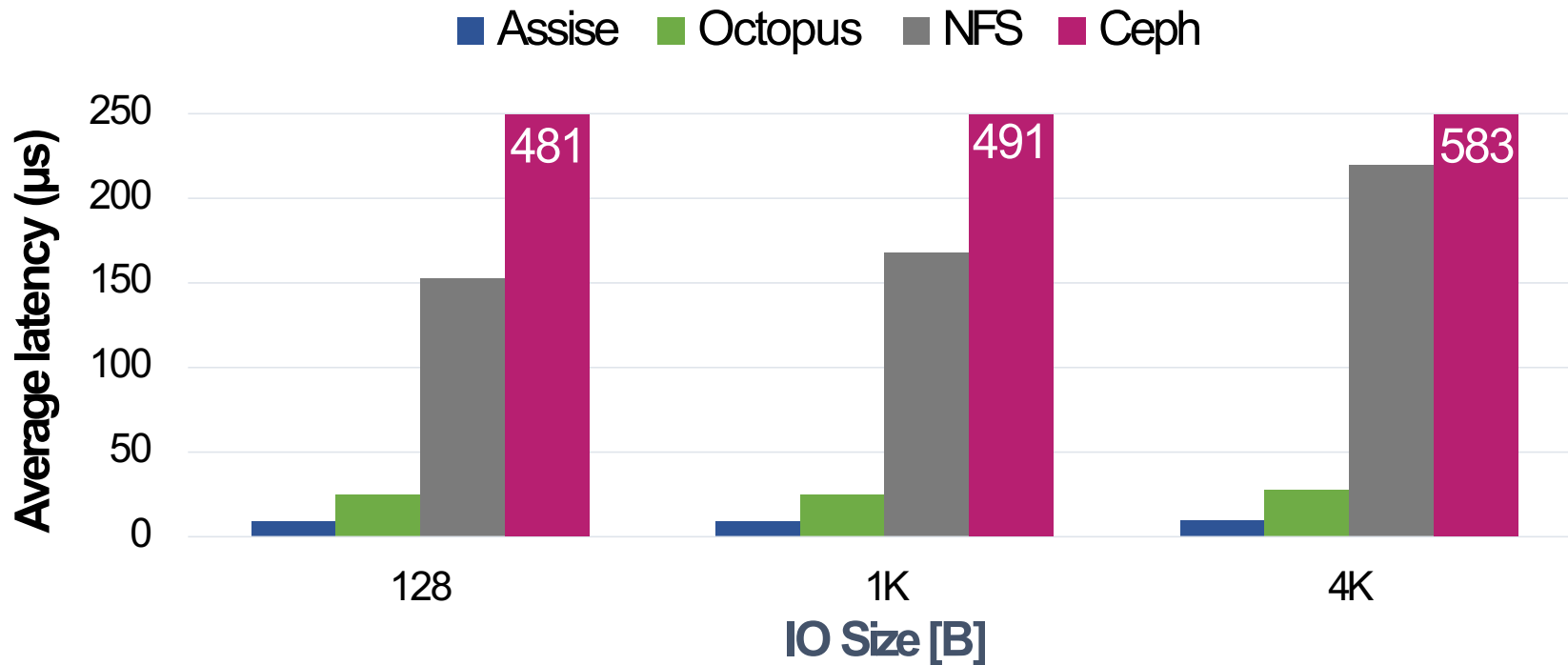
Feature	Assise	CephFS	NFS	ATC'17	FAST'19
				Octopus	Orion
Local consistency	✓				
Kernel bypass	✓				
Linearizability	✓				✓
Byte-oriented	✓			✓	✓
Replication	✓	✓			✓
RDMA	✓	✓	✓	✓	✓

Experimental baselines – Properties

Feature	Assise	CephFS	NFS	ATC'17	FAST'19
				Octopus	Orion
Local consistency	✓				
Kernel bypass	✓				
Linearizability	✓				✓
Byte-oriented	✓			✓	✓
Replication	✓	✓			✓
RDMA	✓	✓	✓	✓	✓

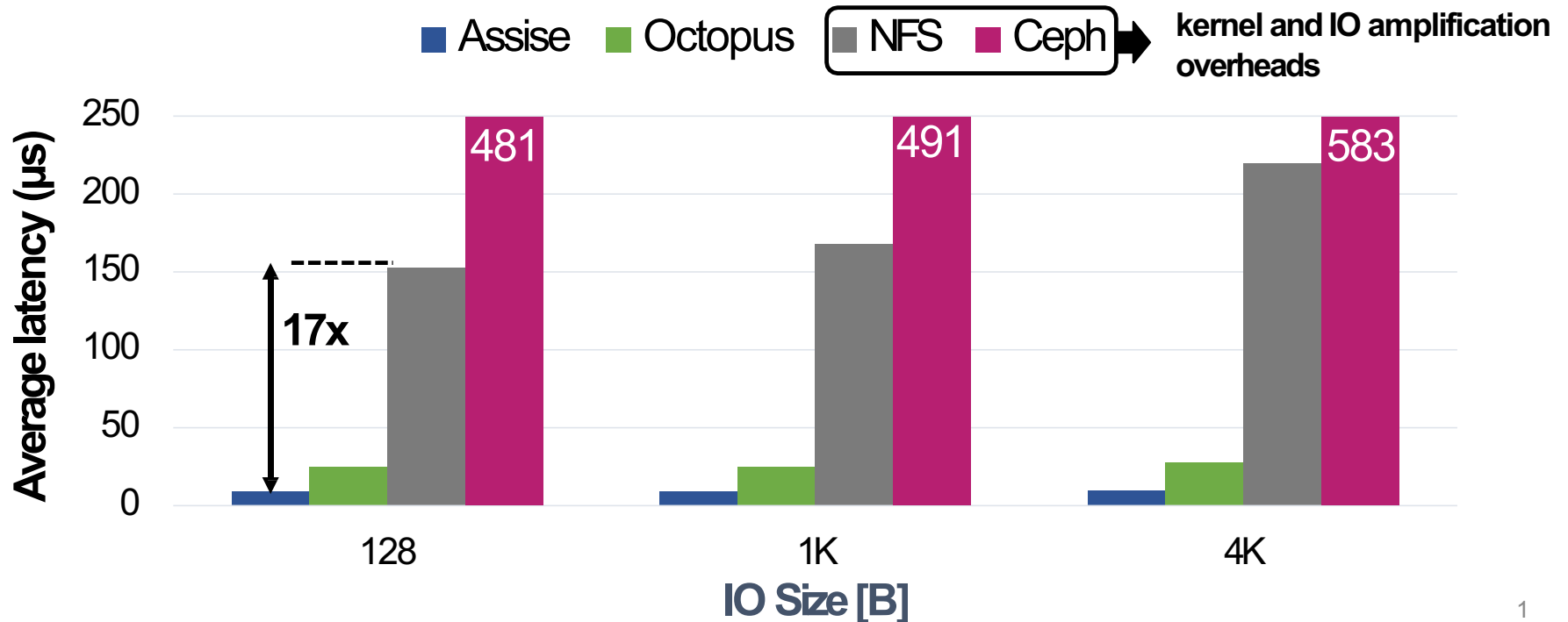
Microbenchmark: Write Latency

- 1GB burst of synchronous writes to a single file
 - Assise and Ceph replicate to an additional node



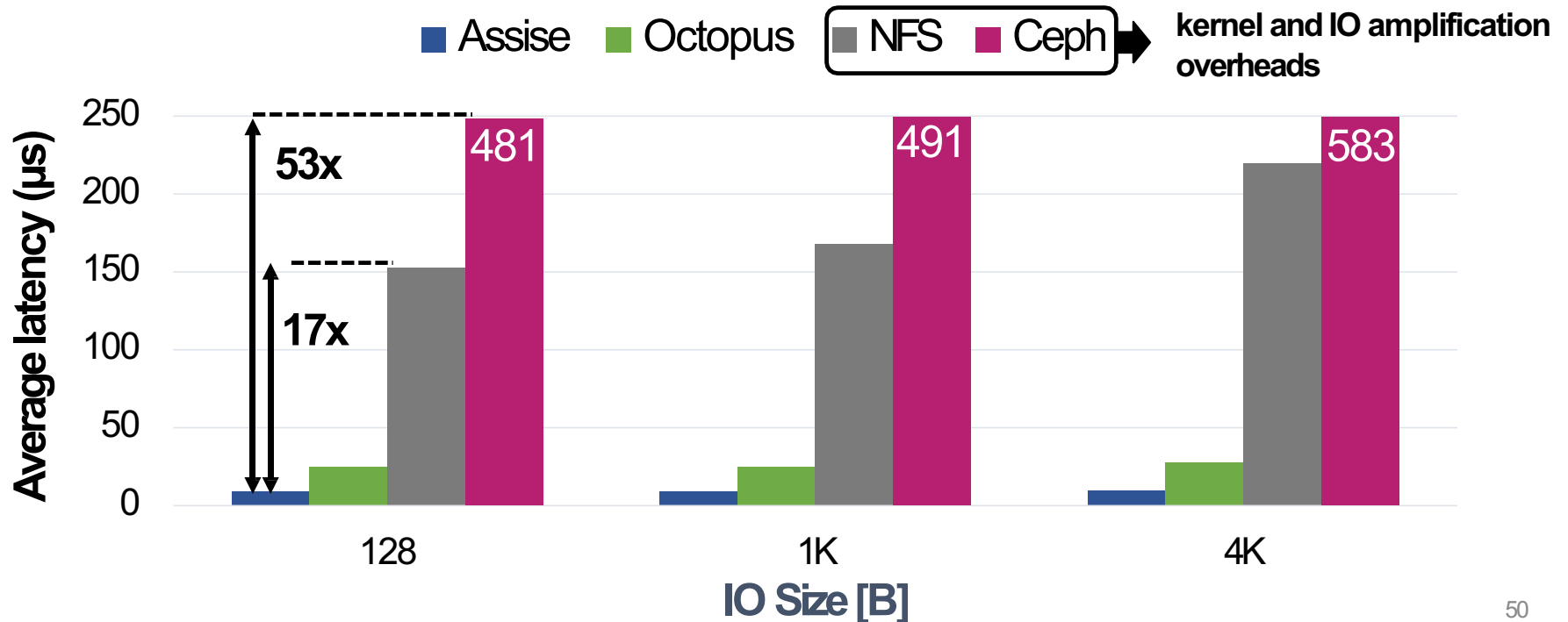
Microbenchmark: Write Latency

- 1GB burst of synchronous writes to a single file
 - Assise and Ceph replicate to an additional node



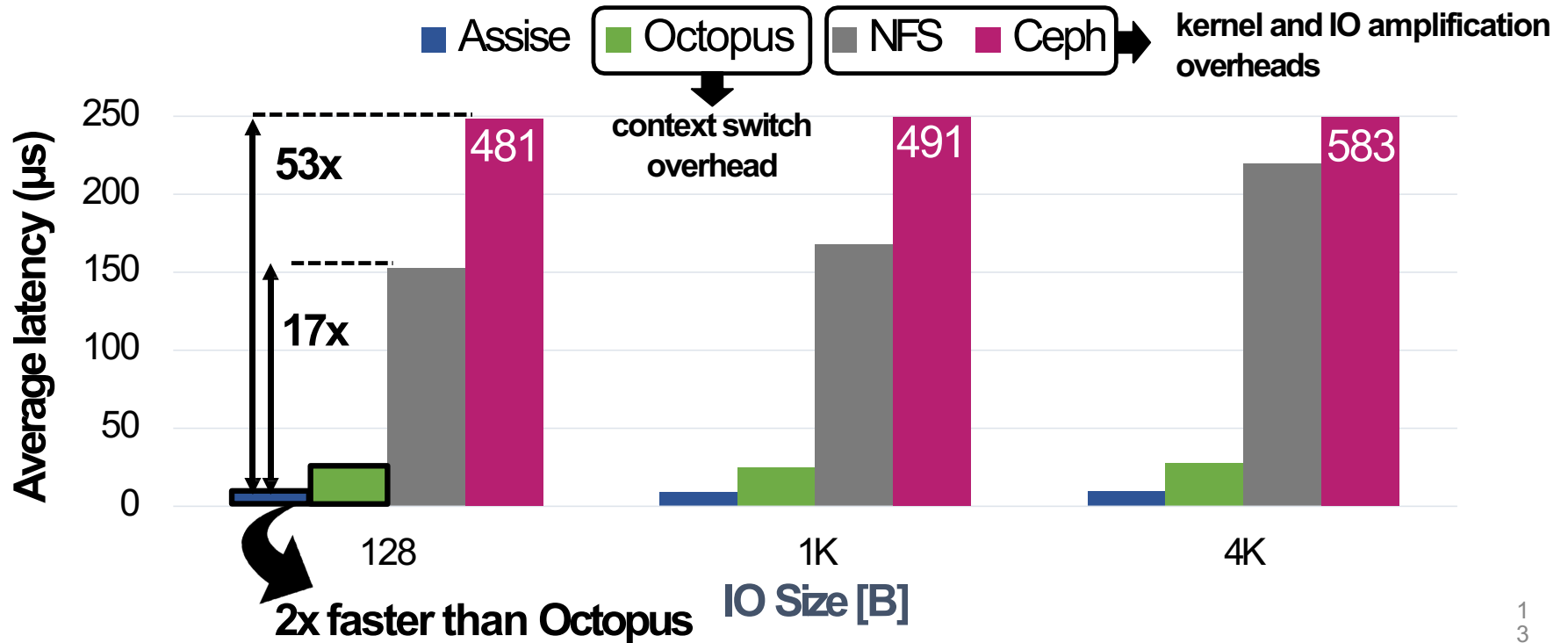
Microbenchmark: Write Latency

- 1GB burst of synchronous writes to a single file
 - Assise and Ceph replicate to an additional node



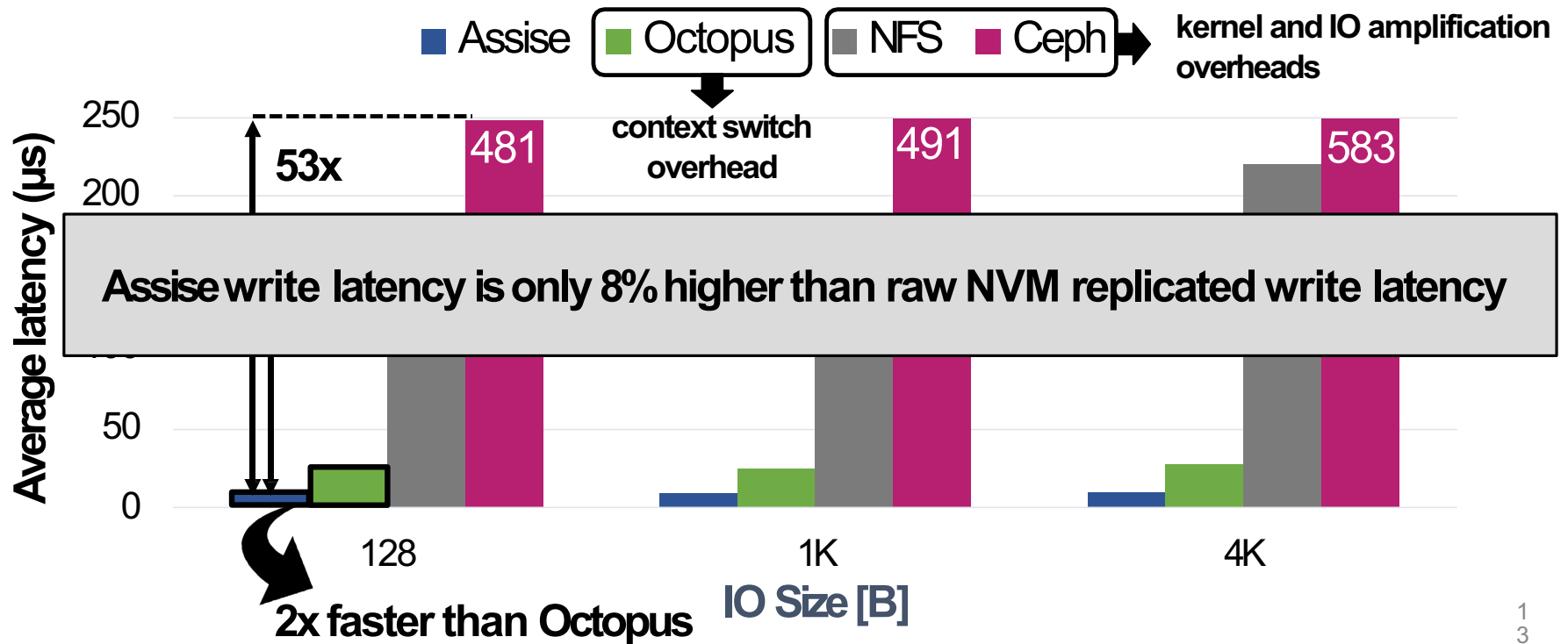
Microbenchmark: Write Latency

- 1GB burst of synchronous writes to a single file
 - Assise and Ceph replicate to an additional node



Microbenchmark: Write Latency

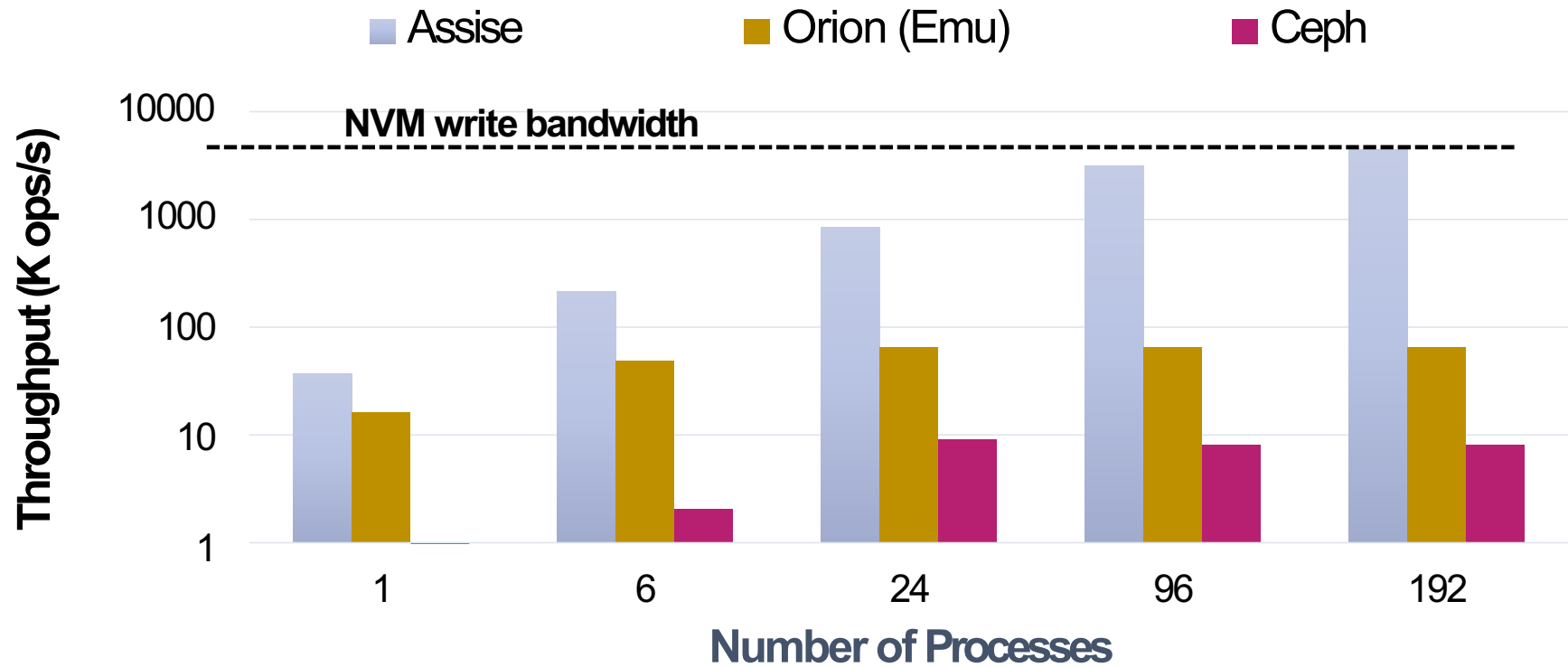
- 1GB burst of synchronous writes to a single file
 - Assise and Ceph replicate to an additional node



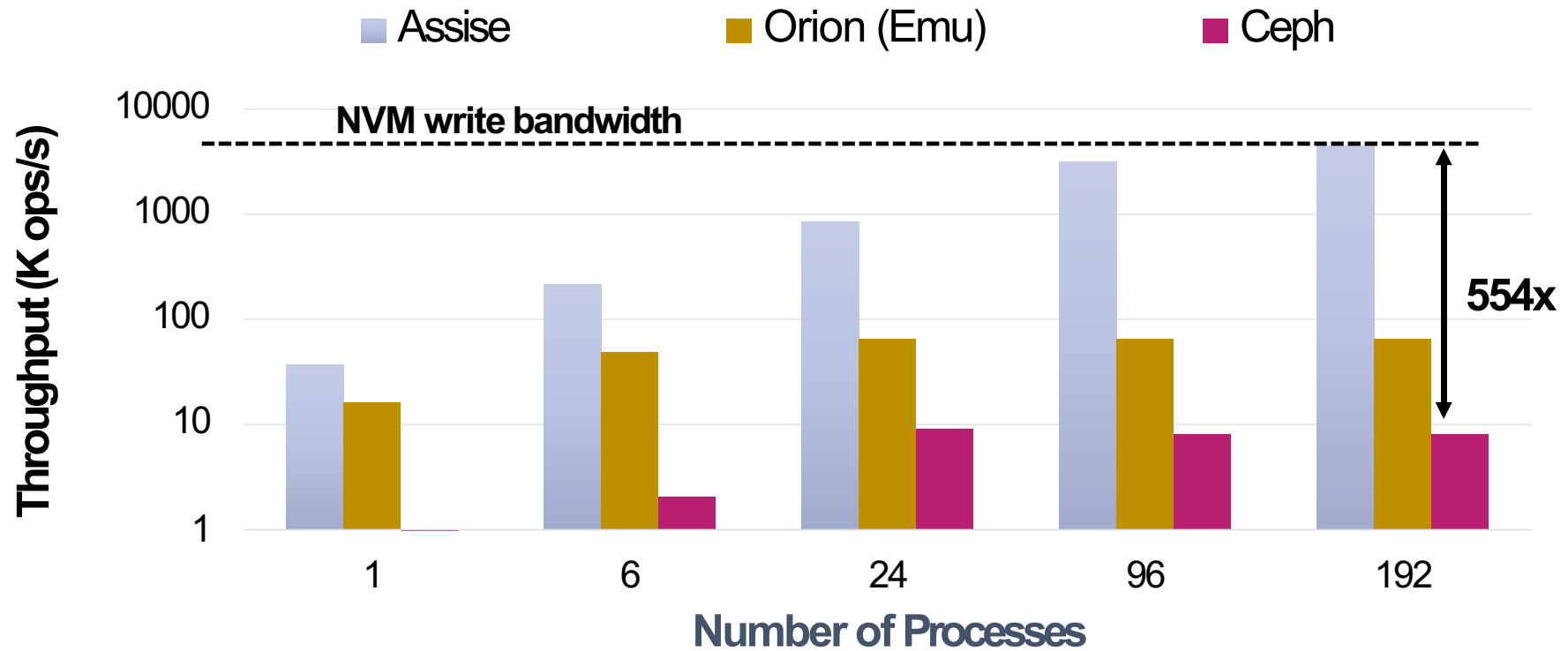
Microbenchmark: Scalability

- Multi-process benchmark to test scalability
 - Data & metadata operations (create file, write 4K, rename file)
 - **Embarrassingly parallel**: Processes write to their own private sub-directories
 - Replication is disabled: Eliminate network bottlenecks of our testbed
- Benchmark runs on 3 testbed nodes
 - Balanced deployment of processes over nodes

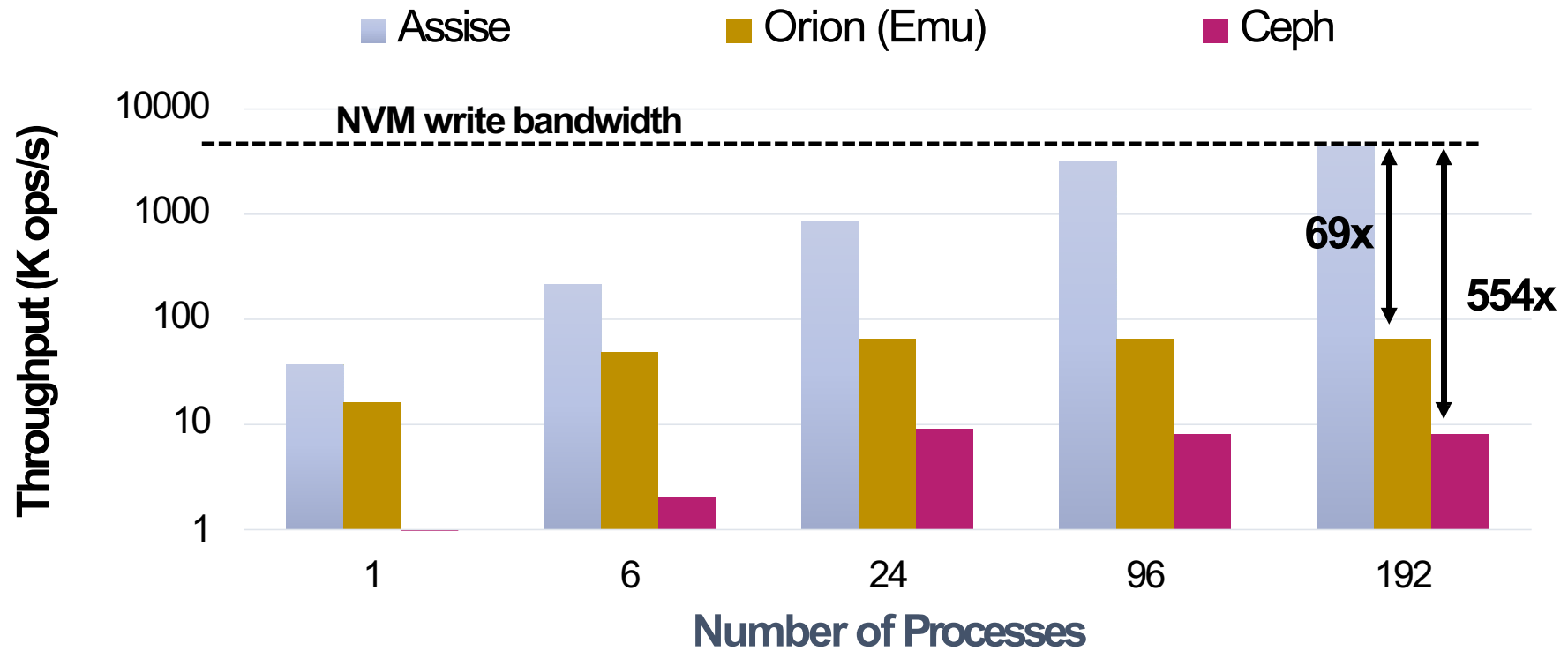
Microbenchmark: Scalability



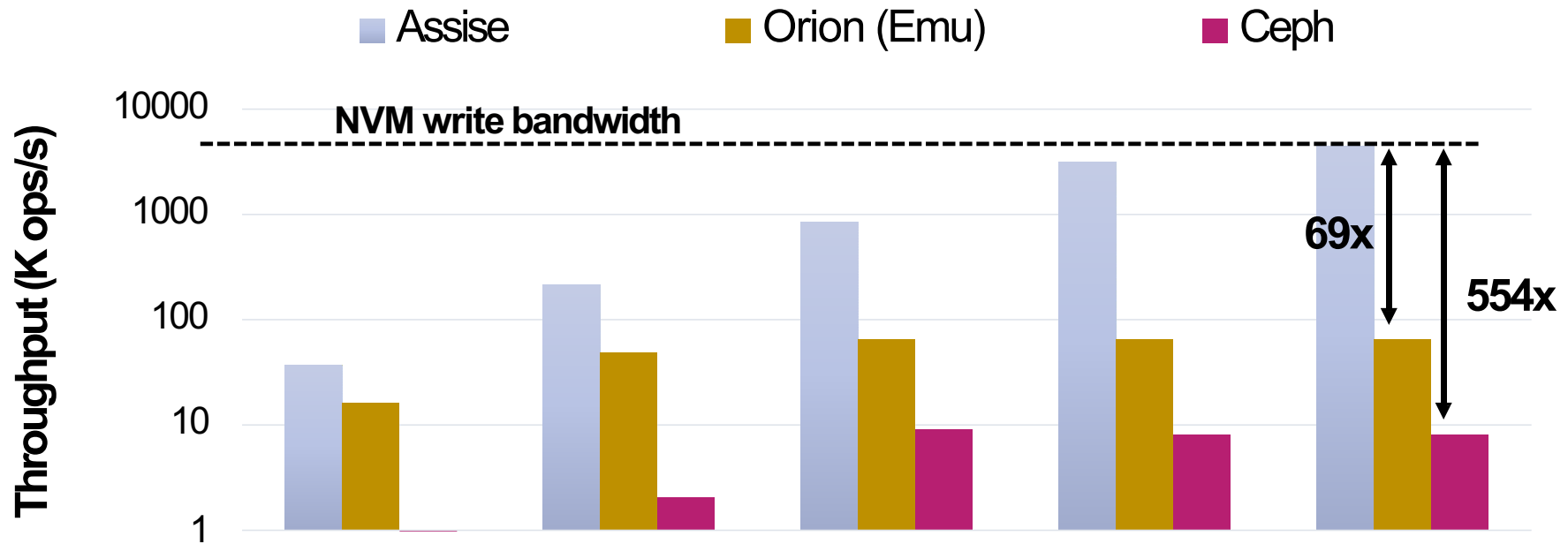
Microbenchmark: Scalability



Microbenchmark: Scalability



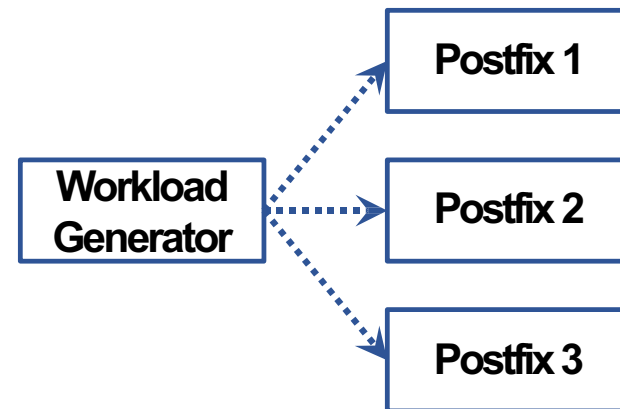
Microbenchmark: Scalability



Assise scales because of client-local synchronization, provided by CC-NVM

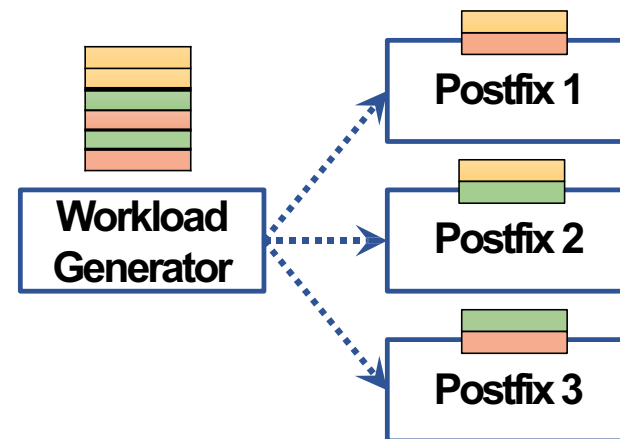
Postfix: Scalable Mail Delivery

- We replay 70GB worth of emails to a pool of Postfix Mail Delivery Agents
 - Deliver to Maildir: Each inbox is a directory, contains 1 file per mail message
 - Real email trace; average mail size is 200 KB
- We use 3 nodes for mail delivery and 1 node for workload generation
 - Delivery processes are distributed uniformly over nodes
- Mail delivery load balancing
 1. Round robin
 2. Sharded across clients by recipient



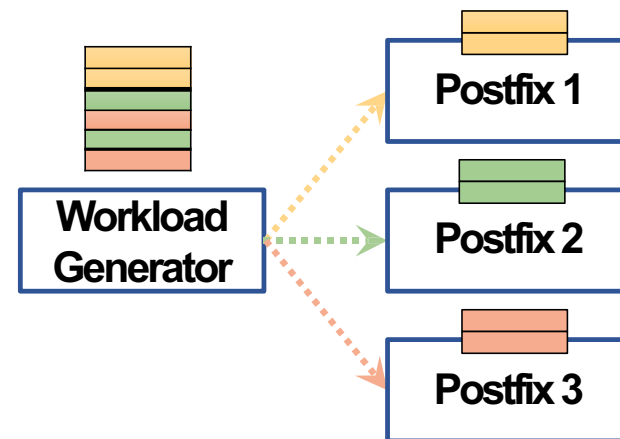
Postfix: Scalable Mail Delivery

- We replay 70GB worth of emails to a pool of Postfix Mail Delivery Agents
 - Deliver to Maildir: Each inbox is a directory, contains 1 file per mail message
 - Real email trace; average mail size is 200 KB
- We use 3 nodes for mail delivery and 1 node for workload generation
 - Delivery processes are distributed uniformly over nodes
- Mail delivery load balancing
 1. Round robin
 2. Sharded across clients by recipient

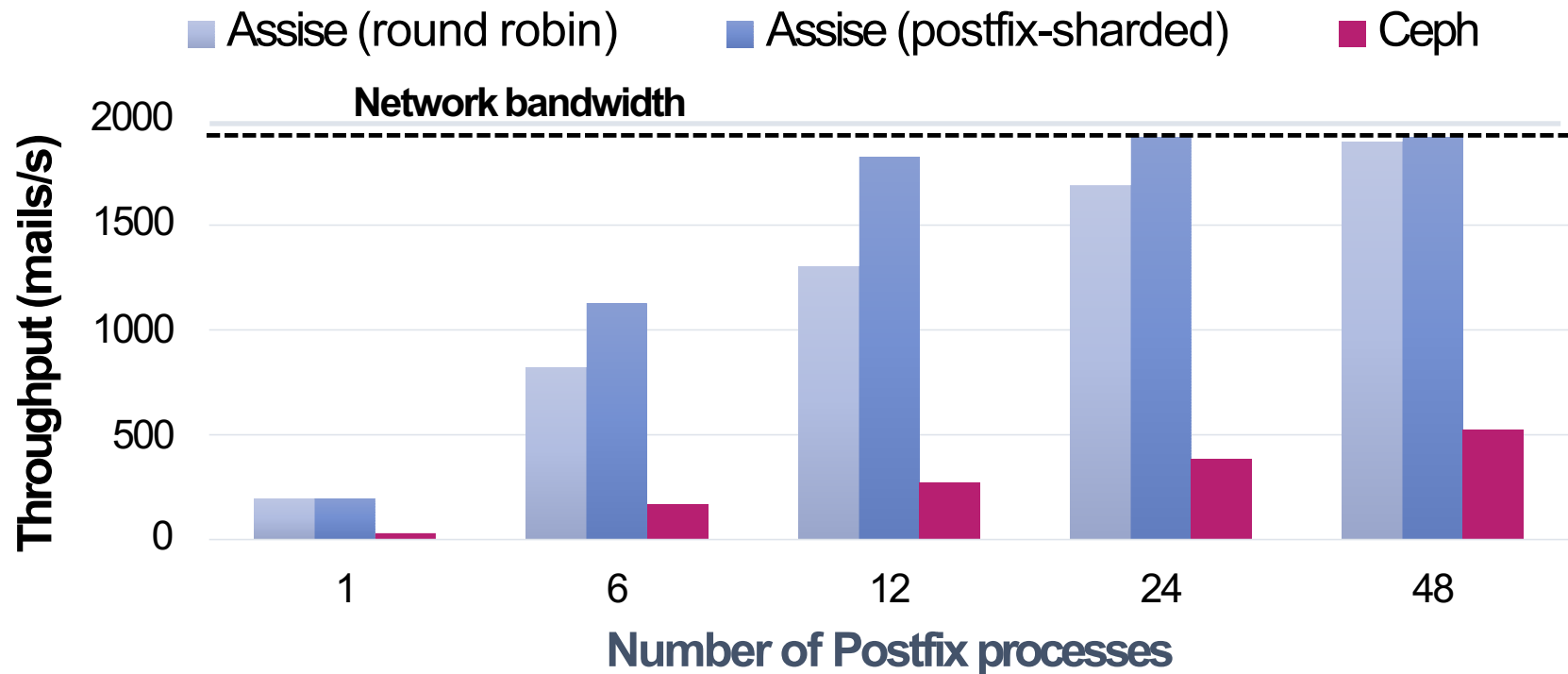


Postfix: Scalable Mail Delivery

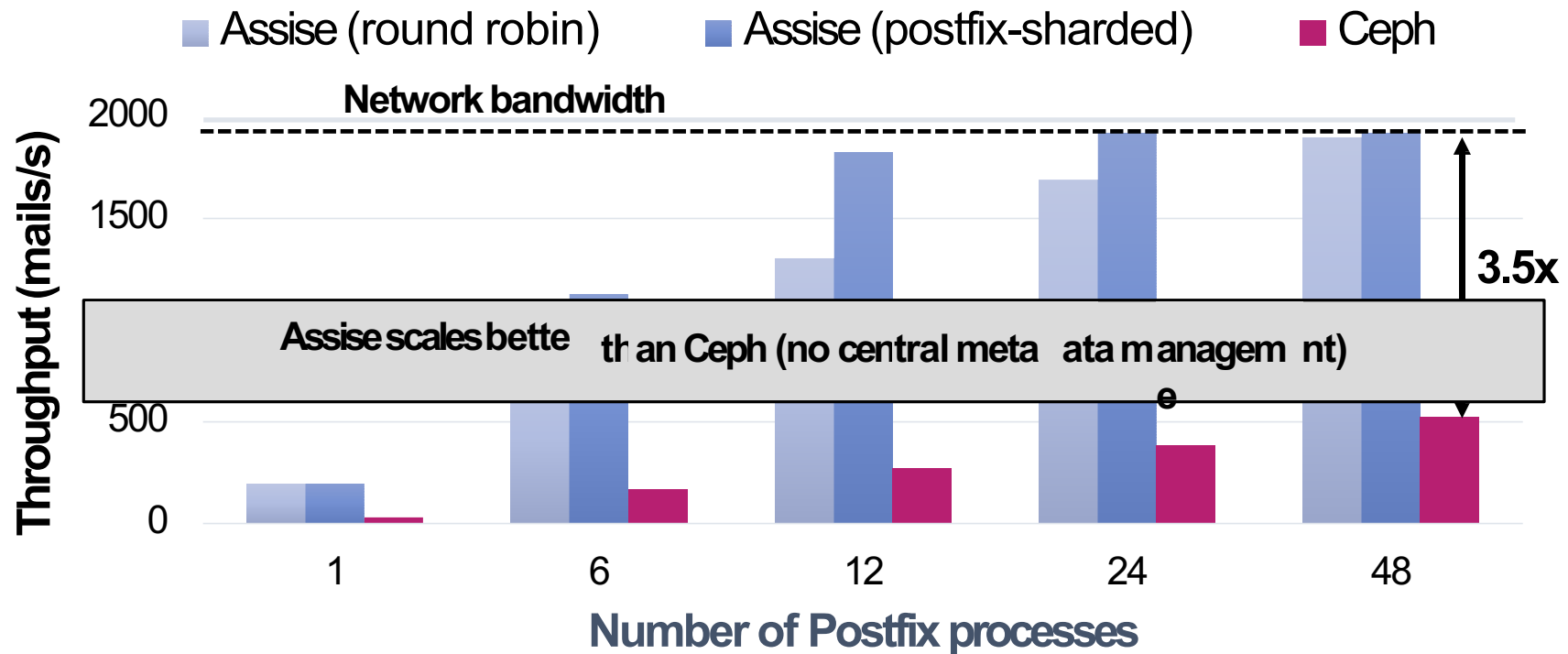
- We replay 70GB worth of emails to a pool of Postfix Mail Delivery Agents
 - Deliver to Maildir: Each inbox is a directory, contains 1 file per mail message
 - Real email trace; average mail size is 200 KB
- We use 3 nodes for mail delivery and 1 node for workload generation
 - Delivery processes are distributed uniformly over nodes
- Mail delivery load balancing
 1. Round robin
 2. Sharded across clients by recipient



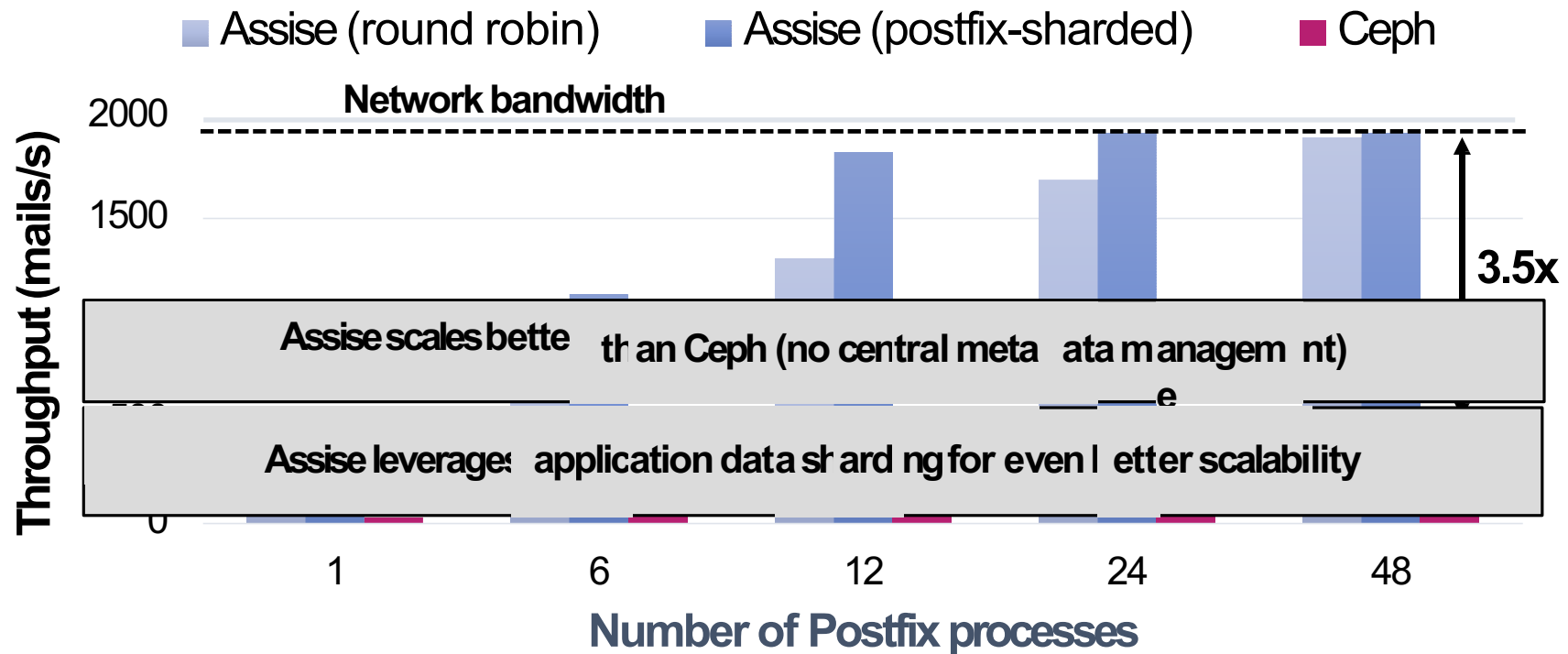
Postfix: Scalable Mail Delivery



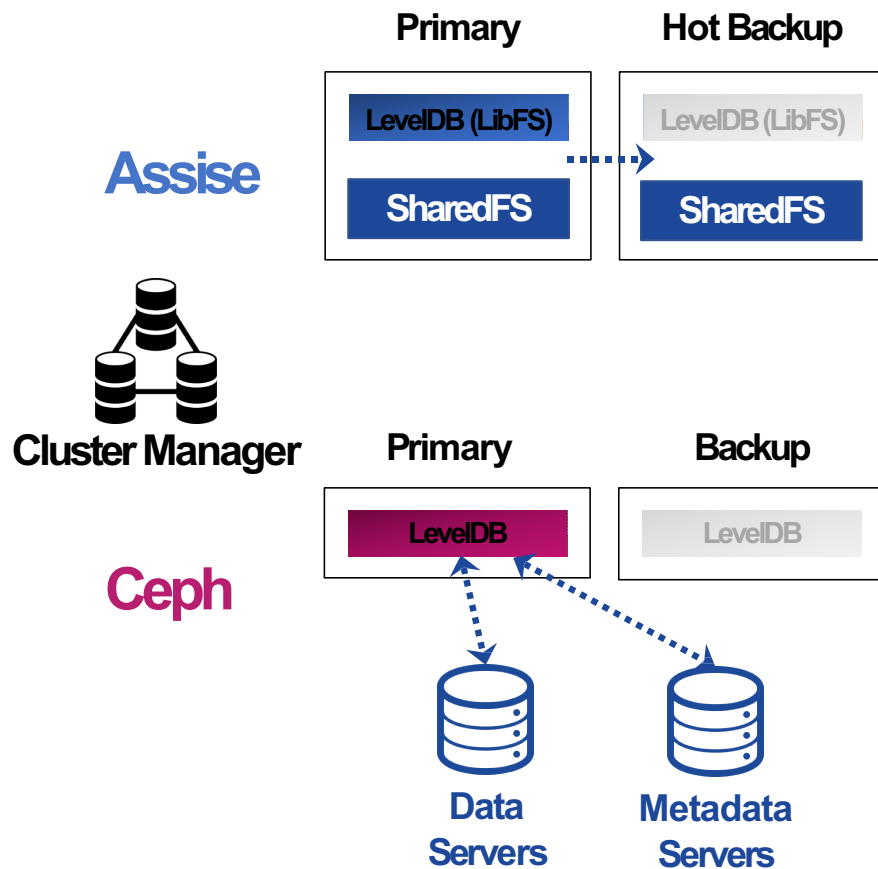
Postfix: Scalable Mail Delivery



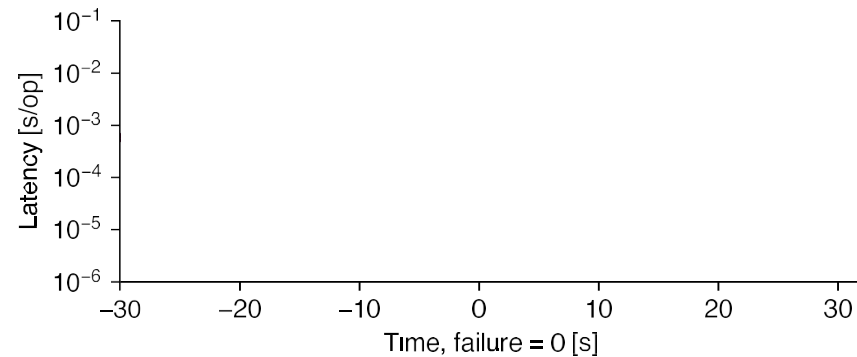
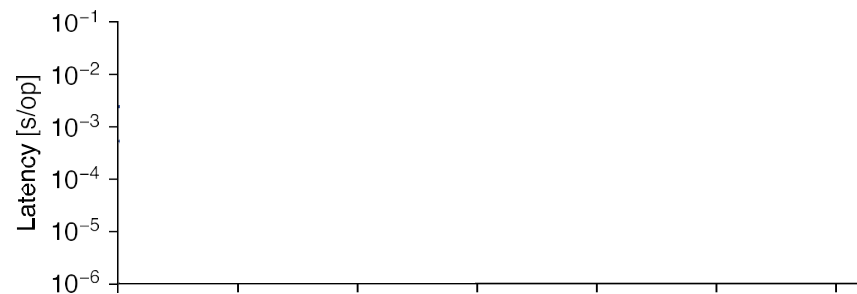
Postfix: Scalable Mail Delivery



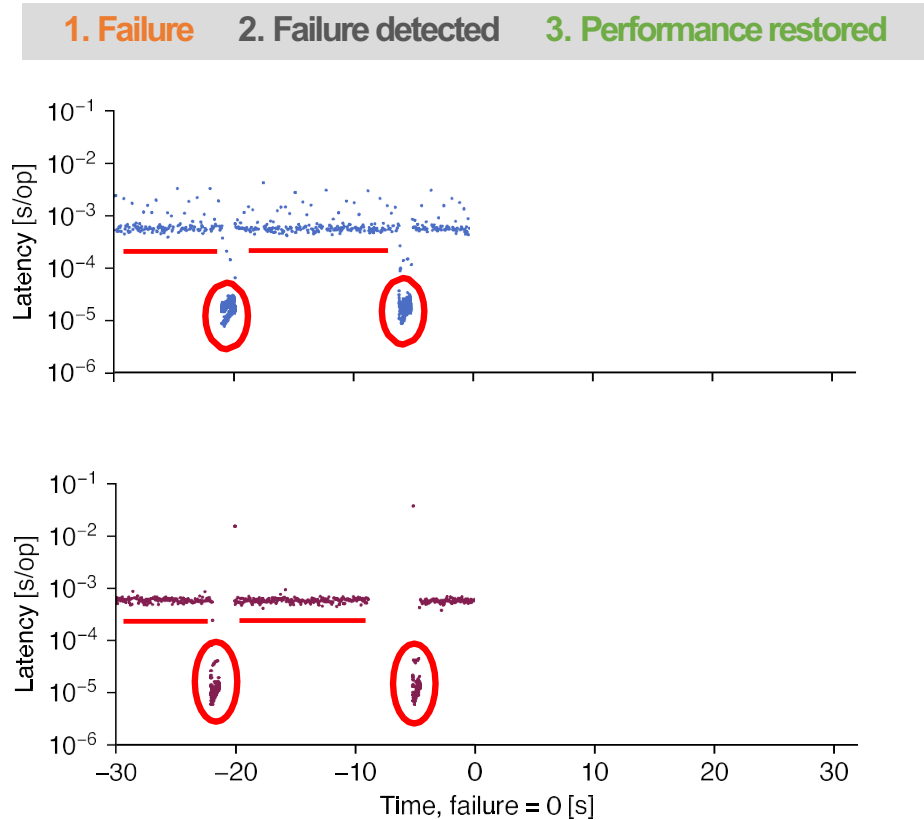
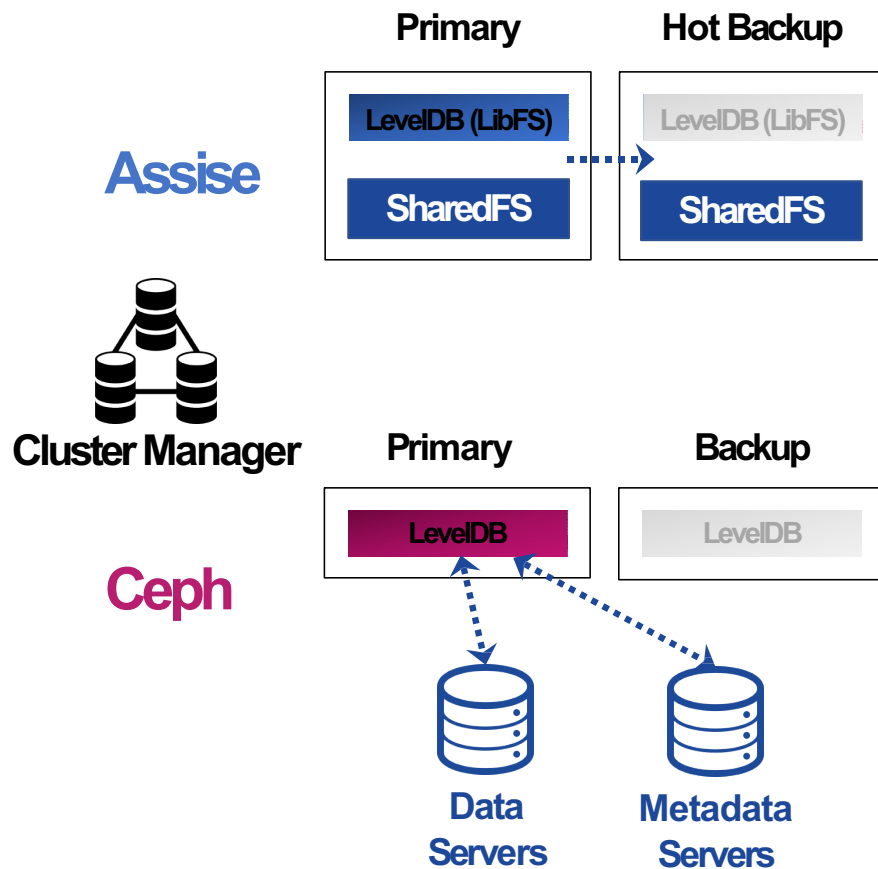
LevelDB: Fail-over to backup



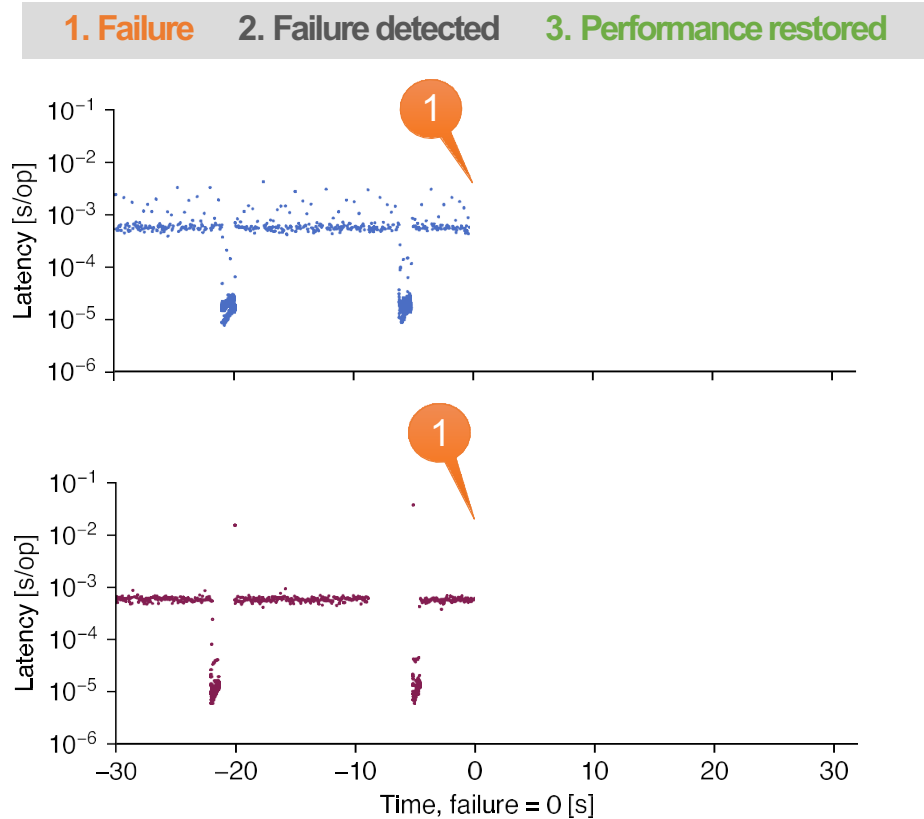
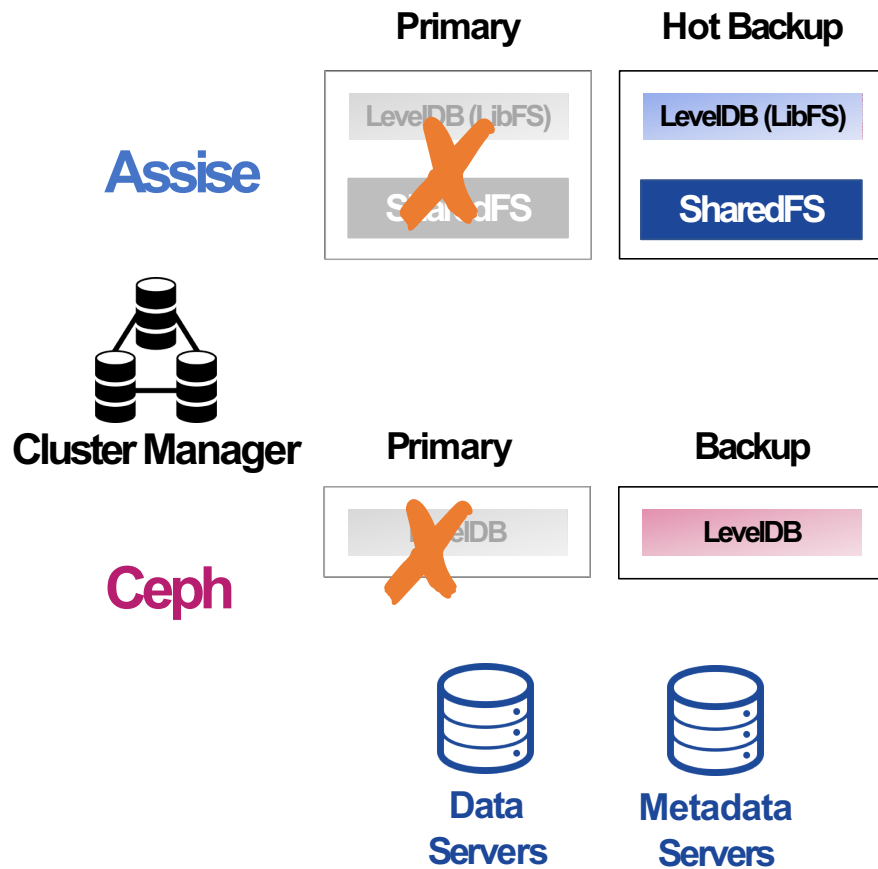
1. Failure 2. Failure detected 3. Performance restored



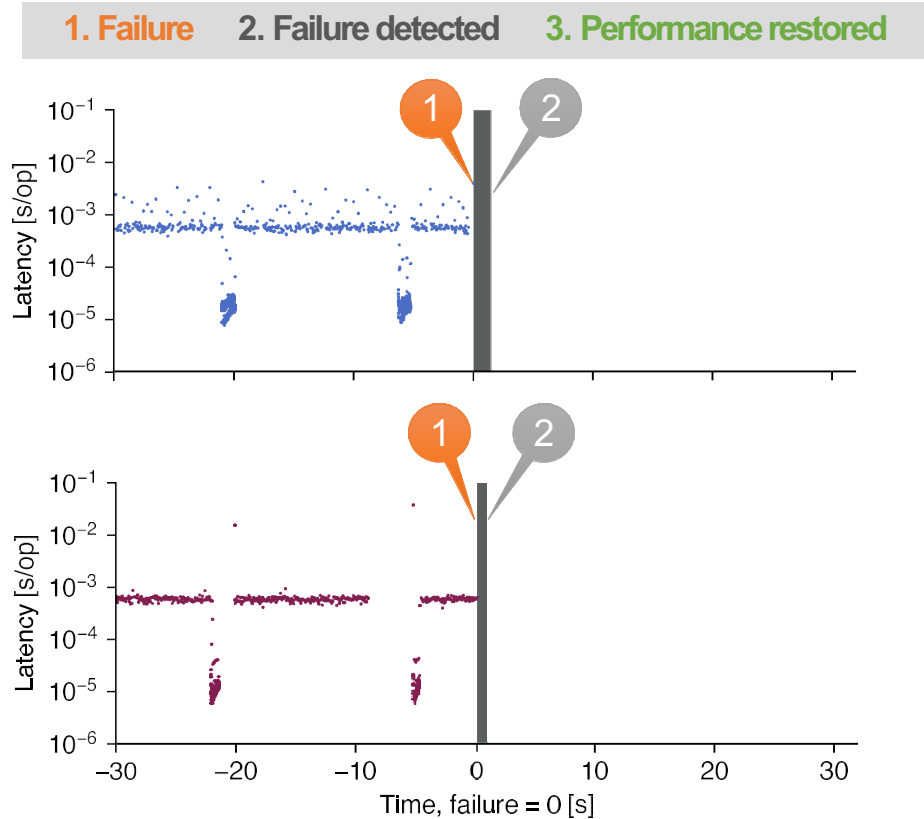
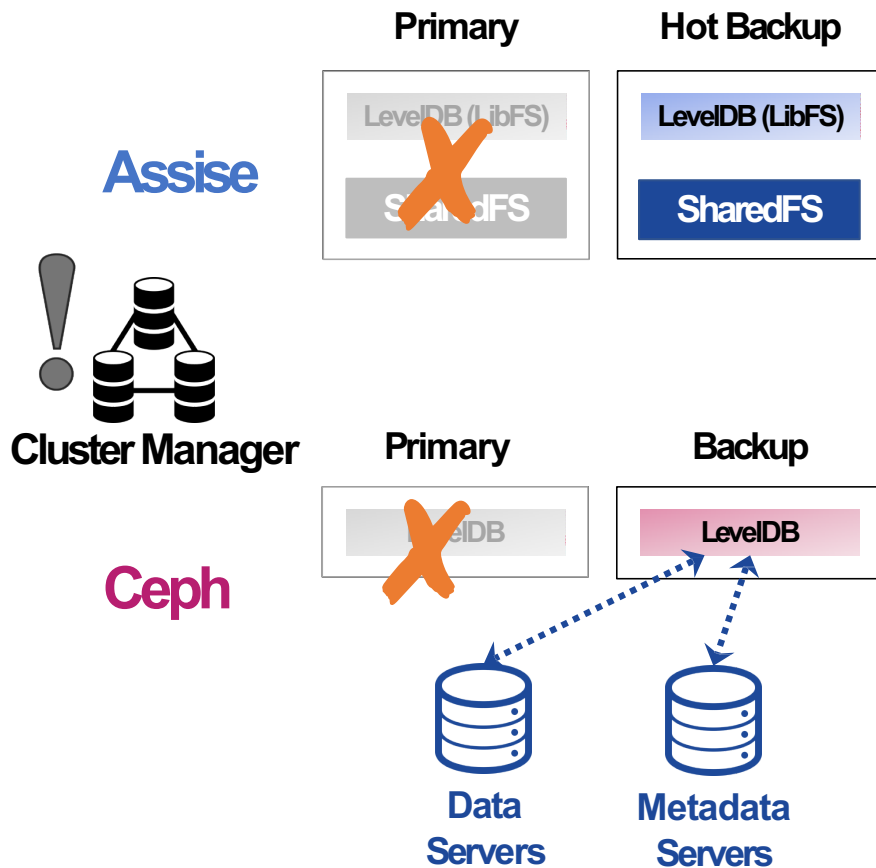
LevelDB: Fail-over to backup



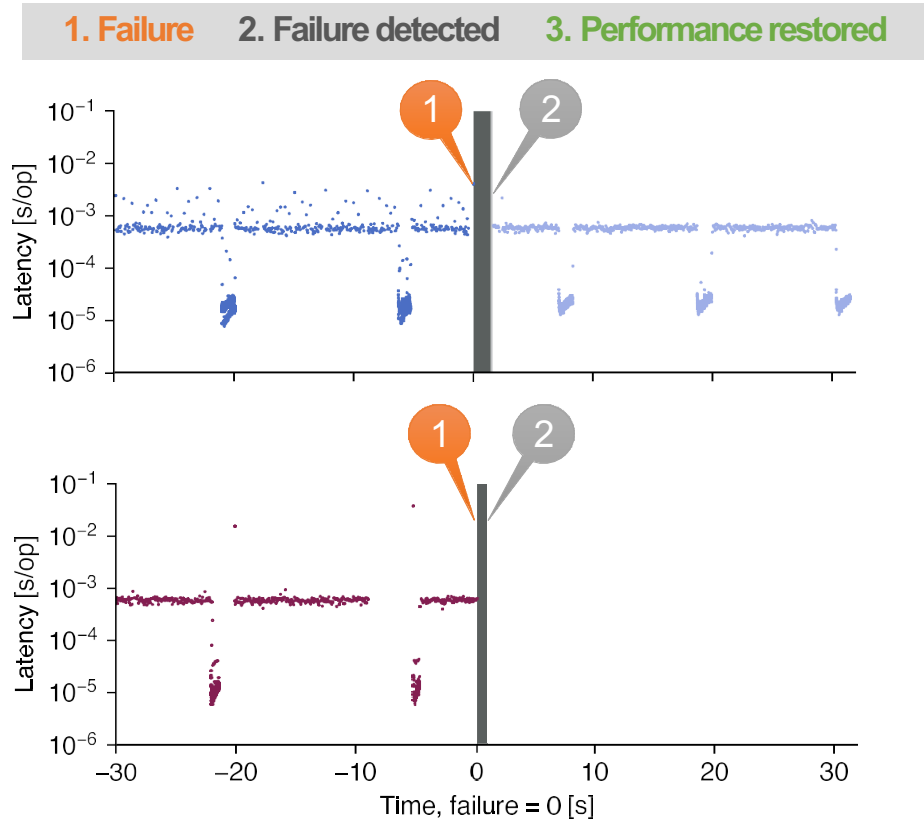
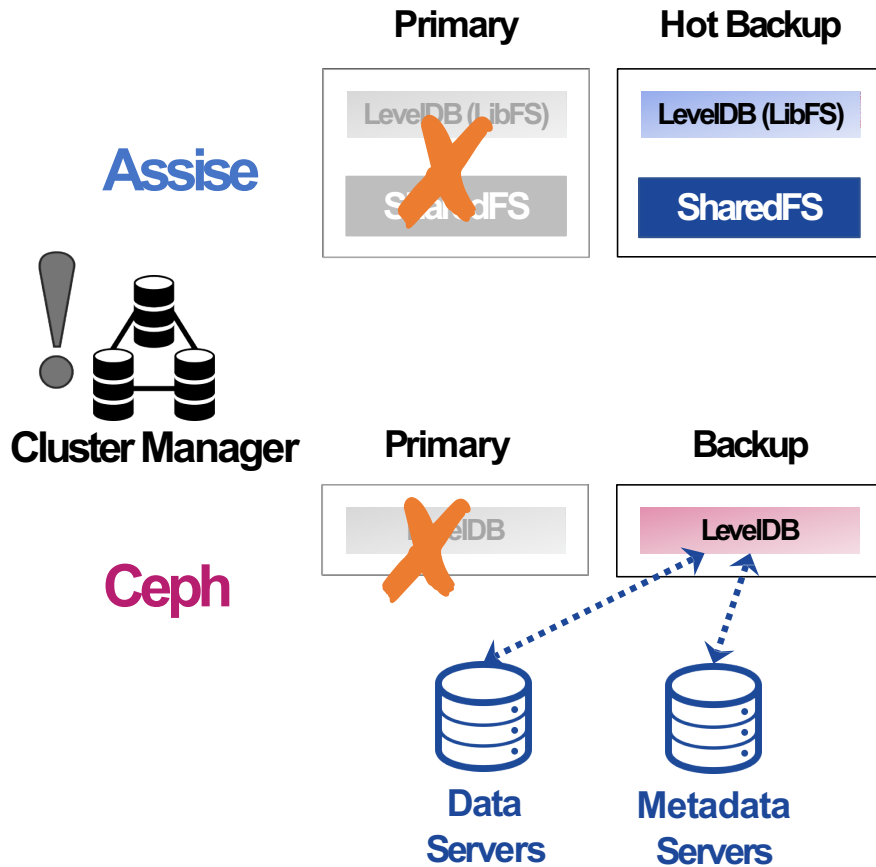
LevelDB: Fail-over to backup



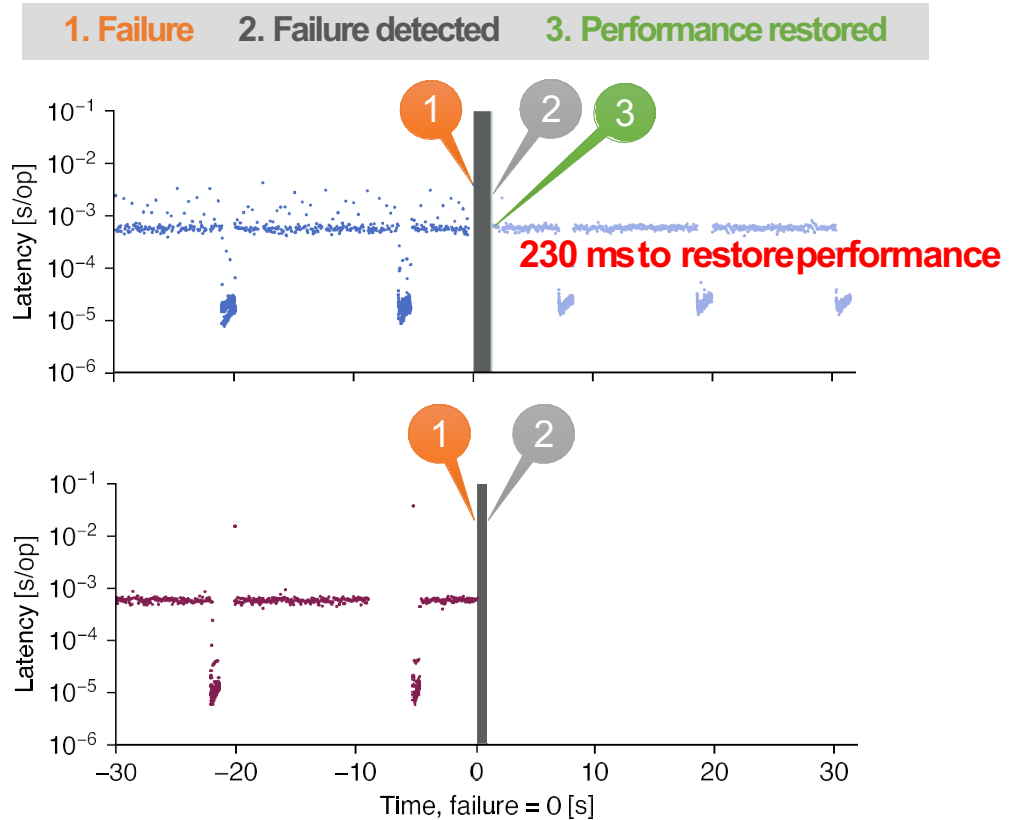
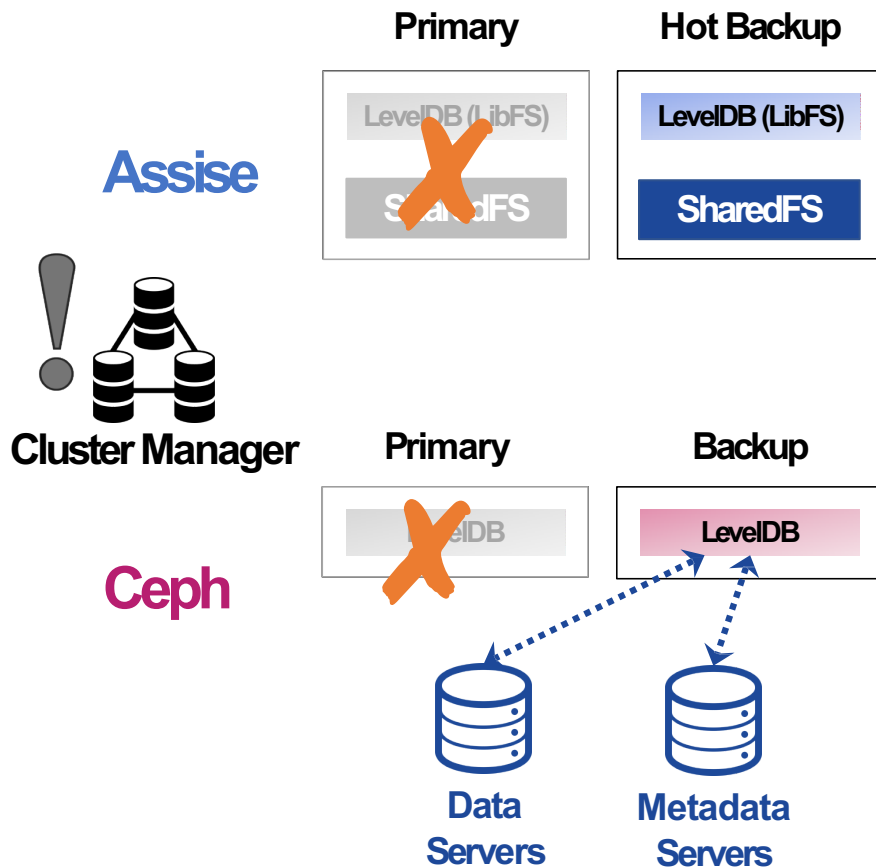
LevelDB: Fail-over to backup



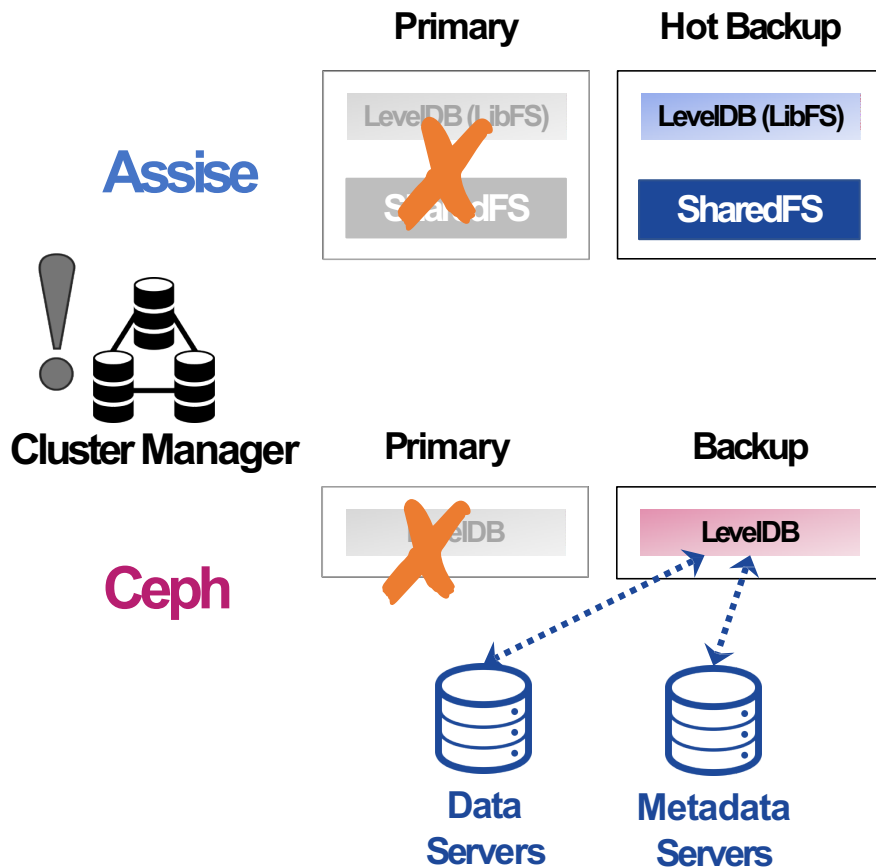
LevelDB: Fail-over to backup



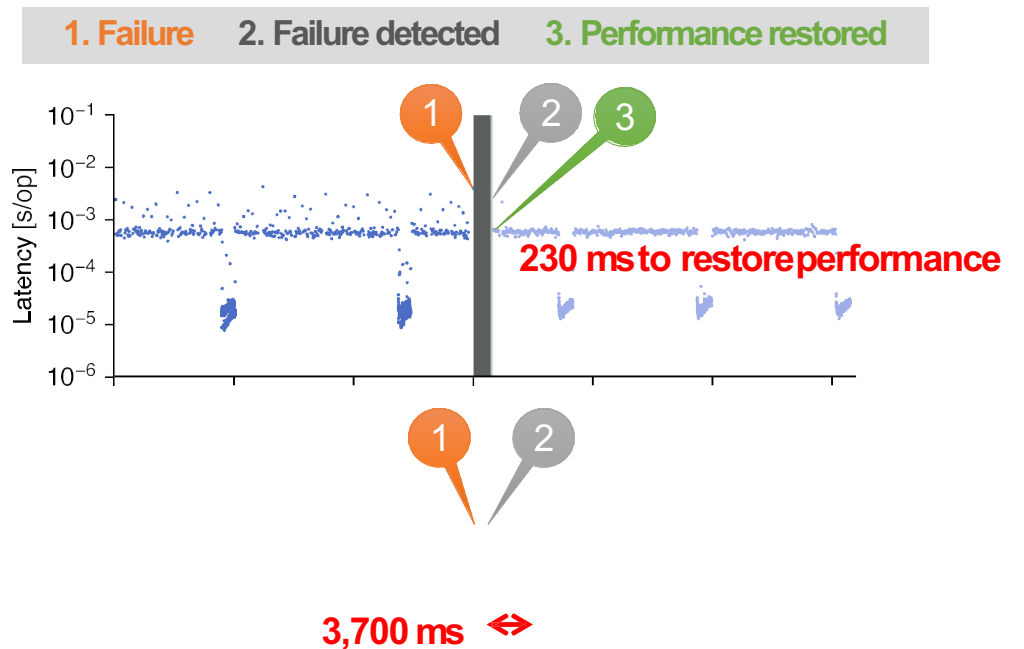
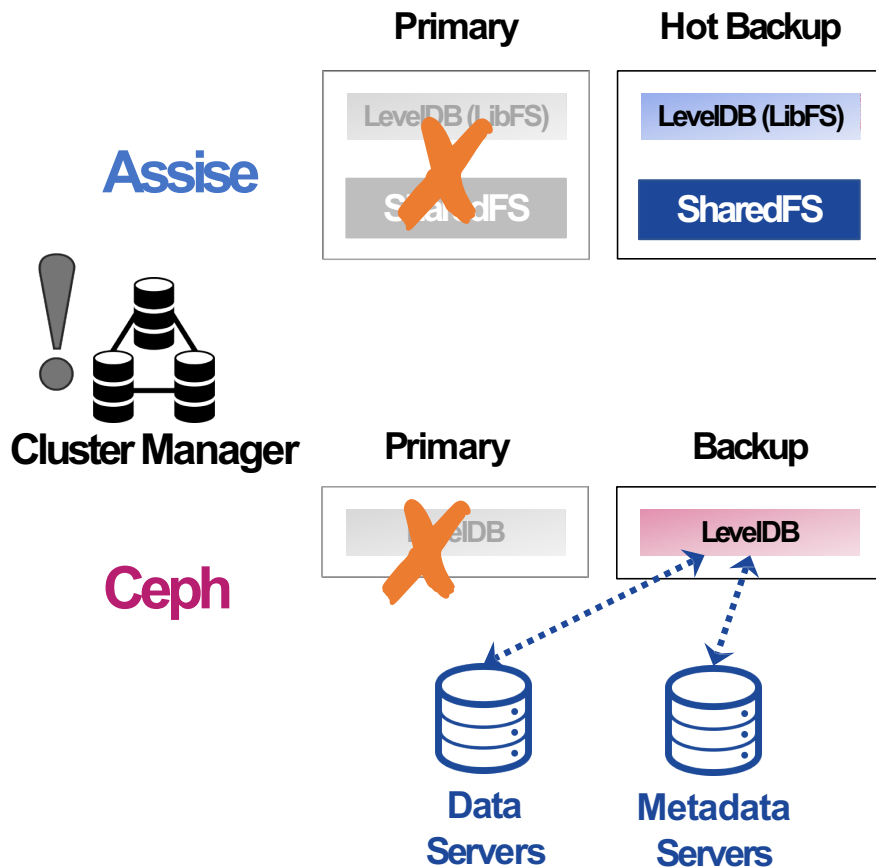
LevelDB: Fail-over to backup



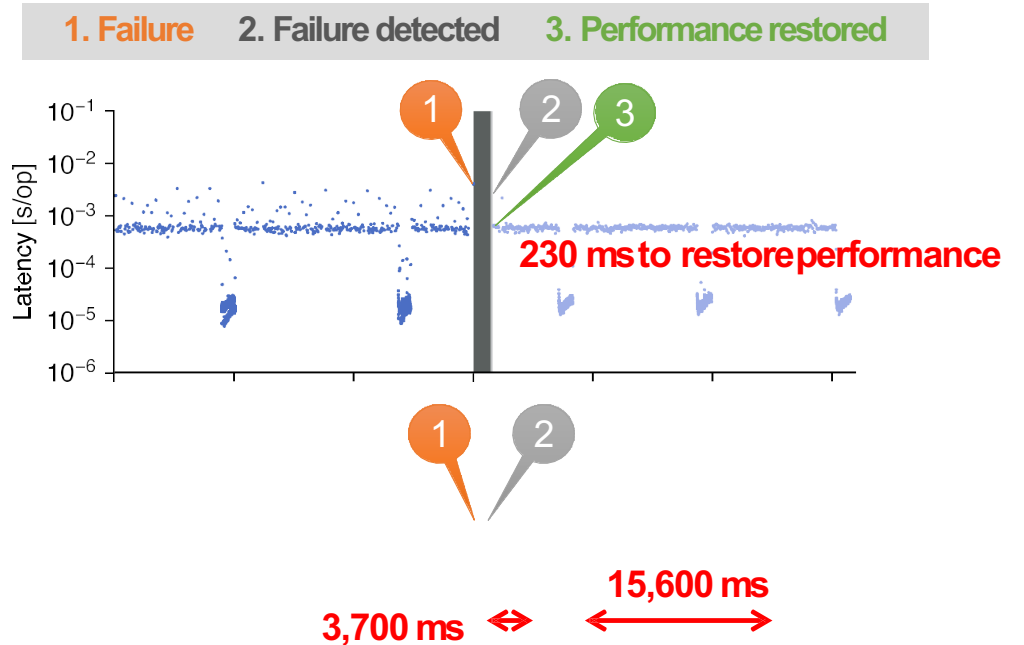
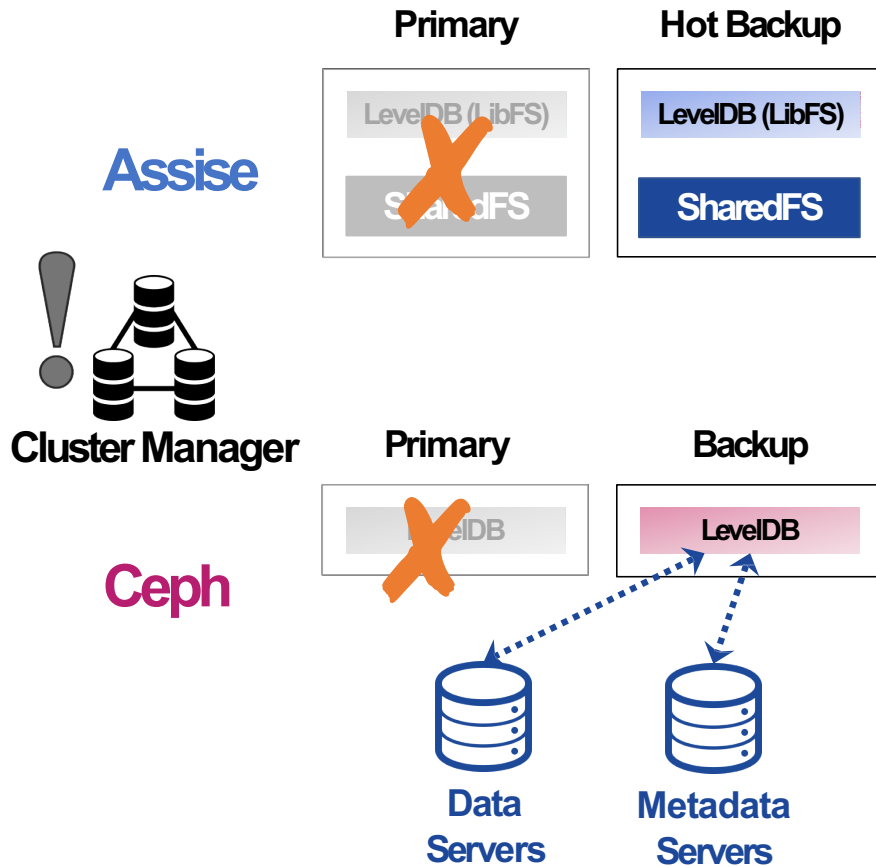
LevelDB: Fail-over to backup



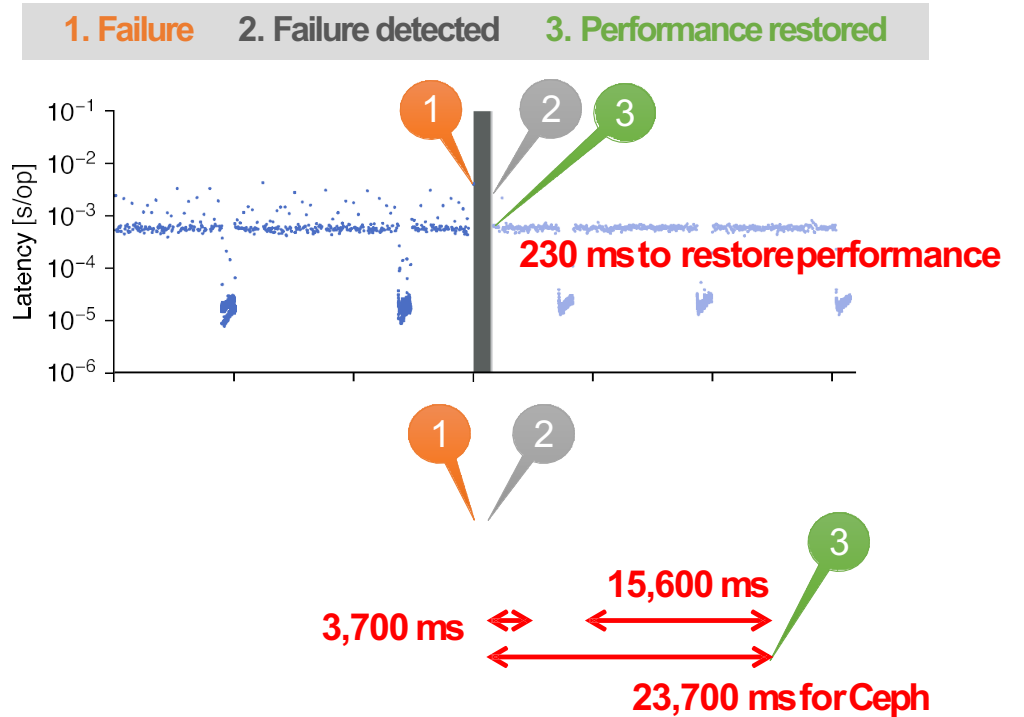
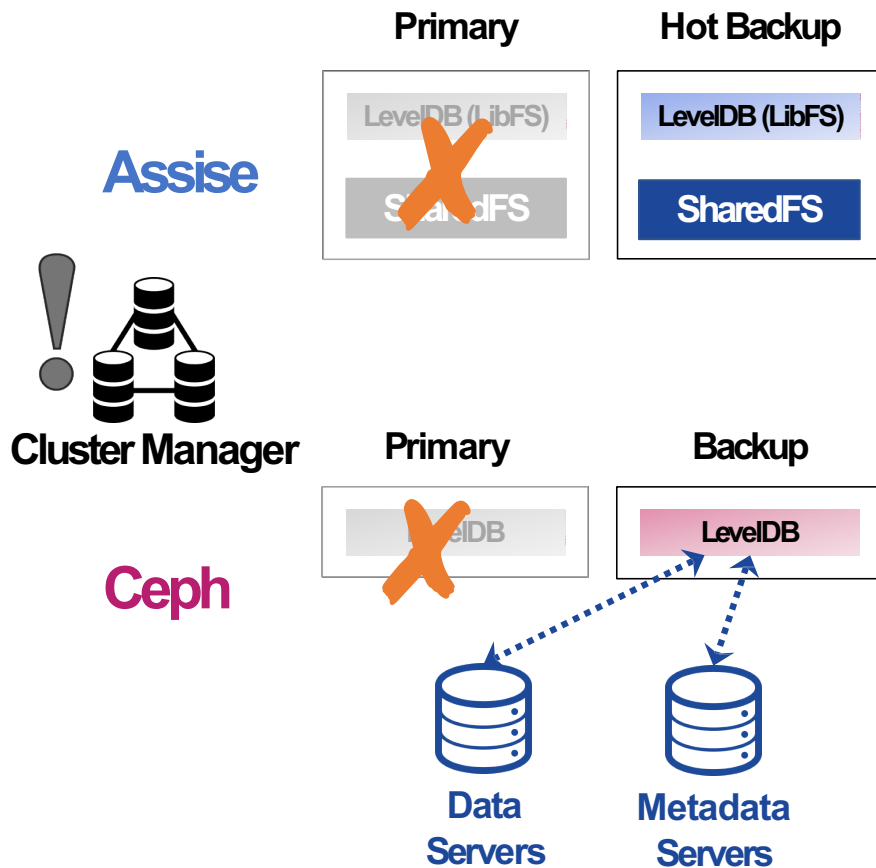
LevelDB: Fail-over to backup



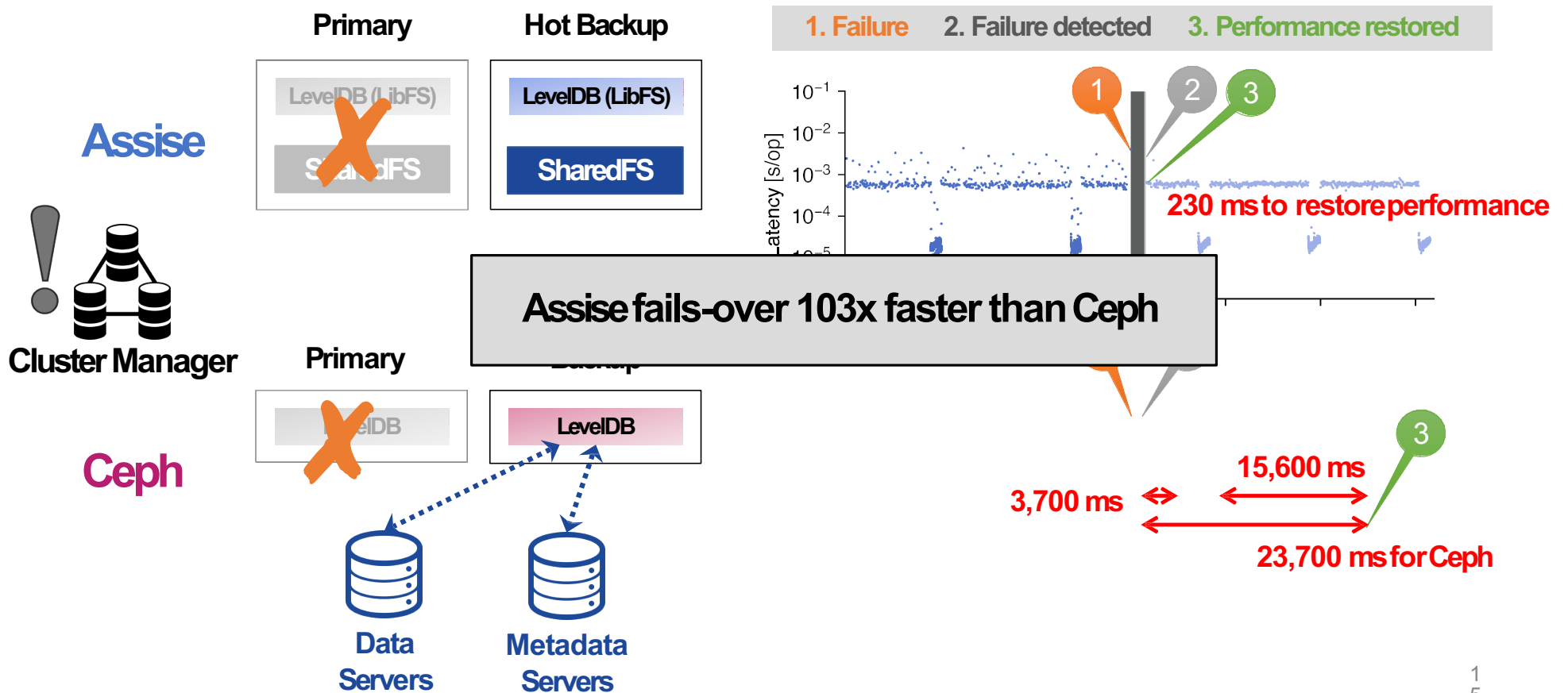
LevelDB: Fail-over to backup



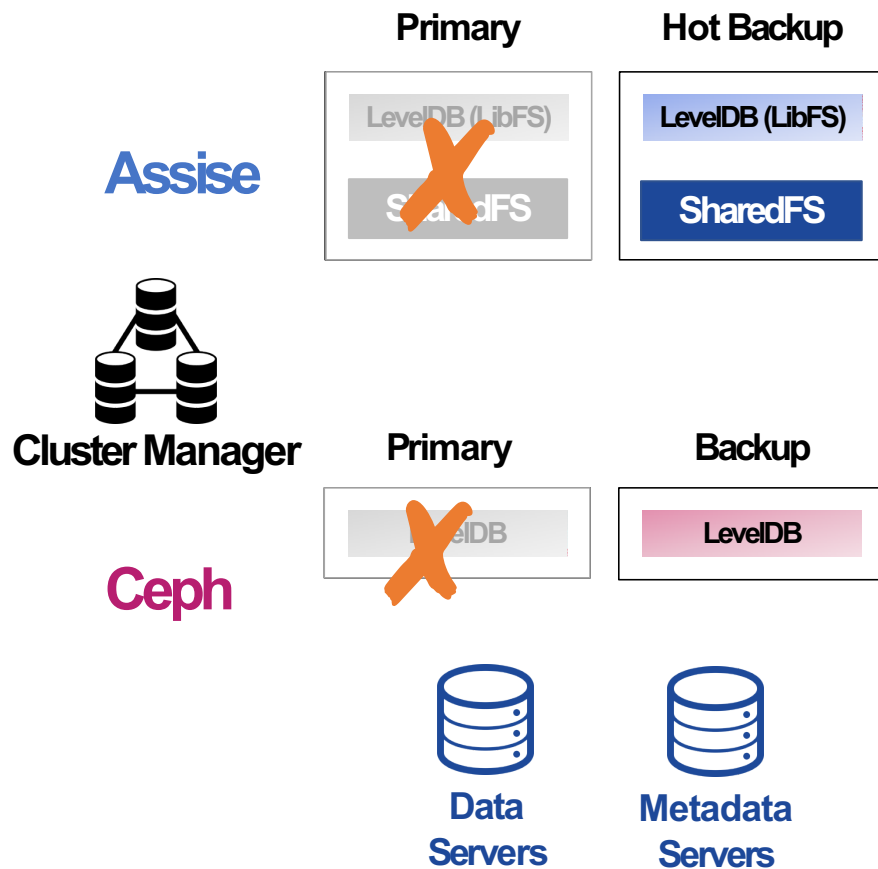
LevelDB: Fail-over to backup



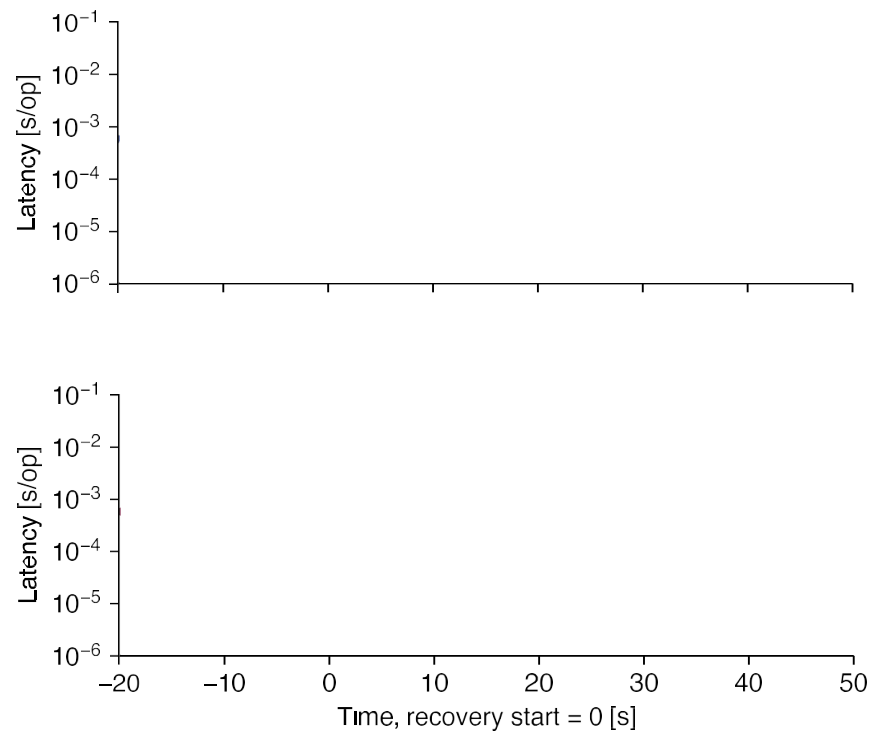
LevelDB: Fail-over to backup



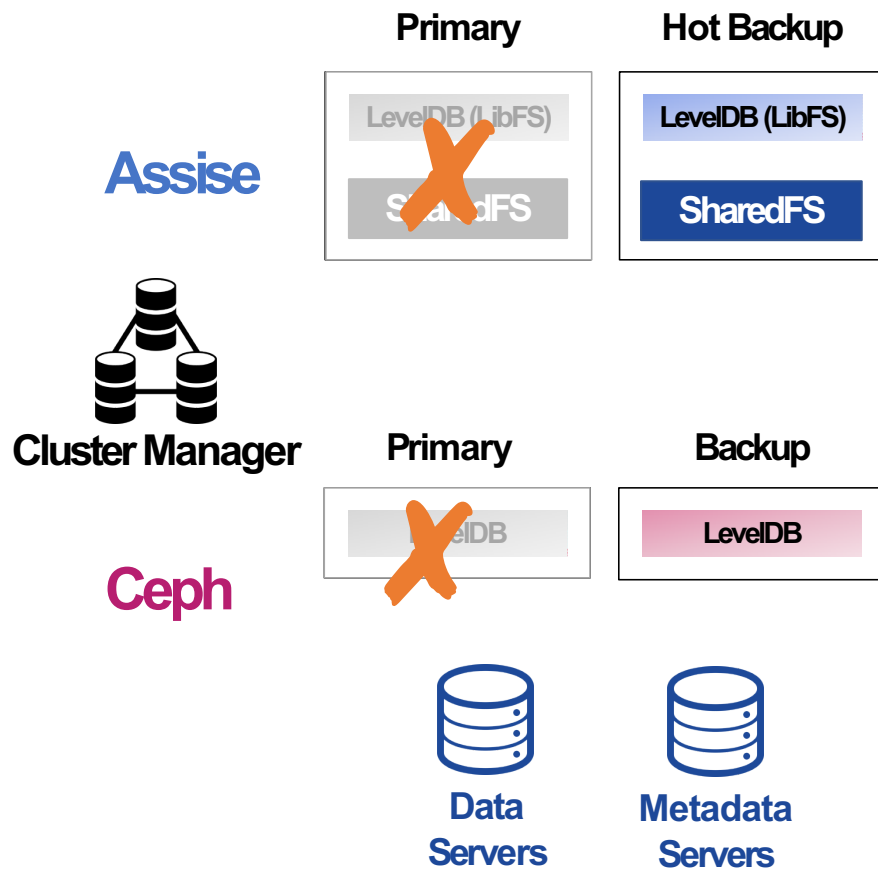
LevelDB: Primary recovery



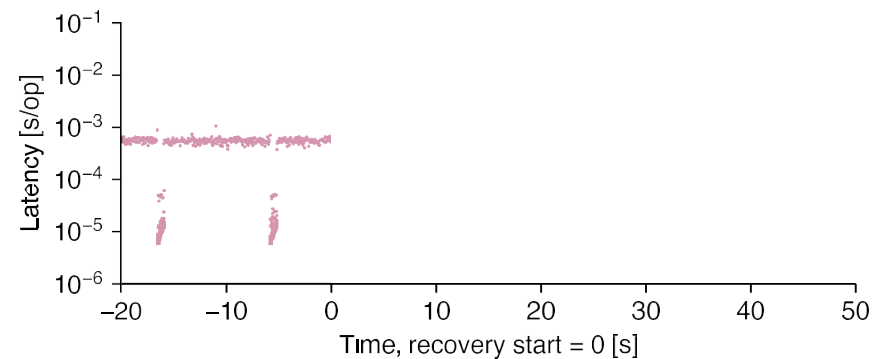
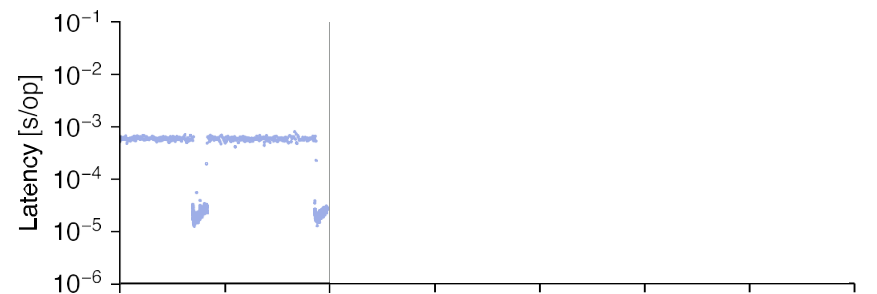
4. Primary restarts 5. Performance restored



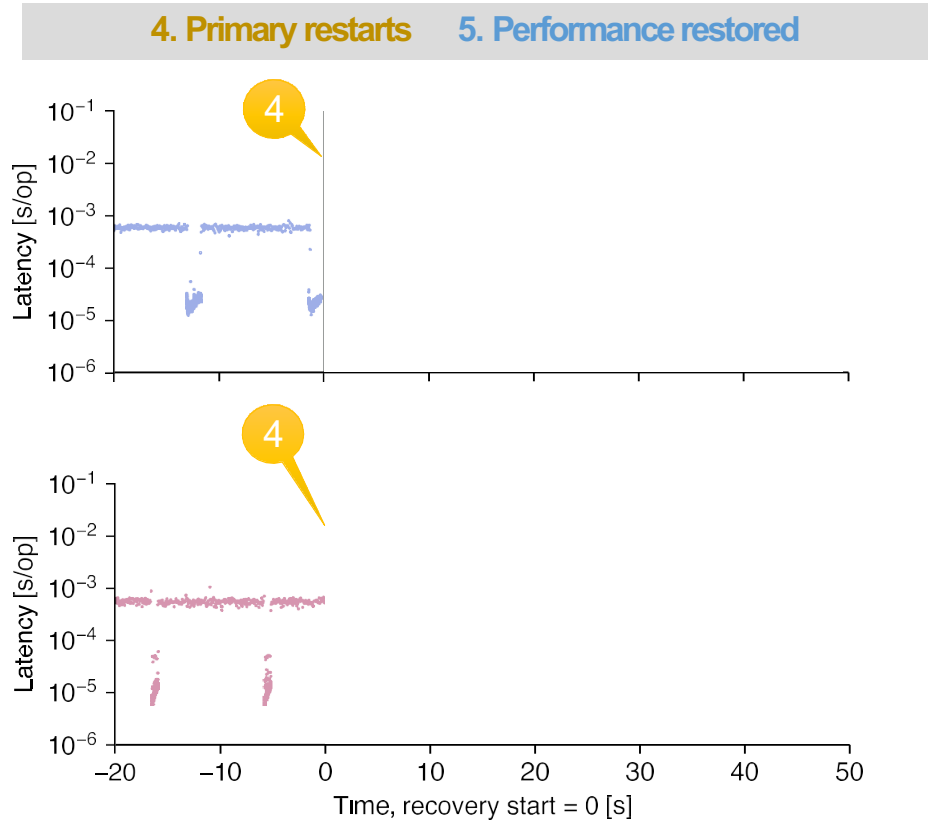
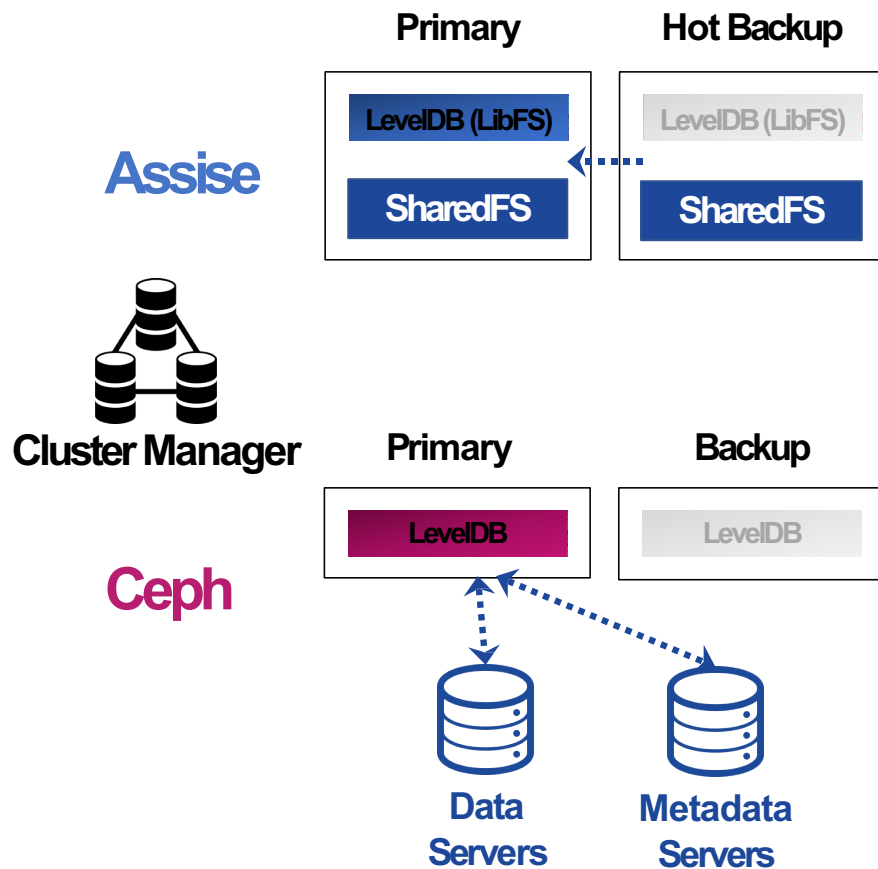
LevelDB: Primary recovery



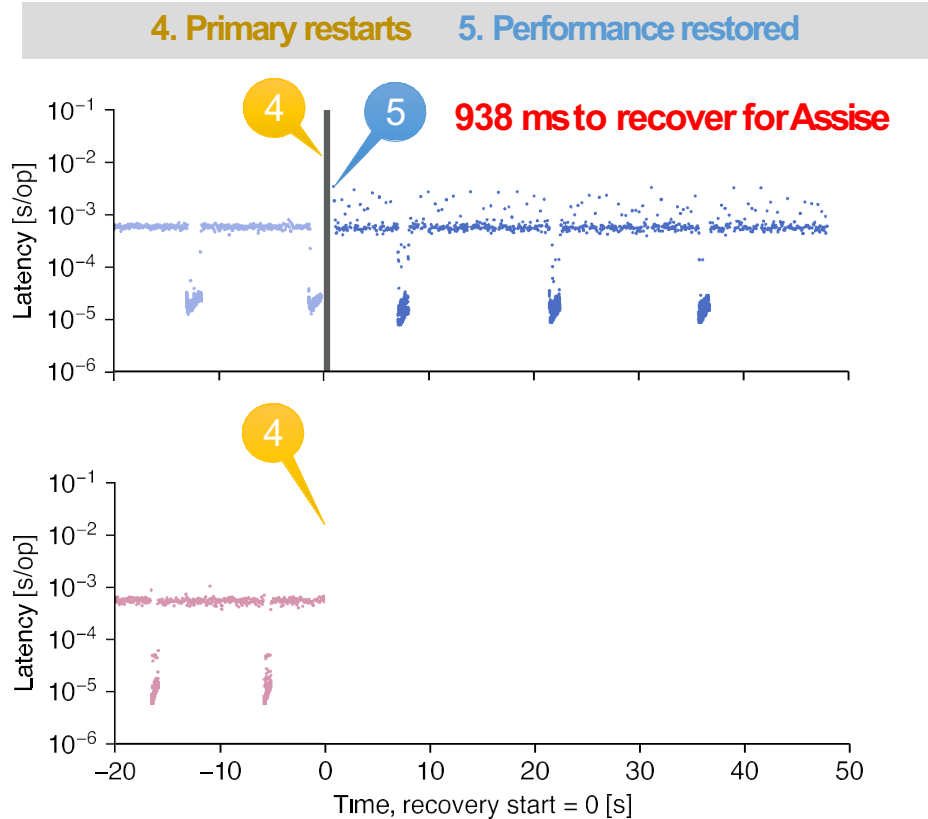
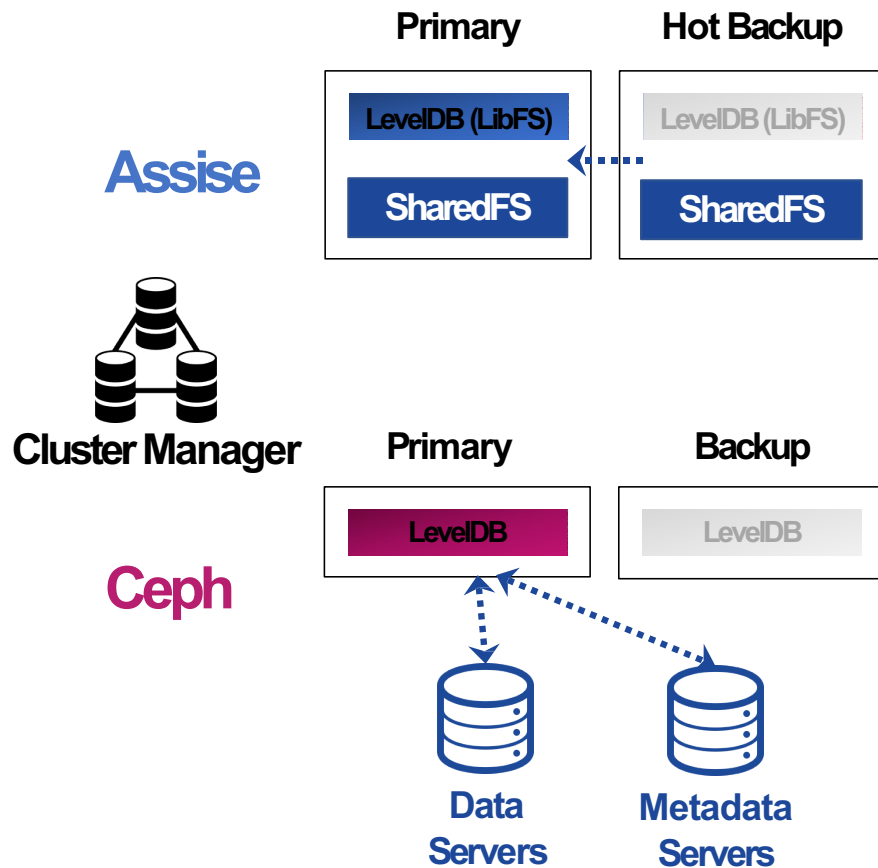
4. Primary restarts 5. Performance restored



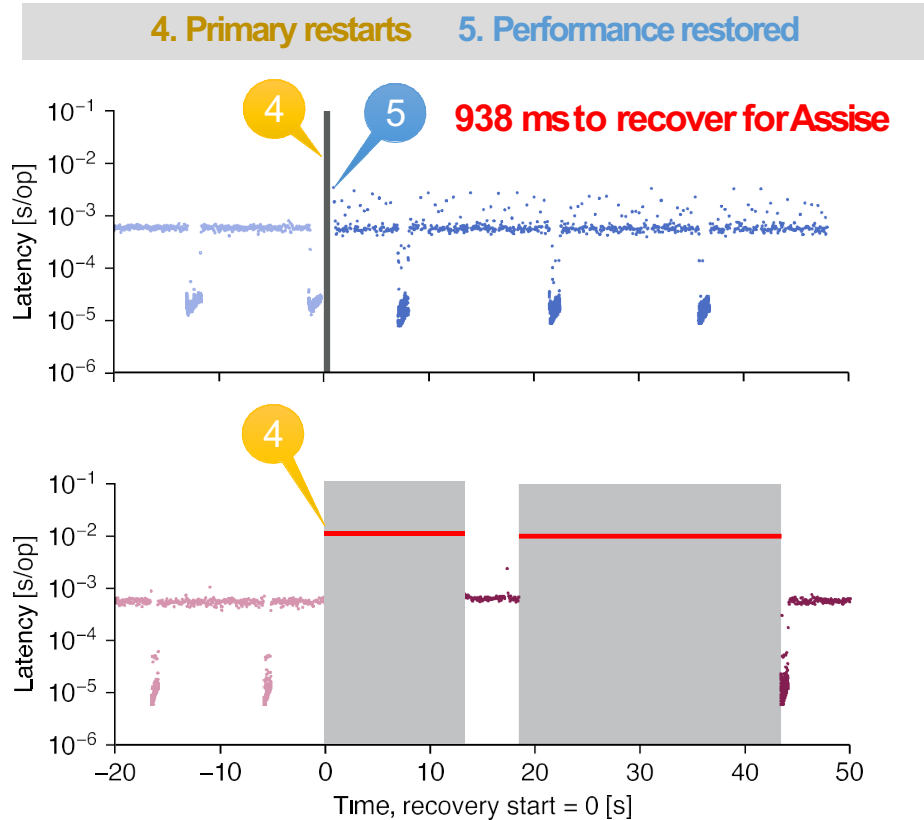
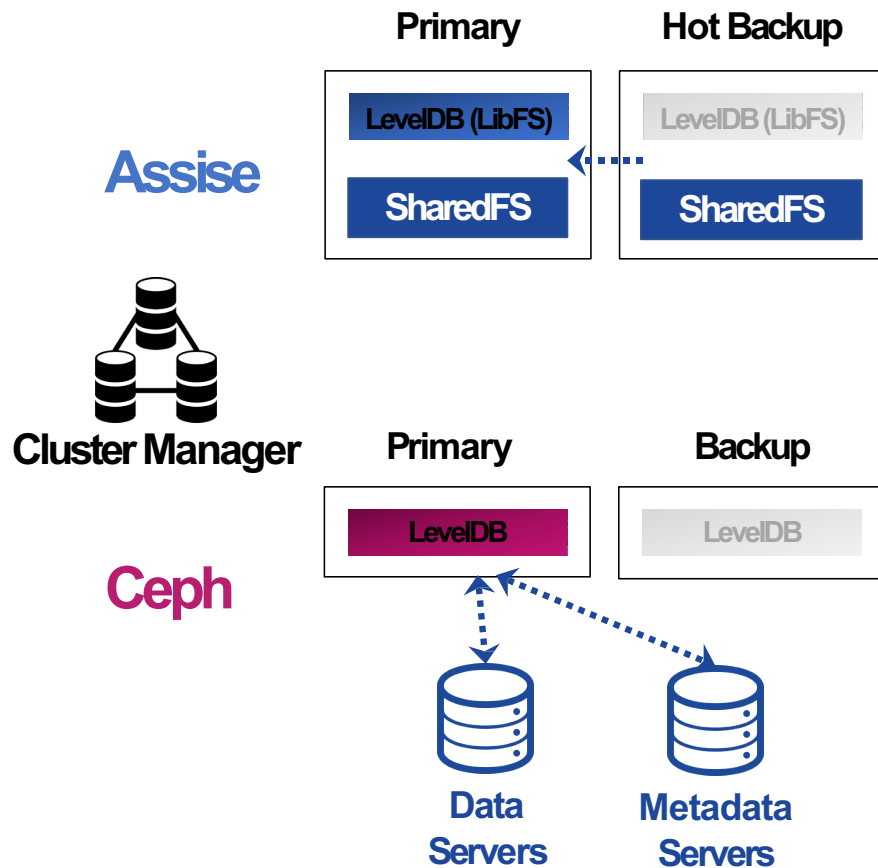
LevelDB: Primary recovery



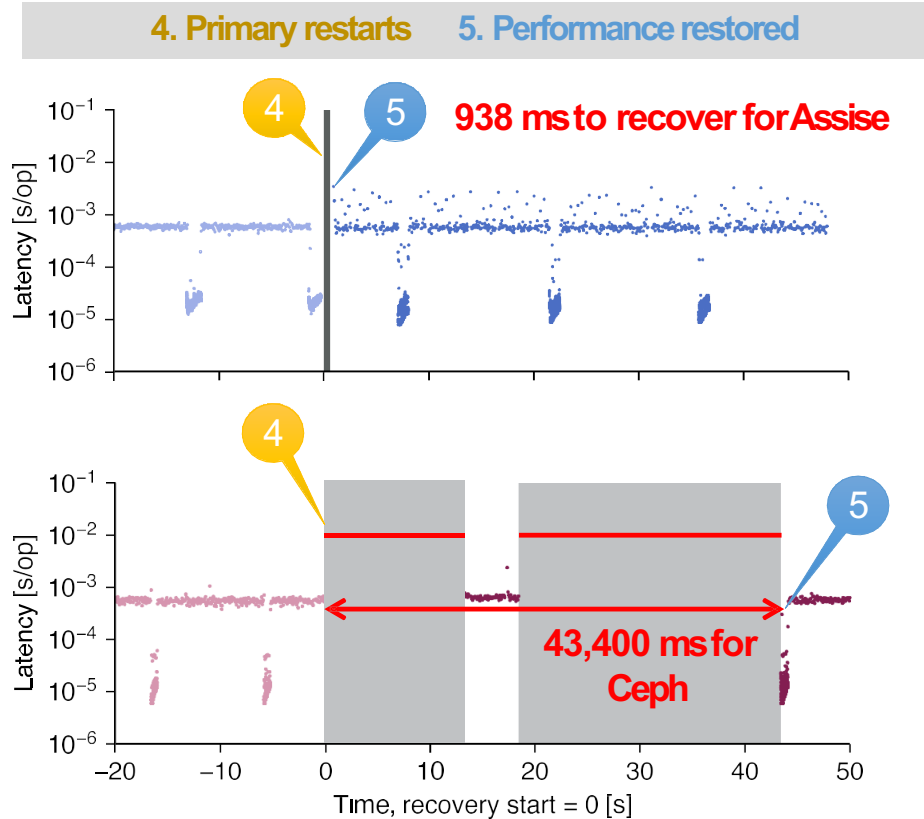
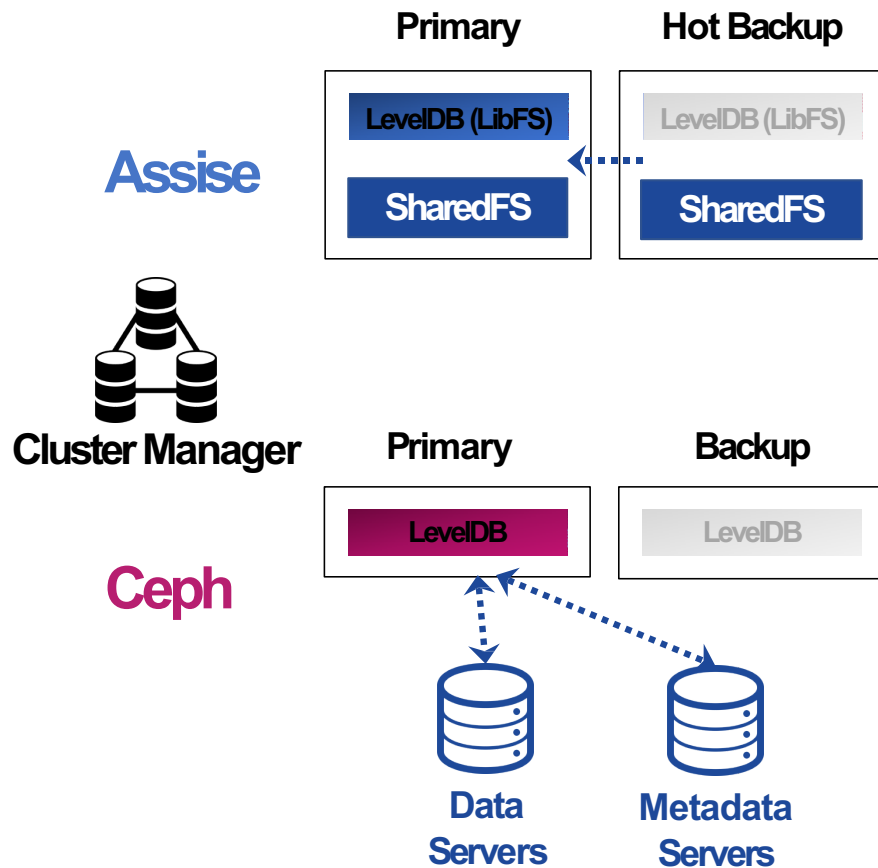
LevelDB: Primary recovery



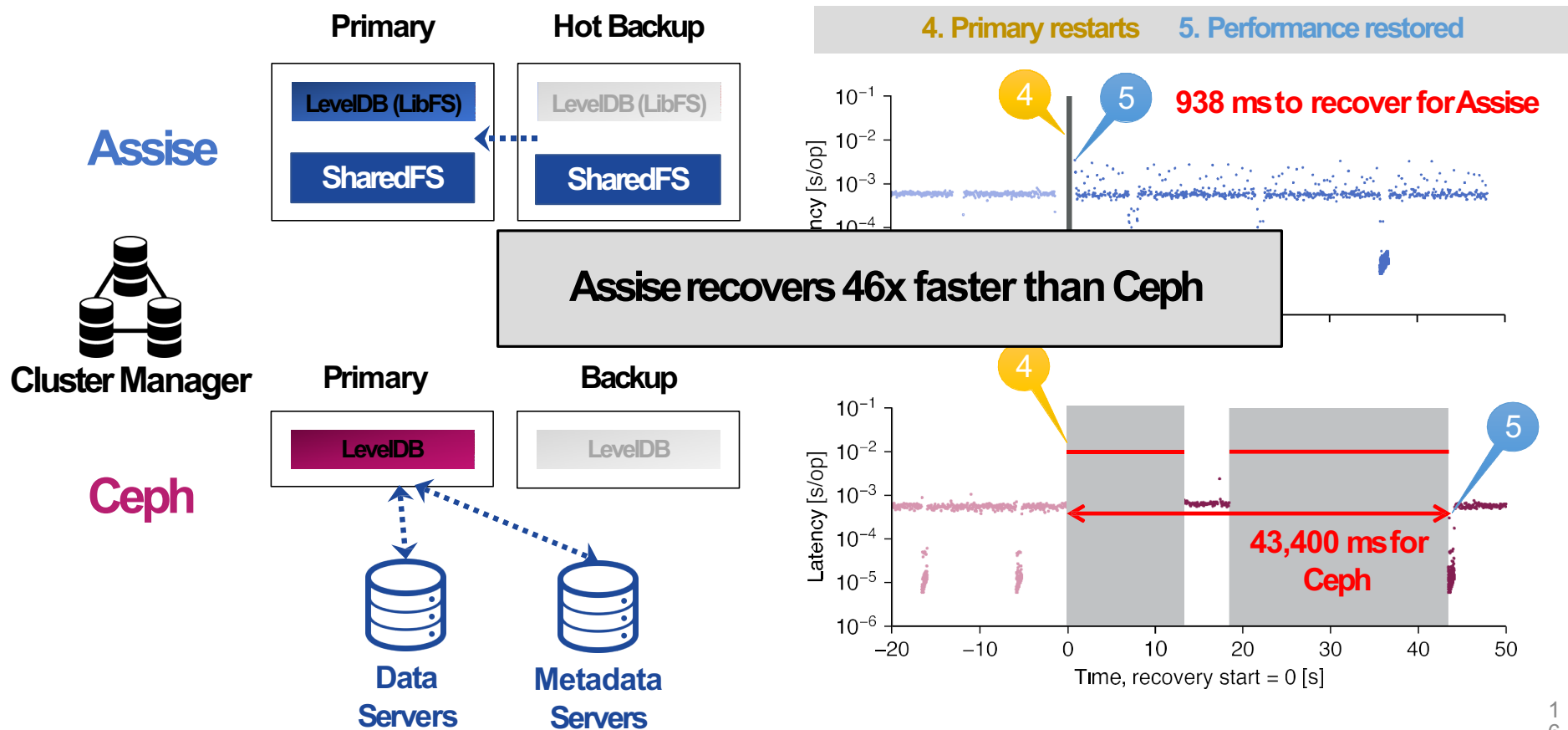
LevelDB: Primary recovery



LevelDB: Primary recovery



LevelDB: Primary recovery



Summary

- Assise: a distributed file system designed for client-local NVM
 - Low IO latency
 - Scalability
 - High availability
- Userspace local/remote IO, crash-consistent**
CC-NVM: scalable, linearizable, crash consistent
Fail-over to cache-hot client replicas
- 22x lower write latency , 103x faster fail-over, 6x better scalability
 - Comparisons against Ceph using real applications (Postfix, LevelDB, ...)

Source code available at: <https://github.com/ut-osa/assise>

Questions? Email assise@cs.utexas.edu

Table of contents

- A file system is no longer tied to one storage device
 - **Strata**: A Cross Media File System, **SOSP'17**
- The file system as a distributed service:
 - **Assise**: Performance and Availability via Client local NVM in a Distributed File System, **OSDI'20**
- Persistence is now also about recovery speed, not just durability
 - **Gemini**: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints, **SOSP'23**

Gemini: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints

Zhuang Wang, Zhen Jia, Shui Zheng, Zhen Zhang, Xinwei Fu,
T. S. Eugene Ng, Yida Wang



Large models

Characteristics

- Recent large language models (LLMs)

Model	Parameters	Accelerators	Training time	Developer	Year
Turing-NLG	17.2B	256 V100	—	Microsoft	2020
GPT-3	175B	—	—	OpenAI	2020
OPT-175B	175B	992 A100	2 months	Meta	2021
Gopher	280B	4096 TPU v3	1.3 months	Google	2021
MT-NLG	530B	4480 A100	3 months	Microsoft & NVIDIA	2022
PaLM	540B	6144 TPU v4	2 months	Google	2022
GPT-4	1.76T	—	4-7 months	OpenAI	2023

Larger training
models

More GPUs
involved

Longer training
time

Failures are frequent

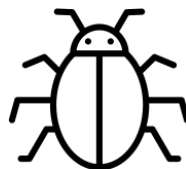
- Software failures



Library failures

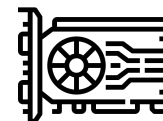


Remote storage failures

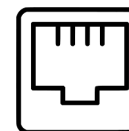


- OPT-175B: 100+ failures^[1] in two months

- Hardware failures



GPU failures



Link failures

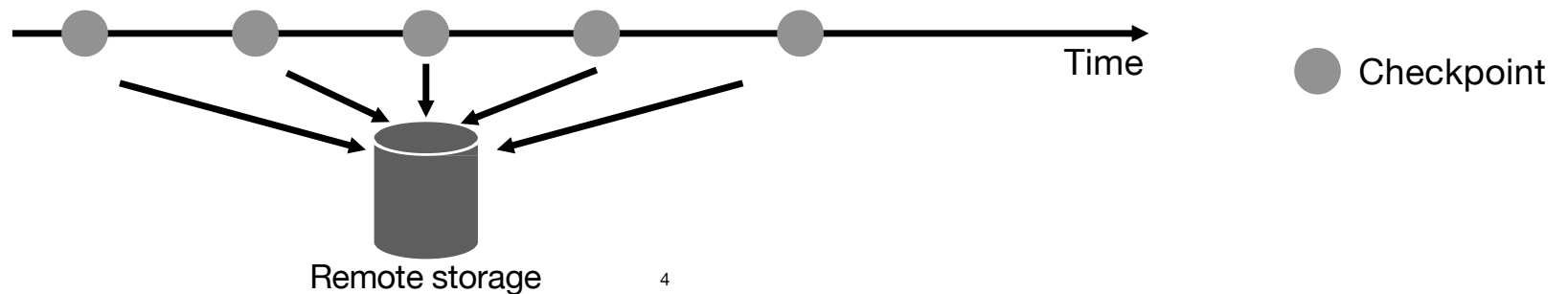
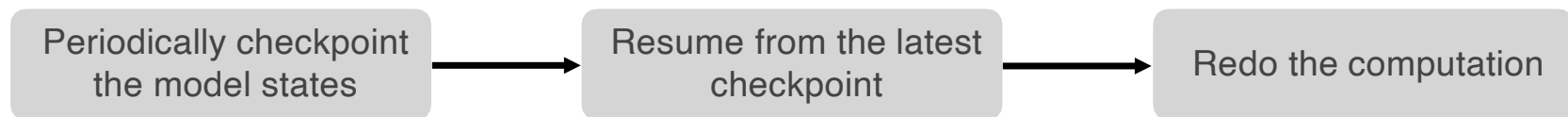


Switch failures

[1] Opt: Open pre-trained transformer language models, arXiv '22

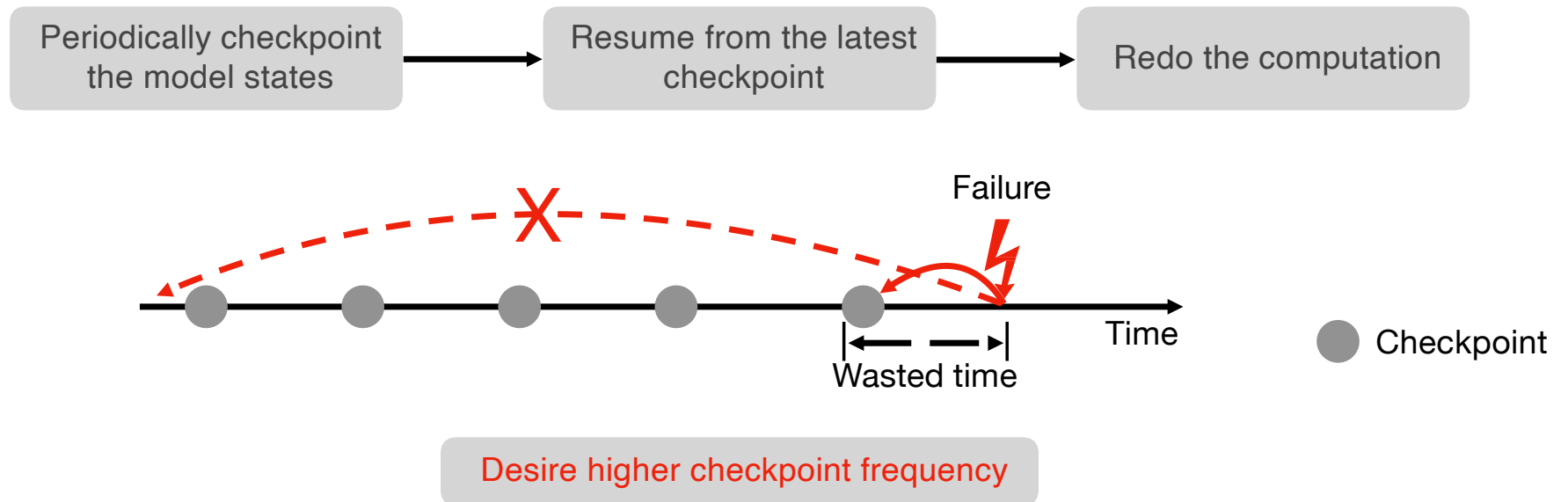
Checkpoint for failure recovery

- How checkpoint works?



Checkpoint for failure recovery

- How checkpoint works?



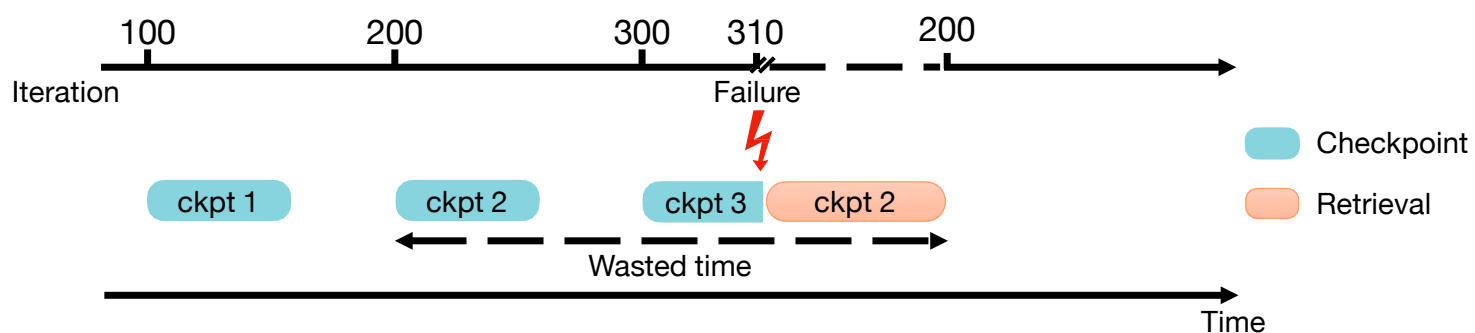
Checkpoint in LLM

Limited checkpoint frequency

- Checkpoint to remote storage takes a long time

Model	Parameters	Checkpoint size	Checkpoint time (20Gbps)
Gopher [56]	280B	3.4 TB	23 min
MT-NLG [62]	530B	6.4 TB	43 min
PaLM [23]	540B	6.5 TB	44 min

- Checkpoint frequency is limited by the checkpoint time



Checkpoint in LLM

Prohibitive failure recovery overhead

- Costly wasted time
 - Even with the highest checkpoint frequency

Model	Parameters	Checkpoint size	Checkpoint time (20Gbps)	Average wasted time
Gopher [56]	280B	3.4 TB	23 min	57 min
MT-NLG [62]	530B	6.4 TB	43 min	108 min
PaLM [23]	540B	6.5 TB	44 min	110 min

- Significant GPU resources are wasted due to failure recovery
 - Thousands of GPUs involved
 - Hundreds of failures during training

Gemini

Checkpoint to CPU memory

- CPU memory is much larger than GPU memory

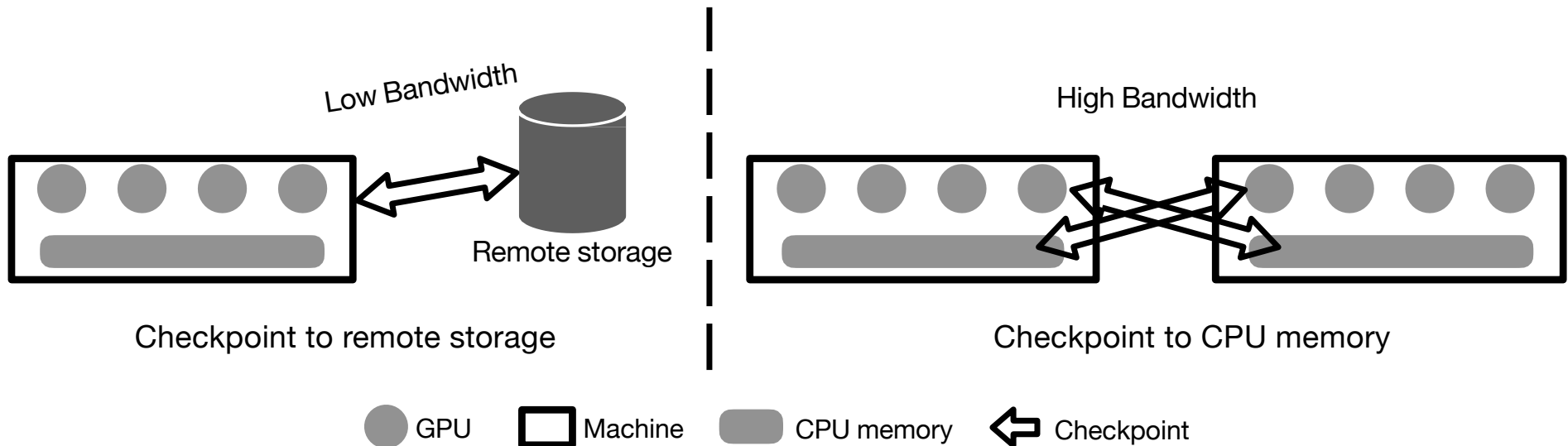
Instance type	Cloud	GPU	GPU memory	CPU memory
p3dn.24xlarge [14]	AWS	8 V100	8 × 32 GB	768 GB
p4d.24xlarge [15]	AWS	8 A100	8 × 40 GB	1152 GB
ND40rs_v2 [10]	Azure	8 V100	8 × 32 GB	672 GB
ND96asr_v4 [11]	Azure	8 A100	8 × 40 GB	900 GB
n1-8-v100 [9]	GCP	8 V100	8 × 32 GB	624 GB
a2-highgpu-8g [9]	GCP	8 A100	8 × 40 GB	640 GB
DGX A100 [12]	NVIDIA	8 A100	8 × 80 GB	2 TB

CPU memory size is sufficient to store checkpoints

Gemini

Checkpoint to CPU memory

- CPU memory is much larger than GPU memory
- Checkpoint to CPU memory enables a much higher frequency

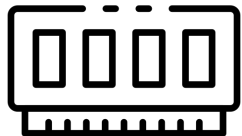


Gemini

Checkpoint to CPU memory

- CPU memory is much larger than GPU memory
- Checkpoint to CPU memory enables a much higher frequency
- CPU memory only stores checkpoints for failure recovery

Decouple the checkpoints for different purposes



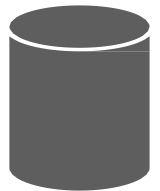
CPU memory

Failure recovery

- High-frequent checkpoints
- Only need the latest one

Debugging, accuracy evaluation

- Need checkpoint history
- Low-frequent checkpoints

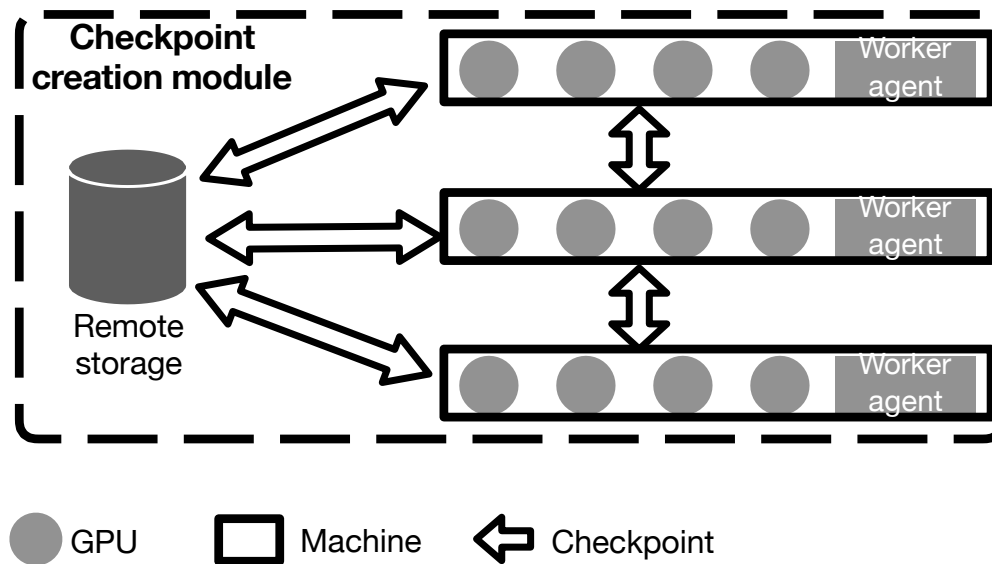


Remote storage

Gemini

Architecture

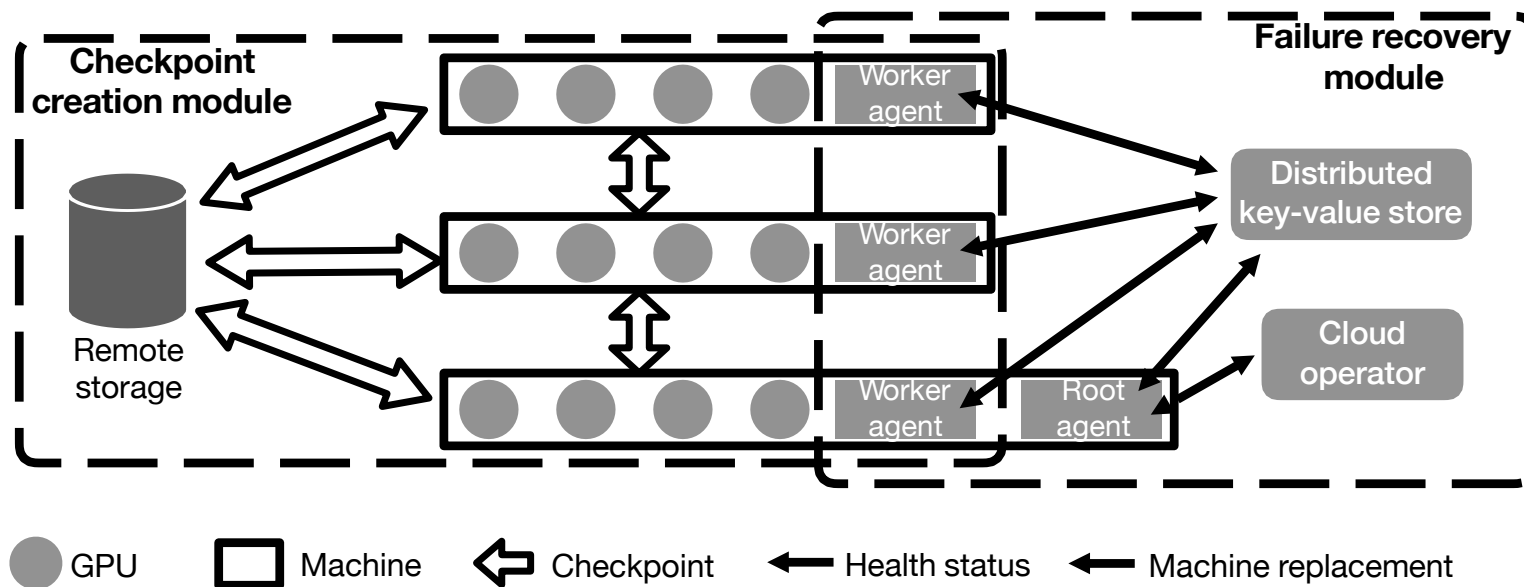
- Two modules



Gemini

Architecture

- Two modules

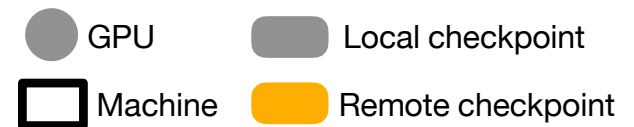
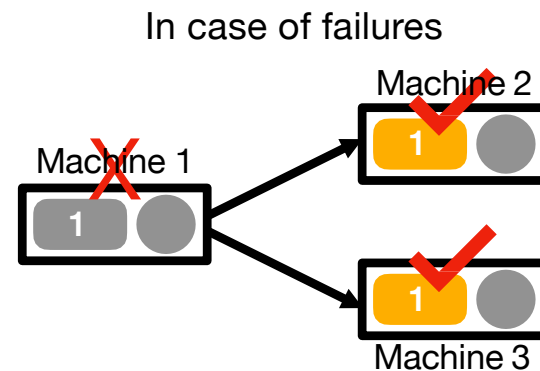
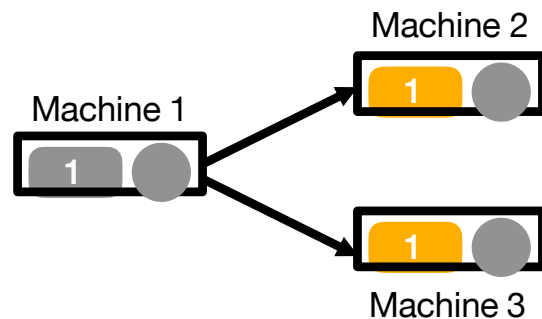


Challenge #1

- Data stored in CPU memory can get lost

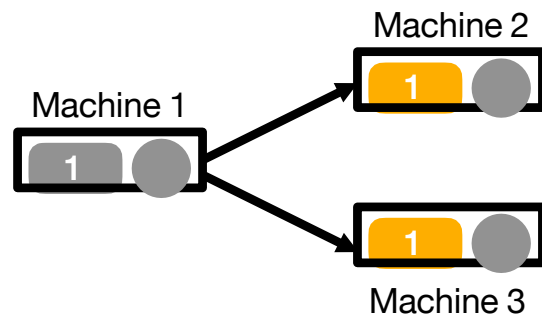
Challenge #1

- Data stored in CPU memory can get lost
- Checkpoint redundancy
 - Design choice: checkpoint replicas



Challenge #1

- Data stored in CPU memory can get lost
- Checkpoint redundancy
 - Design choice: checkpoint replicas



- Why not Erasure Coding?
 - Prohibitive computation cost
 - CPU memory is not a bottleneck



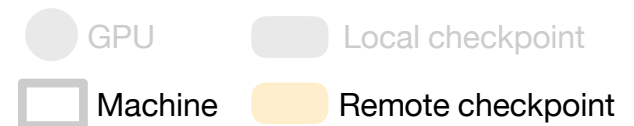
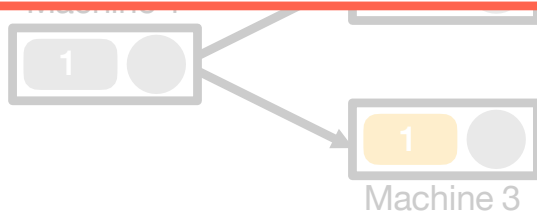
Challenge #1

Optimal checkpoint placement

- Data stored in CPU memory can get lost
- Solution: checkpoint redundancy
- Design choice: checkpoint replicas

Maximize the probability of failure recovery from checkpoints

Machine 2
stored in CPU memory



Solution

Group placement strategy

- Two steps

1. Given m replicas, all machines are divided into disjoint groups and each group has m machines
2. Each machine backups a checkpoint replica for all machines within the same group

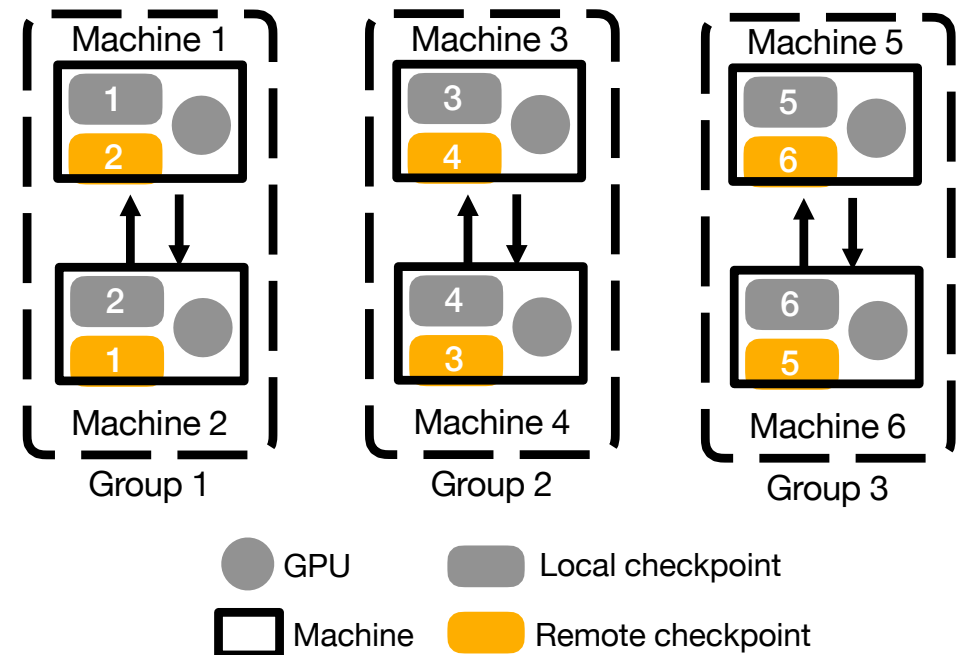
Solution

Group placement strategy

- Two steps

1. Given m replicas, all machines are divided into disjoint groups and each group has m machines
2. Each machine backups a checkpoint replica for all machines within the same group

- An example with two replicas



Solution

Group placement strategy

- Two steps

1. Given m replicas, all machines are divided into disjoint groups and each group has m machines
2. Each machine backups a checkpoint replica for all machines within the same group

Group placement strategy is provably optimal

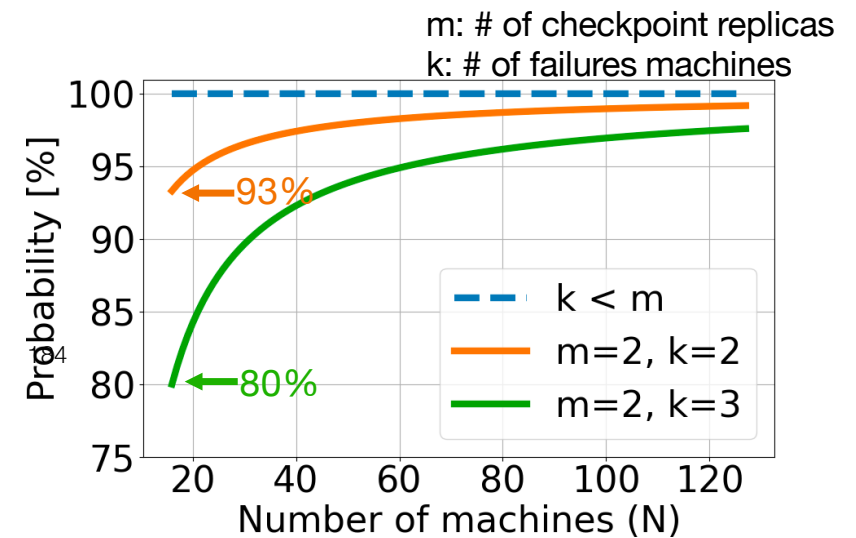
Solution

Group placement strategy

- Two steps

1. Given m replicas, all machines are divided into disjoint groups and each group has m machines
2. Each machine backups a checkpoint replica for all machines within the same group

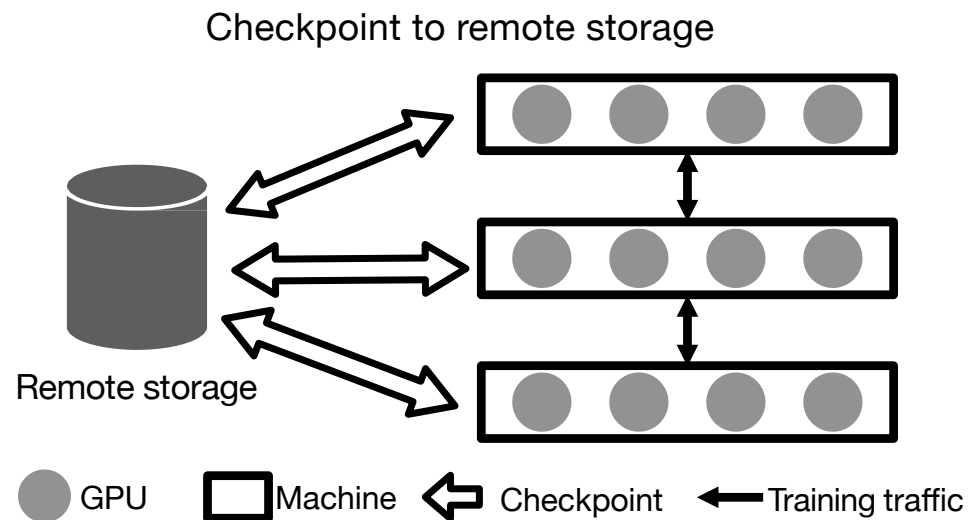
Group placement strategy is provably optimal



Two checkpoint replicas can already handle most failure cases!

Challenge #2

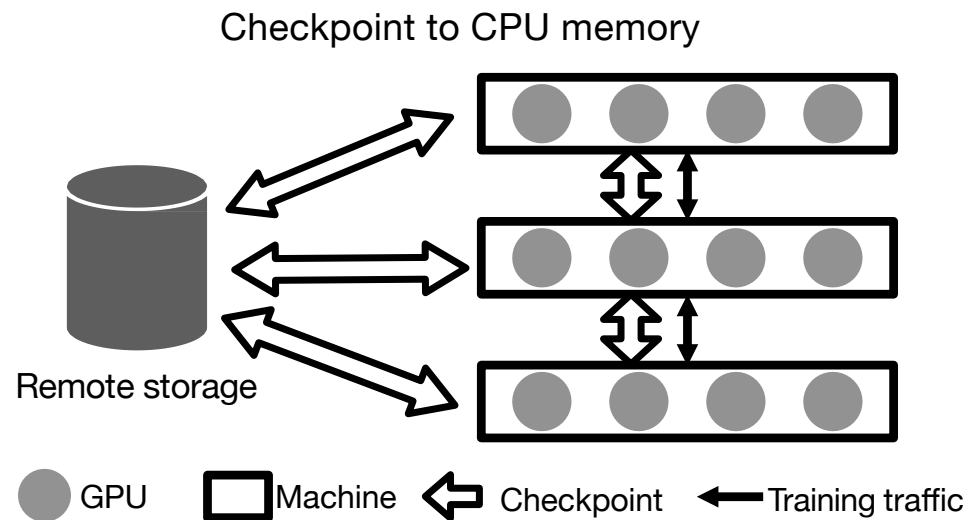
- Checkpoint traffic interferes with training traffic



Checkpoint traffic and training traffic have different networks

Challenge #2

- Checkpoint traffic interferes with training traffic



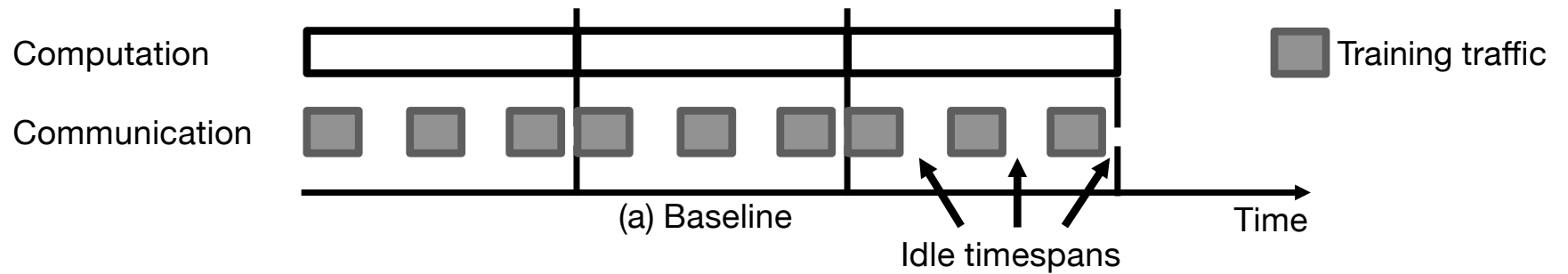
Checkpoint traffic and training traffic
shares the same network

It can harm training throughput

Solution

Traffic interleaving

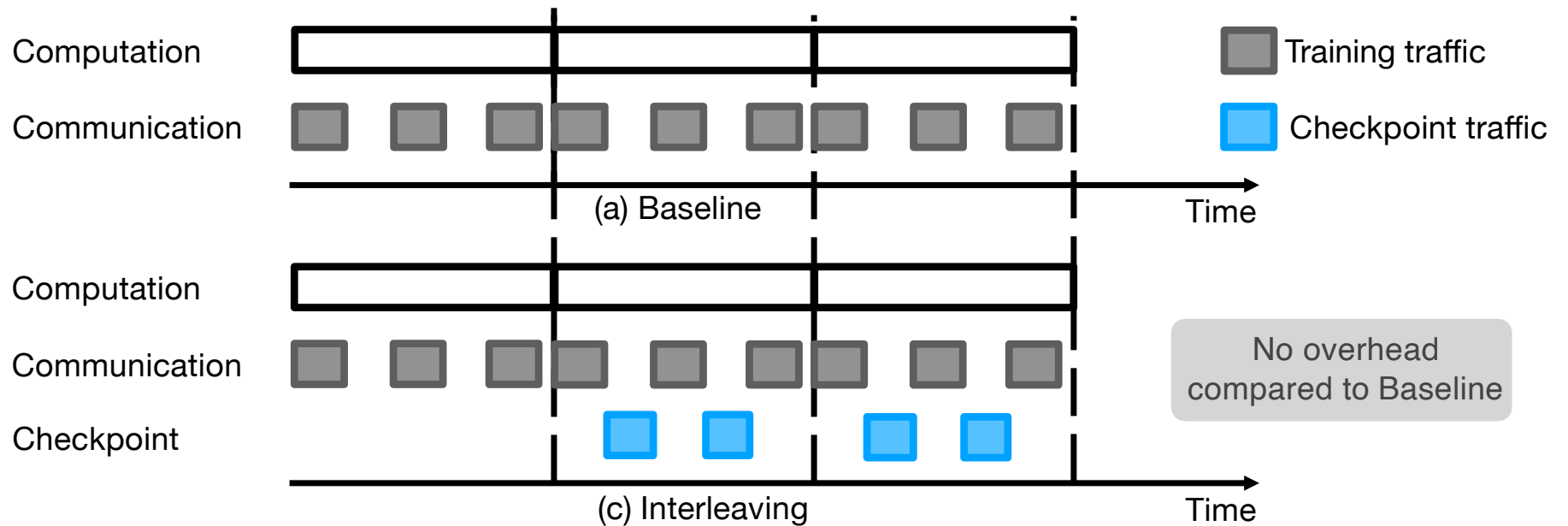
- Observation: Idle timespans in the network



Solution

Traffic interleaving

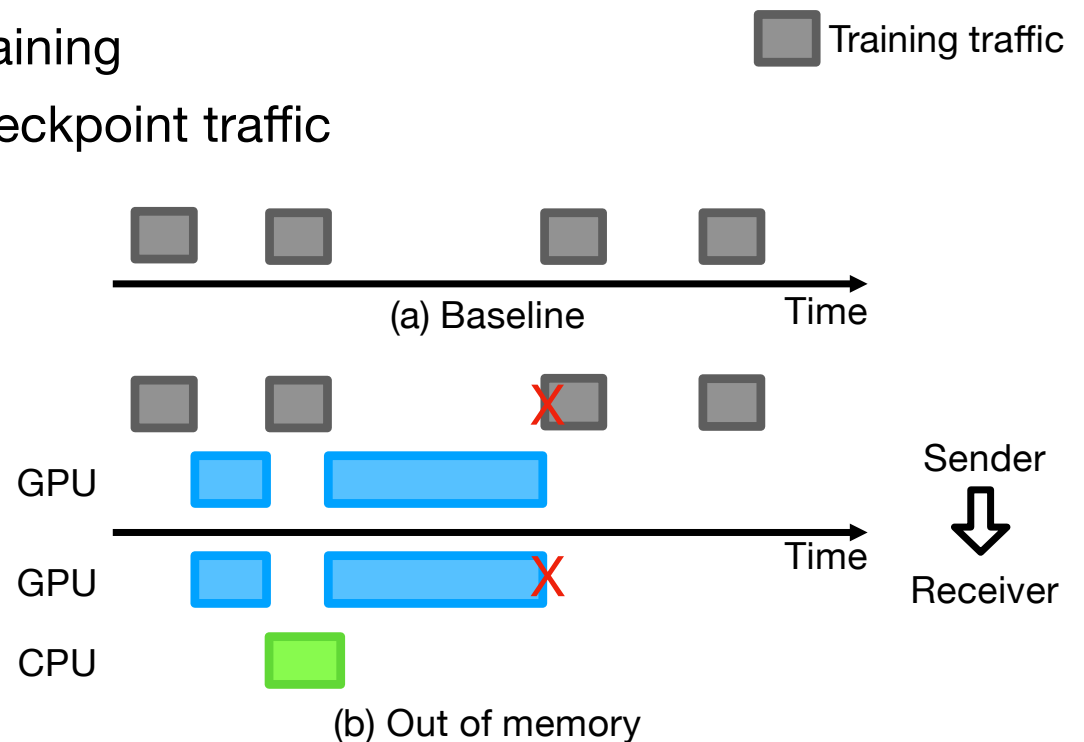
- Insert checkpoint traffic in idle timespans



Out-of-memory issue

- GPU memory is mainly used for training
- Limited spare GPU memory for checkpoint traffic

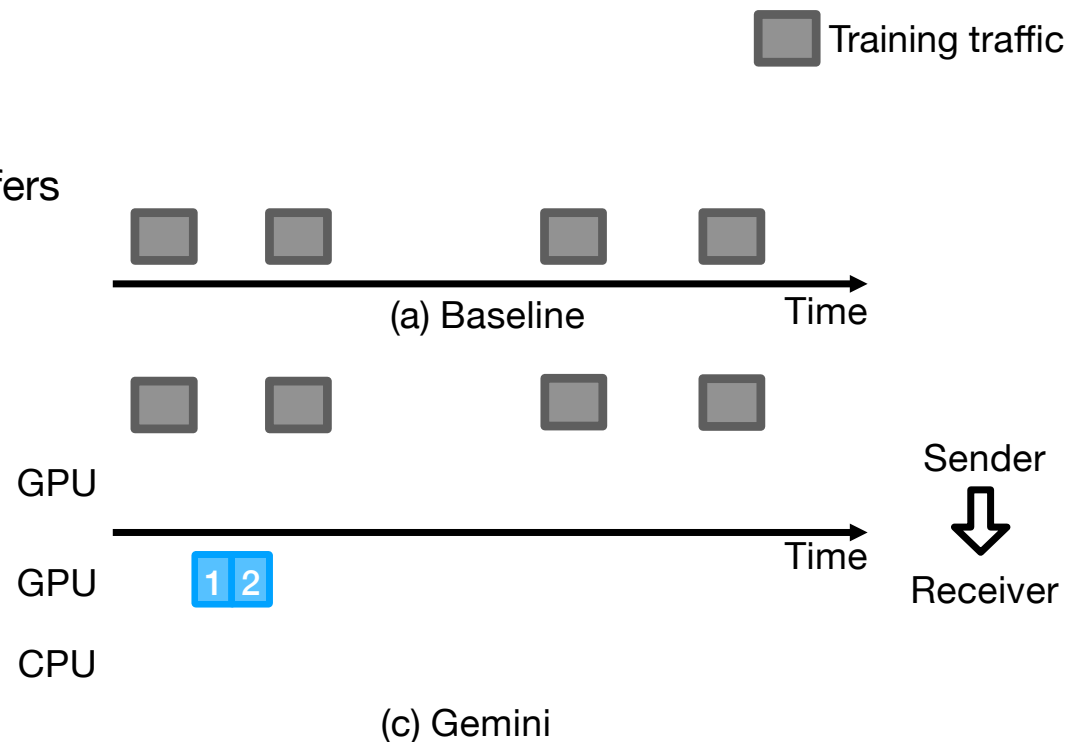
How to minimize the extra GPU memory consumption?



Our design

Checkpoint partition and pipelining

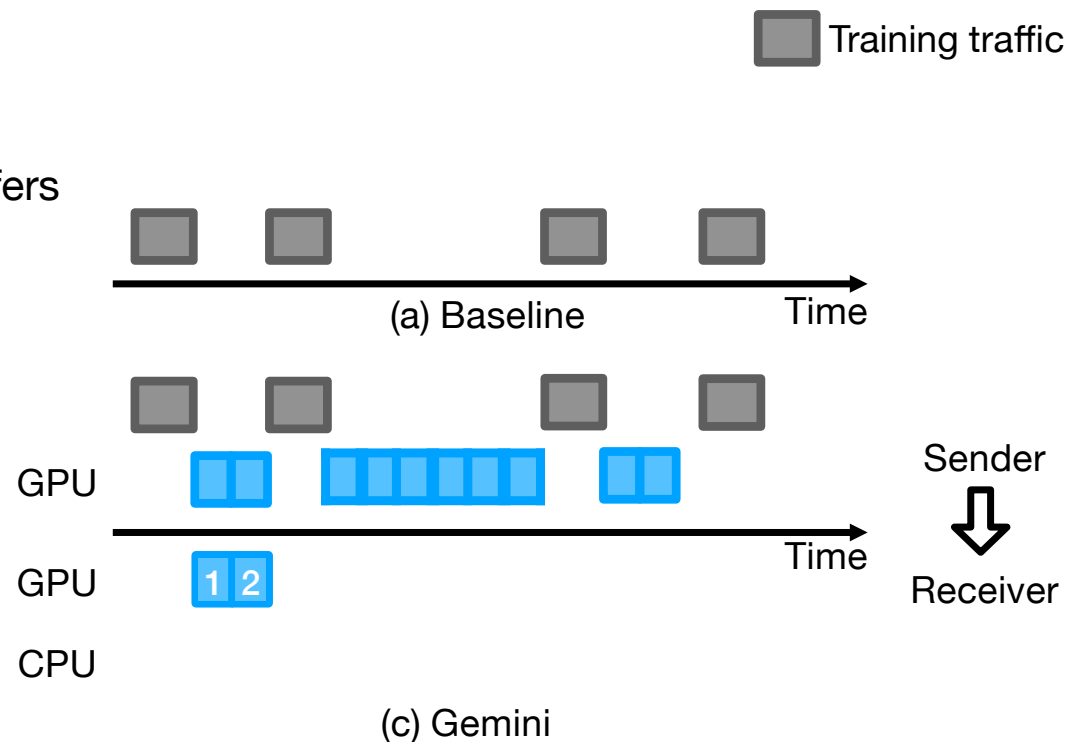
- Keys ideas
 - Reserve a GPU buffer at the receiver
 - Partition the buffer to multiple sub-buffers



Our design

Checkpoint partition and pipelining

- Keys ideas
 - Reserve a GPU buffer at the receiver
 - Partition the buffer to multiple sub-buffers
 - Pipeline checkpoint communications



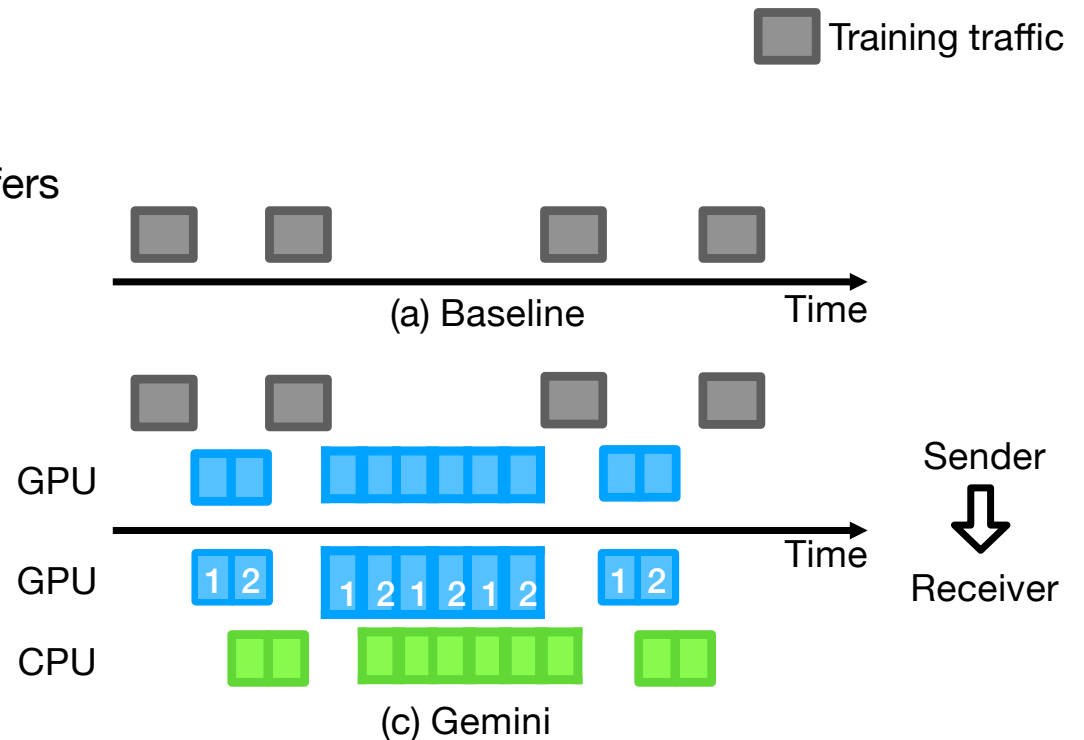
Our design

Checkpoint partition and pipelining

- Keys ideas
 - Reserve a GPU buffer at the receiver
 - Partition the buffer to multiple sub-buffers
 - Pipeline checkpoint communications

The GPU sub-buffers are reused

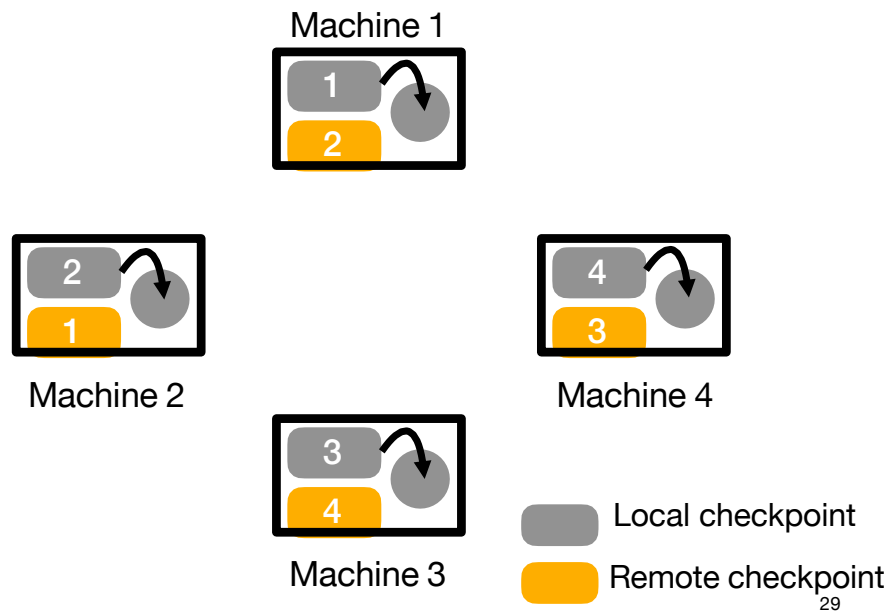
A small GPU buffer, e.g., 128MB, is sufficient



Resume training from failures

Software failures

- Checkpoints are available at local



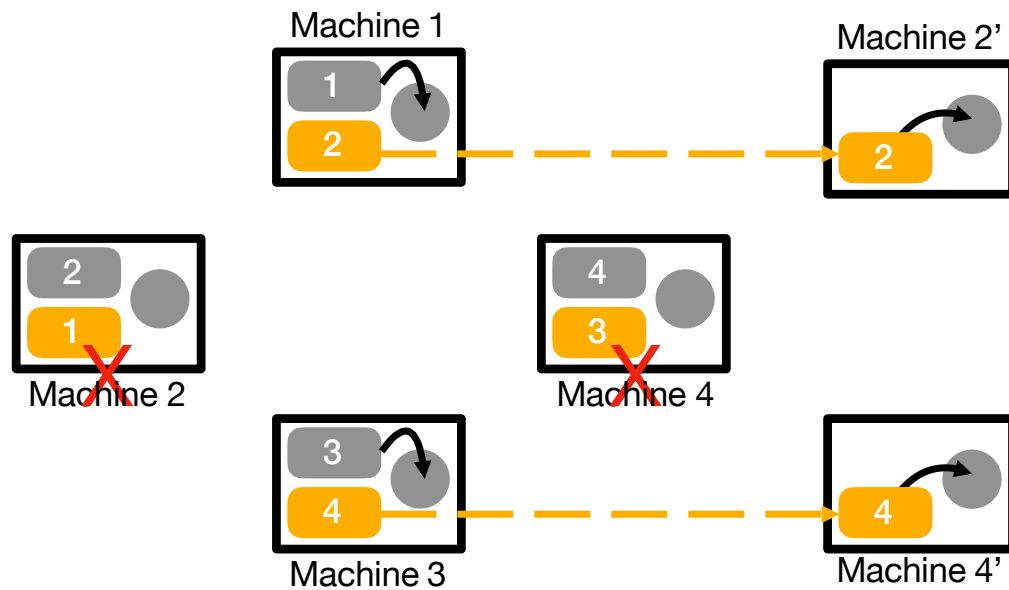
Negligible retrieval time

Just few iterations are lost

Resume training from failures

Hardware failures

- Checkpoints are still available at other machines

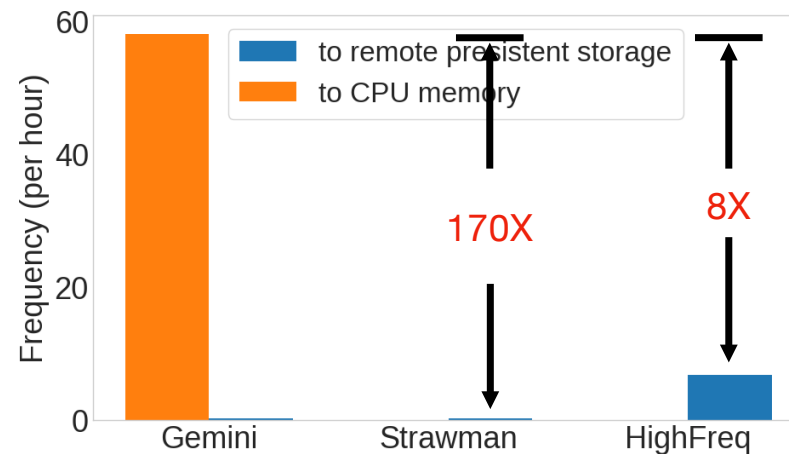


Evaluation

- Framework: DeepSpeed, ZeRO-3
- 16 p4d instances (128 A100 GPUs),
Settings
400Gbps network bandwidth The
• aggregated bandwidth of remote
storage: 20Gbps
•
- The size of LLM:
100 billion
parameters
Reserved GPU

Checkpoint frequency

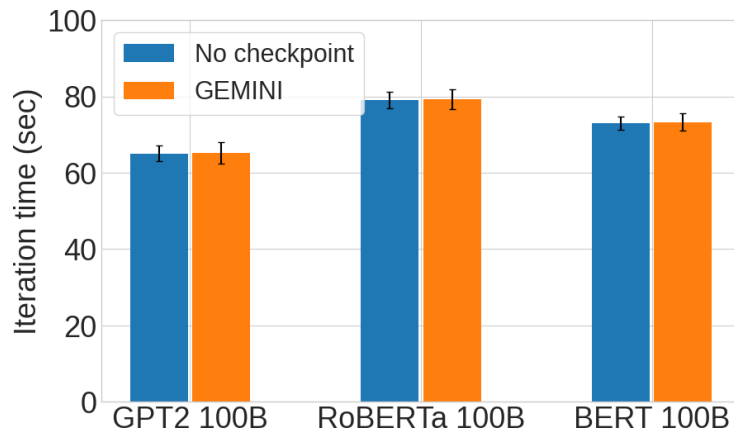
- Baselines
 - Strawman: every 3 hours (BLOOM's frequency [1])
 - HighFreq: saturate the remote storage bandwidth capacity



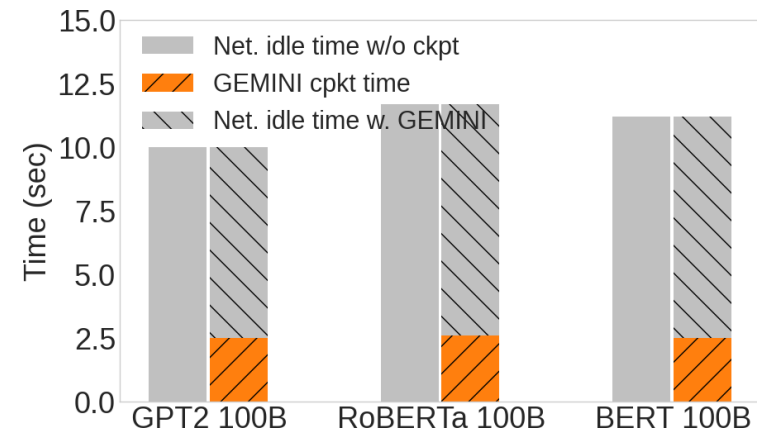
Gemini: Checkpoint model states every iteration

Training efficiency

- Training time
 - Gemini checkpoints model states to CPU memory every iteration



Negligible overhead on iteration time



Idle timespans can accommodate checkpoint traffic

Summary

- Large model training suffers from frequent failures
- Gemini checkpoints model states to CPU memory for failure recovery
 - Optimal checkpoint frequency, i.e., every iteration
 - Negligible overhead on training throughput
 - Applicable to different parallelism strategies of training

Conclusion

- Persistent systems are no longer just local file systems writing blocks to one storage device.
 - Strata: a modern file system should span heterogeneous storage media, using fast NVM for immediate updates and reorganizing data later for lower tiers.
 - Assise: in a distributed file system, client-local persistent memory can become the fast path, as long as coherence, crash consistency, and fail-over are handled correctly.
 - Gemini: for large-scale model training, persistence is no longer only about durability; it is also about fast recovery, using in-memory checkpoints and hierarchical placement to reduce restart cost.
- Big picture: the trend is from single-device local persistence to **multi-tier**, **distributed**, and **recovery-aware** persistent systems.

Thanks